

Replication Code Appendix

ADH China Shock Political Event Study Extension

Rishav Roy

Table of contents

1	00_config.R	2
2	01_helpers.R	3
3	02_build_analysis_data.R	15
4	03_estimate_event_study.R	27
5	04_export_outputs.R	35
6	05_export_diagnostic_maps.R	38
7	06_sixth_extensions.R	45
8	run_pipeline.R	57
9	county_bridge_exceptions.csv	59

This appendix prints the replication source files used to construct the final results. Files are included directly from `../src/` at render time.

1 00_config.R

```
1 # extension-adh2013/src/00_config.R
2
3 if (!requireNamespace("here", quietly = TRUE)) {
4   stop("Package 'here' is required. Install it before sourcing the pipeline config.")
5 }
6
7 REPLICATION_DIR <- here::here()
8 SRC_DIR <- here::here("src")
9 OUTPUT_DIR <- here::here("output")
10 INTERMEDIATE_DIR <- here::here("output", "intermediate")
11 TABLE_DIR <- here::here("output", "tables")
12 FIGURE_DIR <- here::here("output", "figures")
13 DIAGNOSTIC_DIR <- here::here("output", "diagnostics")
14
15 CONFIG <- list(
16   replication_dir = REPLICATION_DIR,
17   src_dir = SRC_DIR,
18   output_dir = OUTPUT_DIR,
19   intermediate_dir = INTERMEDIATE_DIR,
20   table_dir = TABLE_DIR,
21   figure_dir = FIGURE_DIR,
22   diagnostic_dir = DIAGNOSTIC_DIR,
23
24   first_election_year = 1972L,
25   diagnostic_first_election_year = 1952L,
26   last_election_year = 2020L,
27   reference_year = 1988L,
28
29   outcome_var = "rep_margin",
30   outcome_label = "Republican presidential vote margin",
31   exposure_var = "exposure",
32   exposure_label = "ADH China import exposure, 1990-2007",
33   exposure_units = "thousand dollars per worker",
34   weight_var = "adh_weight",
35
36   crosswalk_weight = "m5_weight",
37   crosswalk_missing_weight_policy = "fallback_m2",
38   renormalize_crosswalk_weights = FALSE,
39   weight_sum_tolerance = 1e-6,
40   min_retained_vote_share = 0.98,
41   max_abs_vote_identity_error = 1e-6,
42   max_missing_cz_year_share = 0,
43
44   require_balanced_panel = TRUE,
45   require_no_duplicate_county_years = TRUE,
46   require_no_vcov_repair_for_main = TRUE,
47   allow_vcov_repair = FALSE,
48   allow_failed_diagnostics = FALSE,
49
50   conley_cutoff_km = 500,
51   conley_cutoffs_km = c(250, 500, 750, 1000),
52   conley_distance = "spherical",
53   nw_lag = 1L,
54   dk_lag = 1L,
55   main_se_type = "cluster_cz",
56
57   standardize_interacted_controls = TRUE,
58   export_bandwidth_diagnostic = TRUE,
59   export_crosswalk_maps = TRUE,
60   run_crosswalk_sensitivity = FALSE,
61   run_sixth_extensions = TRUE,
62   extension_crosswalk_weight = "m5_weight",
63   extension_crosswalk_missing_weight_policy = "fallback_m2",
64   save_single_rds = FALSE,
65   rds_chunk_size_mib = 45,
66   county1990_shapefile_path = Sys.getenv("COUNTY1990_SHAPEFILE", unset = file.path(REPLICATION_DIR, "spatial-data", "counties-1990")),
67   nhgis_extract_dir = Sys.getenv("NHGIS_1990_COUNTY_EXTRACT_DIR", unset = file.path(REPLICATION_DIR, "spatial-data", "nhgis_1990_county"))
68 )
69
70 CONFIG$baseline_controls <- c(
71   "l_shind_manuf_cbp",
72   "l_sh_popedu_c",
73   "l_sh_popfborn",
74   "l_sh_empl_f",
75   "l_sh_routine33",
76   "l_task_outsource"
77 )
78
79 CONFIG$interacted_controls <- CONFIG$baseline_controls
80
81 CONFIG$core_interacted_controls <- c(
82   "l_shind_manuf_cbp",
83   "l_sh_popedu_c",
84   "l_sh_popfborn"
85 )
```

2 01_helpers.R

```
1 # extension-adh2013/src/01_helpers.R
2
3 required_packages <- function(config = NULL) {
4   pkgs <- c(
5     "dplyr", "fixest", "ggplot2", "haven", "jsonlite", "kableExtra", "knitr",
6     "purrr", "readr", "readxl", "rlang", "stringr", "tibble", "tidyr"
7   )
8   if (!is.null(config) && isTRUE(config$export_crosswalk_maps)) pkgs <- c(pkgs, "sf")
9   unique(pkgs)
10 }
11
12 optional_packages <- function() {
13   c("ipumsr")
14 }
15
16 configured_shapefile_defaults <- function(config) {
17   c(
18     file.path(config$replication_dir, "spatial-data", "counties-1990"),
19     file.path(config$replication_dir, "spatial-data", "nhgis_1990_county")
20   )
21 }
22
23 load_required_packages <- function(config = NULL) {
24   pkgs <- required_packages(config)
25   missing <- pkgs[!vapply(pkgs, requireNamespace, FUN.VALUE = logical(1), quietly = TRUE)]
26   if (length(missing) > 0) {
27     stop(
28       "Missing required R packages: ", paste(missing, collapse = ", "),
29       ". Install them before running the pipeline, or restore the project dependency manifest.",
30       call. = FALSE
31     )
32   }
33   invisible(lapply(pkgs, library, character.only = TRUE))
34   options(dplyr.summarise.inform = FALSE)
35 }
36
37 ensure_output_dirs <- function(config) {
38   dirs <- c(config$output_dir, config$intermediate_dir, config$stable_dir,
39     config$figure_dir, config$diagnostic_dir)
40   invisible(lapply(dirs, dir.create, recursive = TRUE, showWarnings = FALSE))
41 }
42
43 weight_slug <- function(weight_col) {
44   stringr::str_replace(weight_col, "_weight$", "")
45 }
46
47 finalize_config <- function(config) {
48   allowed_weights <- c("m2_weight", "m4_weight", "m5_weight", "m6_weight")
49   if (!config$crosswalk_weight %in% allowed_weights) {
50     stop("CONFIG$crosswalk_weight must be one of: ", paste(allowed_weights, collapse = ", "))
51   }
52
53   allowed_policies <- c("fail", "fallback_m4", "fallback_m2", "identity_if_same_fips")
54   if (!config$crosswalk_missing_weight_policy %in% allowed_policies) {
55     stop(
56       "CONFIG$crosswalk_missing_weight_policy must be one of: ",
57       paste(allowed_policies, collapse = ", ")
58     )
59   }
60
61   if (is.null(config$interacted_controls)) {
62     config$interacted_controls <- config$baseline_controls
63   }
64   missing_interacted <- setdiff(config$interacted_controls, config$baseline_controls)
65   if (length(missing_interacted) > 0) {
66     stop("CONFIG$interacted_controls must be drawn from CONFIG$baseline_controls. Invalid: ",
67       paste(missing_interacted, collapse = ", "))
68   }
69
70   slug <- weight_slug(config$crosswalk_weight)
71   config$crosswalk_weight_slug <- slug
72   config$event_years <- seq(config$first_election_year, config$last_election_year, by = 4)
73   config$diagnostic_event_years <- seq(config$diagnostic_first_election_year, config$last_election_year, by = 4)
74   config$source_decades <- sort(unique(year_to_source_decade(config$diagnostic_event_years)))
75   if (is.null(config$main_se_type) || !nzchar(config$main_se_type)) {
76     config$main_se_type <- paste0("conley_", config$conley_cutoff_km, "km")
77   }
78
79   # Treat common placeholder strings as unset. This avoids silently skipping maps
80   # when COUNTY1990_SHAPEFILE was previously set to something like
81   # "path/to/nhgis_1990_county.shp".
82   if (is.null(config$county1990_shapefile_path) ||
83     !nzchar(config$county1990_shapefile_path) ||
84     grepl("^path/to", config$county1990_shapefile_path, ignore.case = TRUE)) {
85     config$county1990_shapefile_path <- file.path(config$replication_dir, "spatial-data", "counties-1990")
86   }
87   if (is.null(config$nhgis_extract_dir) || !nzchar(config$nhgis_extract_dir)) {
```

```

88   config$nhgis_extract_dir <- file.path(config$replication_dir, "spatial-data", "nhgis_1990_county")
89 }
90
91 config$analysis_panel_rds <- file.path(
92   config$intermediate_dir, paste0("analysis_panel_aa2021_rep_margin_", slug, ".rds")
93 )
94 config$analysis_panel_full_rds <- file.path(
95   config$intermediate_dir, paste0("analysis_panel_full_aa2021_rep_margin_", slug, ".rds")
96 )
97 config$diagnostic_panel_rds <- file.path(
98   config$intermediate_dir, paste0("analysis_panel_aa2021_rep_margin_1952start_", slug, ".rds")
99 )
100 config$diagnostic_panel_full_rds <- file.path(
101   config$intermediate_dir, paste0("analysis_panel_full_aa2021_rep_margin_1952start_", slug, ".rds")
102 )
103 config$cz_centroids_csv <- file.path(
104   config$intermediate_dir, "cz_centroids_1990_population_weighted.csv"
105 )
106 config$event_study_rds <- file.path(
107   config$intermediate_dir, paste0("event_study_results_aa2021_rep_margin_", slug, ".rds")
108 )
109 config$event_study_csv <- file.path(
110   config$intermediate_dir, paste0("event_study_coefficients_aa2021_rep_margin_", slug, ".csv")
111 )
112 config$all_specs_rds <- file.path(
113   config$intermediate_dir, paste0("event_study_results_all_specs_aa2021_rep_margin_", slug, ".rds")
114 )
115 config$all_specs_csv <- file.path(
116   config$intermediate_dir, paste0("event_study_coefficients_all_specs_aa2021_rep_margin_", slug, ".csv")
117 )
118
119 config$dependency_manifest_json <- file.path(config$diagnostic_dir, "dependency_manifest.json")
120 config$pipeline_manifest_json <- file.path(config$diagnostic_dir, "pipeline_manifest.json")
121 config$output_manifest_csv <- file.path(config$diagnostic_dir, "output_manifest.csv")
122 config$output_manifest_json <- file.path(config$diagnostic_dir, "output_manifest.json")
123 config$validation_checks_csv <- file.path(config$diagnostic_dir, "pipeline_validation_checks.csv")
124 config$crosswalk_diagnostics_csv <- file.path(
125   config$diagnostic_dir, paste0("ftz_", slug, "_crosswalk_source_diagnostics.csv")
126 )
127 config$crosswalk_zero_support_csv <- file.path(
128   config$diagnostic_dir, paste0("ftz_", slug, "_undefined_or_zero_support_counties.csv")
129 )
130 config$crosswalk_row_diagnostics_csv <- file.path(
131   config$diagnostic_dir, paste0("ftz_", slug, "_crosswalk_row_diagnostics.csv")
132 )
133 config$crosswalk_issue_vote_report_csv <- file.path(
134   config$diagnostic_dir, paste0("ftz_", slug, "_crosswalk_issue_vote_report.csv")
135 )
136 config$crosswalk_preflight_summary_csv <- file.path(
137   config$diagnostic_dir, "ftz_m5_m6_zero_support_preflight_summary.csv"
138 )
139 config$crosswalk_preflight_counties_csv <- file.path(
140   config$diagnostic_dir, "ftz_m5_m6_zero_support_preflight_counties.csv"
141 )
142 config$all_weight_support_csv <- file.path(
143   config$diagnostic_dir, "ftz_all_weight_support_by_source_county.csv"
144 )
145 config$target_weight_support_county_csv <- file.path(
146   config$diagnostic_dir, "ftz_weight_support_by_1990_county.csv"
147 )
148 config$target_weight_support_cz_csv <- file.path(
149   config$diagnostic_dir, "ftz_weight_support_by_cz.csv"
150 )
151 config$county_bridge_classification_csv <- file.path(
152   config$diagnostic_dir, paste0("unmatched_source_county_classification_", slug, ".csv")
153 )
154 config$mainland_retention_csv <- file.path(
155   config$diagnostic_dir, paste0("county_bridge_retention_by_scope_", slug, ".csv")
156 )
157 config$baseline_control_summary_csv <- file.path(
158   config$diagnostic_dir, "baseline_control_summary.csv"
159 )
160 config$baseline_control_correlation_csv <- file.path(
161   config$diagnostic_dir, "baseline_control_correlation_matrix.csv"
162 )
163 config$interacted_control_variance_csv <- file.path(
164   config$diagnostic_dir, "interacted_control_variance_diagnostics.csv"
165 )
166 config$conley_neighbor_counts_csv <- file.path(
167   config$diagnostic_dir, "conley_neighbor_count_by_cutoff.csv"
168 )
169 config$vcov_rank_csv <- file.path(
170   config$diagnostic_dir, "event_study_vcov_rank_by_se_type.csv"
171 )
172 config$se_warning_diagnostics_csv <- file.path(
173   config$diagnostic_dir, "event_study_se_warning_diagnostics.csv"
174 )
175 config$crosswalk_comparison_csv <- file.path(
176   config$diagnostic_dir, "crosswalk_specification_comparison_coefficients.csv"
177 )

```

```

178 config$crosswalk_comparison_png <- file.path(
179   config$figure_dir, "fig_crosswalk_specification_comparison.png"
180 )
181 config$crosswalk_comparison_pdf <- file.path(
182   config$figure_dir, "fig_crosswalk_specification_comparison.pdf"
183 )
184 config$crosswalk_comparison_delta_csv <- file.path(
185   config$diagnostic_dir, "crosswalk_specification_comparison_deltas_vs_pure_m2.csv"
186 )
187 config$crosswalk_comparison_delta_png <- file.path(
188   config$figure_dir, "fig_crosswalk_specification_comparison_deltas_vs_pure_m2.png"
189 )
190 config$crosswalk_comparison_delta_pdf <- file.path(
191   config$figure_dir, "fig_crosswalk_specification_comparison_deltas_vs_pure_m2.pdf"
192 )
193 config$crosswalk_support_county_map_png <- file.path(
194   config$figure_dir, "fig_crosswalk_unavailable_weights_1990_counties.png"
195 )
196 config$crosswalk_support_county_map_pdf <- file.path(
197   config$figure_dir, "fig_crosswalk_unavailable_weights_1990_counties.pdf"
198 )
199 config$crosswalk_support_cz_map_png <- file.path(
200   config$figure_dir, "fig_crosswalk_unavailable_weights_1990_czs.png"
201 )
202 config$crosswalk_support_cz_map_pdf <- file.path(
203   config$figure_dir, "fig_crosswalk_unavailable_weights_1990_czs.pdf"
204 )
205 config$crosswalk_support_county_map_simplified_png <- file.path(
206   config$figure_dir, "fig_crosswalk_unavailable_weights_1990_counties_simplified.png"
207 )
208 config$crosswalk_support_county_map_small_png <- file.path(
209   config$figure_dir, "fig_crosswalk_unavailable_weights_1990_counties_small.png"
210 )
211 config$crosswalk_support_county_map_simplified_pdf <- file.path(
212   config$figure_dir, "fig_crosswalk_unavailable_weights_1990_counties_simplified.pdf"
213 )
214 config$crosswalk_support_cz_map_simplified_png <- file.path(
215   config$figure_dir, "fig_crosswalk_unavailable_weights_1990_czs_simplified.png"
216 )
217 config$crosswalk_support_cz_map_small_png <- file.path(
218   config$figure_dir, "fig_crosswalk_unavailable_weights_1990_czs_small.png"
219 )
220 config$crosswalk_support_cz_map_simplified_pdf <- file.path(
221   config$figure_dir, "fig_crosswalk_unavailable_weights_1990_czs_simplified.pdf"
222 )
223 config$outcome_duplicate_report_csv <- file.path(
224   config$diagnostic_dir, "aa2021_presidential_duplicate_county_years.csv"
225 )
226 config$outcome_validation_csv <- file.path(
227   config$diagnostic_dir, "aa2021_presidential_county_year_validation.csv"
228 )
229 config$national_vote_totals_csv <- file.path(
230   config$diagnostic_dir, "aa2021_presidential_national_vote_totals.csv"
231 )
232 config$state_vote_totals_csv <- file.path(
233   config$diagnostic_dir, "aa2021_presidential_state_vote_totals.csv"
234 )
235 config$bridge_diagnostics_csv <- file.path(
236   config$diagnostic_dir, paste0("county_bridge_diagnostics_by_year_", slug, ".csv")
237 )
238 config$unmatched_source_counties_csv <- file.path(
239   config$diagnostic_dir, paste0("unmatched_source_counties_", slug, ".csv")
240 )
241 config$missing_cz_years_csv <- file.path(
242   config$diagnostic_dir, paste0("missing_cz_year_outcomes_", slug, ".csv")
243 )
244 config$panel_coverage_csv <- file.path(
245   config$diagnostic_dir, paste0("panel_coverage_by_year_", slug, ".csv")
246 )
247 config$spec_status_csv <- file.path(config$diagnostic_dir, "event_study_spec_status.csv")
248 config$model_diagnostics_csv <- file.path(config$diagnostic_dir, "event_study_model_diagnostics.csv")
249 config$controls_manifest_csv <- file.path(config$diagnostic_dir, "event_study_controls_manifest.csv")
250 config$vcov_diagnostics_csv <- file.path(config$diagnostic_dir, "event_study_vcov_diagnostics.csv")
251 config$vcov_eigenvalues_csv <- file.path(config$diagnostic_dir, "event_study_vcov_eigenvalues.csv")
252 config$pretrend_tests_csv <- file.path(config$diagnostic_dir, "event_study_pretrend_tests.csv")
253 config$pretrend_coefficients_csv <- file.path(config$diagnostic_dir, "event_study_pretrend_coefficients.csv")
254
255 config$table_tex <- file.path(config$table_dir, paste0("tab_event_study_rep_margin_main_specs_", slug, ".tex"))
256 config$table_csv <- file.path(config$table_dir, paste0("tab_event_study_rep_margin_main_specs_", slug, ".csv"))
257 config$table_pdf <- file.path(config$table_dir, paste0("tab_event_study_rep_margin_main_specs_", slug, ".pdf"))
258 config$table_standalone_tex <- file.path(
259   config$table_dir, paste0("tab_event_study_rep_margin_main_specs_", slug, "_standalone.tex")
260 )
261 config$figure_pdf <- file.path(config$figure_dir, paste0("fig_event_study_rep_margin_main_specs_", slug, ".pdf"))
262 config$figure_png <- file.path(config$figure_dir, paste0("fig_event_study_rep_margin_main_specs_", slug, ".png"))
263
264 # Sixth-section extension outputs. These are intentionally keyed to the
265 # primary crosswalk slug when relevant, but are otherwise single-project
266 # diagnostics that should not be re-estimated separately for every
267 # crosswalk-sensitivity specification.

```

```

268 config$bartik_first_stage_csv <- file.path(config$diagnostic_dir, "bartik_first_stage_diagnostics.csv")
269 config$bartik_balance_csv <- file.path(config$diagnostic_dir, "bartik_balance_correlations.csv")
270 config$bartik_pretrend_csv <- file.path(config$diagnostic_dir, "bartik_pretrend_placebos.csv")
271 config$bartik_data_availability_csv <- file.path(config$diagnostic_dir, "bartik_rotemberg_data_availability.csv")
272 config$bartik_identification_memo_md <- file.path(config$diagnostic_dir, "bartik_identification_memo.md")
273 config$bartik_first_stage_plot_png <- file.path(config$figure_dir, "fig_bartik_first_stage.png")
274 config$bartik_first_stage_plot_pdf <- file.path(config$figure_dir, "fig_bartik_first_stage.pdf")
275 config$bartik_industry_shift_summary_csv <- file.path(config$diagnostic_dir, "bartik_industry_shift_summary.csv")
276 config$bartik_preperiod_workfile_csv <- file.path(config$diagnostic_dir, "bartik_preperiod_workfile_diagnostics.csv")
277
278 config$alternative_outcomes_coefficients_csv <- file.path(config$diagnostic_dir, "alternative_political_outcome_event_studies.csv")
279 config$alternative_outcomes_summary_csv <- file.path(config$diagnostic_dir, "alternative_political_outcome_summary.csv")
280 config$alternative_outcomes_plot_png <- file.path(config$figure_dir, "fig_alternative_political_outcomes.png")
281 config$alternative_outcomes_plot_pdf <- file.path(config$figure_dir, "fig_alternative_political_outcomes.pdf")
282 config$alternative_outcomes_status_csv <- file.path(config$diagnostic_dir, "alternative_political_outcome_status.csv")
283 config$alternative_outcomes_decomposition_plot_png <- file.path(config$figure_dir,
  ↳ "fig_alternative_political_outcome_decomposition.png")
284 config$alternative_outcomes_decomposition_plot_pdf <- file.path(config$figure_dir,
  ↳ "fig_alternative_political_outcome_decomposition.pdf")
285 config$alternative_outcomes_vote_choice_plot_png <- file.path(config$figure_dir, "fig_alternative_political_outcomes_vote_choice.png")
286 config$alternative_outcomes_vote_choice_plot_pdf <- file.path(config$figure_dir, "fig_alternative_political_outcomes_vote_choice.pdf")
287 config$alternative_outcomes_vote_volume_plot_png <- file.path(config$figure_dir, "fig_alternative_political_outcomes_vote_volume.png")
288 config$alternative_outcomes_vote_volume_plot_pdf <- file.path(config$figure_dir, "fig_alternative_political_outcomes_vote_volume.pdf")
289
290 config$nanda_cz_panel_csv <- file.path(config$intermediate_dir, "nanda_2004_2022_cz_panel.csv")
291 config$nanda_outcomes_coefficients_csv <- file.path(config$diagnostic_dir, "nanda_outcome_event_studies.csv")
292 config$nanda_outcomes_summary_csv <- file.path(config$diagnostic_dir, "nanda_outcome_summary.csv")
293 config$nanda_outcomes_plot_png <- file.path(config$figure_dir, "fig_nanda_outcome_event_studies.png")
294 config$nanda_outcomes_plot_pdf <- file.path(config$figure_dir, "fig_nanda_outcome_event_studies.pdf")
295 config$nanda_outcomes_status_csv <- file.path(config$diagnostic_dir, "nanda_outcome_event_study_status.csv")
296 config$nanda_county_validation_csv <- file.path(config$diagnostic_dir, "nanda_county_year_validation.csv")
297 config$nanda_cz_validation_csv <- file.path(config$diagnostic_dir, "nanda_cz_year_validation.csv")
298 config$nanda_outlier_rates_csv <- file.path(config$diagnostic_dir, "nanda_outlier_rates.csv")
299 config$nanda_outcome_availability_csv <- file.path(config$diagnostic_dir, "nanda_outcome_year_availability.csv")
300 config$nanda_turnout_plot_png <- file.path(config$figure_dir, "fig_nanda_turnout_registration.png")
301 config$nanda_turnout_plot_pdf <- file.path(config$figure_dir, "fig_nanda_turnout_registration.pdf")
302 config$nanda_vote_plot_png <- file.path(config$figure_dir, "fig_nanda_presidential_vote_ratios.png")
303 config$nanda_vote_plot_pdf <- file.path(config$figure_dir, "fig_nanda_presidential_vote_ratios.pdf")
304 config$nanda_partisanship_plot_png <- file.path(config$figure_dir, "fig_nanda_partisanship_indices.png")
305 config$nanda_partisanship_plot_pdf <- file.path(config$figure_dir, "fig_nanda_partisanship_indices.pdf")
306
307 config$adhm2020_summary_csv <- file.path(config$diagnostic_dir, "adhm2020_public_data_summary.csv")
308 config$adhm2020_regressions_csv <- file.path(config$diagnostic_dir, "adhm2020_mechanism_regressions.csv")
309 config$adhm2020_outcome_dictionary_csv <- file.path(config$diagnostic_dir, "adhm2020_outcome_dictionary.csv")
310 config$adhm2020_status_csv <- file.path(config$diagnostic_dir, "adhm2020_mechanism_regression_status.csv")
311 config$adhm2020_heterogeneity_availability_csv <- file.path(config$diagnostic_dir, "adhm2020_heterogeneity_data_availability.csv")
312 config$adhm2020_plot_png <- file.path(config$figure_dir, "fig_adhm2020_mechanism_regressions.png")
313 config$adhm2020_plot_pdf <- file.path(config$figure_dir, "fig_adhm2020_mechanism_regressions.pdf")
314
315 config$subperiod_exposure_csv <- file.path(config$intermediate_dir, "adh_subperiod_exposure_by_cz.csv")
316 config$subperiod_exposure_summary_csv <- file.path(config$diagnostic_dir, "adh_subperiod_exposure_summary.csv")
317 config$subperiod_event_study_coefficients_csv <- file.path(config$diagnostic_dir, "subperiod_exposure_event_study_coefficients.csv")
318 config$subperiod_event_study_status_csv <- file.path(config$diagnostic_dir, "subperiod_exposure_event_study_status.csv")
319 config$subperiod_event_study_plot_png <- file.path(config$figure_dir, "fig_subperiod_exposure_event_study.png")
320 config$subperiod_event_study_plot_pdf <- file.path(config$figure_dir, "fig_subperiod_exposure_event_study.pdf")
321 config$subperiod_first_stage_csv <- file.path(config$diagnostic_dir, "subperiod_exposure_first_stage.csv")
322 config$subperiod_equality_tests_csv <- file.path(config$diagnostic_dir, "subperiod_exposure_equality_tests.csv")
323 config$subperiod_selected_horizons_csv <- file.path(config$diagnostic_dir, "subperiod_exposure_selected_horizons.csv")
324
325 config
326 }
327
328 fips_to_gisjoin <- function(fips) {
329   fips <- stringr::str_pad(as.character(fips), width = 5, side = "left", pad = "0")
330   paste0("G", substr(fips, 1, 2), "0", substr(fips, 3, 5), "0")
331 }
332
333 gisjoin_to_fips <- function(gisjoin) {
334   gisjoin <- as.character(gisjoin)
335   out <- ifelse(is.na(gisjoin), NA_character_, paste0(substr(gisjoin, 2, 3), substr(gisjoin, 5, 7)))
336   stringr::str_pad(out, width = 5, side = "left", pad = "0")
337 }
338
339 year_to_source_decade <- function(year) {
340   dplyr::case_when(
341     year < 1980 ~ 1970L,
342     year < 1990 ~ 1980L,
343     year < 2000 ~ 1990L,
344     year < 2010 ~ 2000L,
345     year < 2020 ~ 2010L,
346     TRUE ~ 2020L
347   )
348 }
349
350 read_aa_rdata_object <- function(path, object_name = NULL) {
351   e <- new.env(parent = emptyenv())
352   loaded <- load(path, envir = e)
353   if (!is.null(object_name)) return(e[[object_name]])
354   candidates <- setdiff(loaded, ".Random.seed")
355   if (length(candidates) != 1) stop("Expected exactly one non-.Random.seed object in: ", path)

```

```

356   e[[candidates]]
357 }
358
359 sum_preserve_all_na <- function(x) {
360   if (length(x) == 0 || all(is.na(x))) NA_real_ else sum(x, na.rm = TRUE)
361 }
362
363 read_ftz_crosswalk <- function(source_decade, zip_path, county_fips_1990_identity = NULL,
364                               weight_col = "m5_weight",
365                               missing_weight_policy = "fail",
366                               renormalize_weights = FALSE,
367                               tolerance = 1e-8) {
368   if (source_decade == 1990L) {
369     ids <- unique(as.character(county_fips_1990_identity))
370     return(tibble::tibble(
371       source_decade = 1990L,
372       gisjoin_source = fips_to_gisjoin(ids),
373       gisjoin_1990 = fips_to_gisjoin(ids),
374       county_fips_source = stringr::str_pad(ids, 5, pad = "0"),
375       county_fips_1990 = stringr::str_pad(ids, 5, pad = "0"),
376       raw_weight = 1,
377       fallback_m4_weight = 1,
378       fallback_m2_weight = 1,
379       raw_weight_missing_parts = 0L,
380       raw_weight_parts = 1L,
381       weight_sum_raw = 1,
382       selected_weight_sum_raw = 1,
383       crosswalk_weight = 1,
384       crosswalk_weight_sum = 1,
385       raw_weight_undefined = FALSE,
386       raw_weight_sum_valid = TRUE,
387       raw_weight_requires_policy = FALSE,
388       fallback_used = FALSE,
389       fallback_policy = "identity_1990",
390       weight_source = "identity_1990"
391     ))
392   }
393
394   file_name <- paste0("Crosswalk_", source_decade, "_1990.csv")
395   source_col <- paste0("gisjoin_", source_decade)
396   raw <- readr::read_csv(unz(zip_path, file_name), show_col_types = FALSE)
397   required <- c(source_col, "gisjoin_1990", weight_col, "m4_weight", "m2_weight")
398   missing <- setdiff(required, names(raw))
399   if (length(missing) > 0) {
400     stop("Missing required FTZ columns in ", file_name, ": ", paste(missing, collapse = ", "))
401   }
402
403   parts <- raw %>%
404     dplyr::transmute(
405       source_decade = as.integer(source_decade),
406       gisjoin_source = as.character(.data[[source_col]]),
407       gisjoin_1990 = as.character(.data[["gisjoin_1990"]]),
408       raw_weight_part = as.numeric(.data[[weight_col]]),
409       fallback_m4_part = as.numeric(.data[["m4_weight"]]),
410       fallback_m2_part = as.numeric(.data[["m2_weight"]])
411     ) %>%
412     dplyr::mutate(
413       gisjoin_source = dplyr::if_else(
414         source_decade == 2020L,
415         fips_to_gisjoin(stringr::str_pad(gisjoin_source, 5, pad = "0")),
416         gisjoin_source
417       )
418     )
419
420   out <- parts %>%
421     dplyr::group_by(source_decade, gisjoin_source, gisjoin_1990) %>%
422     dplyr::summarise(
423       raw_weight = sum_preserve_all_na(raw_weight_part),
424       fallback_m4_weight = sum_preserve_all_na(fallback_m4_part),
425       fallback_m2_weight = sum_preserve_all_na(fallback_m2_part),
426       raw_weight_missing_parts = sum(is.na(raw_weight_part)),
427       raw_weight_parts = dplyr::n(),
428       .groups = "drop"
429     ) %>%
430     dplyr::mutate(
431       county_fips_source = gisjoin_to_fips(gisjoin_source),
432       county_fips_1990 = gisjoin_to_fips(gisjoin_1990)
433     ) %>%
434     dplyr::group_by(source_decade, gisjoin_source) %>%
435     dplyr::mutate(
436       weight_sum_raw = if (all(is.na(raw_weight))) NA_real_ else sum(raw_weight, na.rm = TRUE),
437       raw_weight_undefined = is.na(weight_sum_raw) | weight_sum_raw <= tolerance,
438       raw_weight_sum_valid = !is.na(weight_sum_raw) & abs(weight_sum_raw - 1) <= tolerance,
439       raw_weight_requires_policy = !raw_weight_sum_valid
440     ) %>%
441     dplyr::ungroup() %>%
442     dplyr::mutate(
443       selected_raw_weight = dplyr::case_when(
444         !raw_weight_requires_policy ~ raw_weight,
445         missing_weight_policy == "fallback_m4" ~ fallback_m4_weight,

```

```

446     missing_weight_policy == "fallback_m2" ~ fallback_m2_weight,
447     missing_weight_policy == "identity_if_same_fips" & county_fips_source == county_fips_1990 ~ 1,
448     missing_weight_policy == "identity_if_same_fips" ~ 0,
449     TRUE ~ NA_real_
450   ),
451   fallback_used = raw_weight_requires_policy & missing_weight_policy != "fail",
452   fallback_policy = dplyr::case_when(
453     !fallback_used ~ "none",
454     missing_weight_policy == "fallback_m4" ~ "fallback_m4",
455     missing_weight_policy == "fallback_m2" ~ "fallback_m2",
456     missing_weight_policy == "identity_if_same_fips" ~ "identity_if_same_fips",
457     TRUE ~ missing_weight_policy
458   ),
459   weight_source = dplyr::case_when(
460     !fallback_used ~ weight_col,
461     fallback_policy == "fallback_m4" ~ "m4_weight",
462     fallback_policy == "fallback_m2" ~ "m2_weight",
463     fallback_policy == "identity_if_same_fips" ~ "identity_if_same_fips",
464     TRUE ~ weight_col
465   )
466 ) %>%
467 dplyr::group_by(source_decade, gisjoin_source) %>%
468 dplyr::mutate(
469   selected_weight_sum_raw = if (all(is.na(selected_raw_weight))) {
470     NA_real_
471   } else {
472     sum(selected_raw_weight, na.rm = TRUE)
473   },
474   crosswalk_weight = dplyr::if_else(
475     !is.na(selected_weight_sum_raw) & selected_weight_sum_raw > tolerance,
476     if (renormalize_weights) selected_raw_weight / selected_weight_sum_raw else selected_raw_weight,
477     NA_real_
478   ),
479   crosswalk_weight_sum = if (all(is.na(crosswalk_weight))) NA_real_ else sum(crosswalk_weight, na.rm = TRUE)
480 ) %>%
481 dplyr::ungroup() %>%
482 dplyr::select(
483   source_decade, gisjoin_source, gisjoin_1990, county_fips_source, county_fips_1990,
484   raw_weight, fallback_m4_weight, fallback_m2_weight, raw_weight_missing_parts,
485   raw_weight_parts, weight_sum_raw, selected_weight_sum_raw, crosswalk_weight,
486   crosswalk_weight_sum, raw_weight_undefined, raw_weight_sum_valid,
487   raw_weight_requires_policy, fallback_used, fallback_policy, weight_source
488 )
489
490 out
491 }
492
493 fmt_num <- function(x, digits = 3) {
494   ifelse(is.na(x), "", formatC(x, digits = digits, format = "f"))
495 }
496
497 star_code <- function(p) {
498   dplyr::case_when(
499     is.na(p) ~ "",
500     p < 0.01 ~ "****",
501     p < 0.05 ~ "***",
502     p < 0.10 ~ "**",
503     TRUE ~ ""
504   )
505 }
506
507 make_interacted_control_vars <- function(data, controls, years, ref_year) {
508   years_est <- setdiff(sort(unique(years)), ref_year)
509   new_names <- character(0)
510   for (ctrl in controls) {
511     for (yr in years_est) {
512       nm <- paste0("ctrl_", ctrl, "_", yr)
513       data[[nm]] <- data[[ctrl]] * as.integer(data$year == yr)
514       new_names <- c(new_names, nm)
515     }
516   }
517   list(data = data, vars = new_names)
518 }
519
520 validation_check <- function(name, passed, fatal = TRUE, n_failed = NA_integer_, details = "") {
521   tibble::tibble(
522     check = name,
523     status = if (isTRUE(passed)) "pass" else "fail",
524     fatal = isTRUE(fatal),
525     n_failed = n_failed,
526     details = as.character(details)
527   )
528 }
529
530 reported_check <- function(name, n_reported = 0L, details = "") {
531   tibble::tibble(
532     check = name,
533     status = if (is.na(n_reported) || n_reported == 0L) "pass" else "reported",
534     fatal = FALSE,
535     n_failed = 0L,

```



```

536     n_reported = as.integer(dplyr::coalesce(as.integer(n_reported), 0L)),
537     details = as.character(details)
538   )
539 }
540
541 has_fatal_failures <- function(checks) {
542   if (nrow(checks) == 0 || !("status" %in% names(checks))) return(FALSE)
543   any(checks$fatal %in% TRUE & checks$status == "fail", na.rm = TRUE)
544 }
545
546 has_any_failures <- function(checks) {
547   if (nrow(checks) == 0 || !("status" %in% names(checks))) return(FALSE)
548   any(checks$status == "fail", na.rm = TRUE)
549 }
550
551 has_reported_conditions <- function(checks) {
552   if (nrow(checks) == 0 || !("status" %in% names(checks))) return(FALSE)
553   any(checks$status == "reported", na.rm = TRUE)
554 }
555
556 write_dependency_manifest <- function(config) {
557   core <- required_packages(config)
558   optional <- optional_packages()
559   pkgs <- unique(c(core, optional))
560   dep <- tibble::tibble(
561     package = pkgs,
562     dependency_type = dplyr::case_when(
563       package %in% core ~ "core",
564       package %in% optional ~ "optional",
565       TRUE ~ "other"
566     ),
567     installed = vapply(pkgs, requireNamespace, FUN.VALUE = logical(1), quietly = TRUE),
568     version = vapply(pkgs, function(pkg) {
569       if (!requireNamespace(pkg, quietly = TRUE)) return(NA_character_)
570       as.character(utils::packageVersion(pkg))
571     }, character(1))
572   )
573   jsonlite::write_json(
574     list(
575       generated_at = format(Sys.time(), "%Y-%m-%d %H:%M:%S %Z"),
576       r_version = R.version.string,
577       packages = dep
578     ),
579     config$dependency_manifest_json,
580     pretty = TRUE,
581     auto_unbox = TRUE
582   )
583   invisible(dep)
584 }
585
586 file_checksum <- function(path) {
587   if (!file.exists(path)) return(NA_character_)
588   unname(tools::md5sum(path))
589 }
590
591 portable_path <- function(path, config = CONFIG) {
592   if (is.null(path) || length(path) == 0 || is.na(path)) return(path)
593   path_chr <- as.character(path)
594   root <- normalizePath(config$replication_dir, winslash = "/", mustWork = FALSE)
595   norm <- normalizePath(path_chr, winslash = "/", mustWork = FALSE)
596   prefix <- paste0(root, "/")
597   dplyr::case_when(
598     norm == root ~ ".",
599     startsWith(norm, prefix) ~ substring(norm, nchar(prefix) + 1L),
600     TRUE ~ path_chr
601   )
602 }
603
604 sanitize_manifest_paths <- function(x, config = CONFIG) {
605   if (is.list(x)) {
606     nms <- names(x)
607     out <- lapply(x, sanitize_manifest_paths, config = config)
608     names(out) <- nms
609     return(out)
610   }
611   if (is.character(x) && length(x) == 1 && (grepl("/", x) || grepl("\\\\", x))) {
612     return(portable_path(x, config))
613   }
614   x
615 }
616
617 chunk_dir_for_rds <- function(path) {
618   file.path(dirname(path), paste0(tools::file_path_sans_ext(basename(path)), "_rds_chunks"))
619 }
620
621 write_chunked_rds <- function(object, path, config = CONFIG) {
622   raw <- serialize(object, connection = NULL, xdr = TRUE)
623   compressed <- memCompress(raw, type = "xz")
624   chunk_size <- as.integer((config$rds_chunk_size_mib %||% 45) * 1024^2)
625   if (!is.finite(chunk_size) || chunk_size <= 0) chunk_size <- 45L * 1024L^2L

```

```

626
627 chunk_dir <- chunk_dir_for_rds(path)
628 if (dir.exists(chunk_dir)) unlink(chunk_dir, recursive = TRUE, force = TRUE)
629 dir.create(chunk_dir, recursive = TRUE, showWarnings = FALSE)
630
631 n <- length(compressed)
632 n_chunks <- max(1L, ceiling(n / chunk_size))
633 chunk_files <- character(n_chunks)
634 for (i in seq_len(n_chunks)) {
635   start <- (i - 1L) * chunk_size + 1L
636   end <- min(i * chunk_size, n)
637   chunk_files[[i]] <- file.path(chunk_dir, sprintf("part-%03d.rdsbin", i))
638   writeBin(compressed[start:end], chunk_files[[i]], useBytes = TRUE)
639 }
640
641 manifest <- list(
642   format = "serialized-r-object-xdr-memCompress-xz-chunked",
643   original_path = portable_path(path, config),
644   created_at = format(Sys.time(), "%Y-%m-%d %H:%M:%S %Z"),
645   chunk_size_mib = config$rds_chunk_size_mib %||% 45,
646   n_chunks = n_chunks,
647   uncompressed_bytes = length(raw),
648   compressed_bytes = length(compressed),
649   chunks = lapply(chunk_files, function(cf) list(
650     path = portable_path(cf, config),
651     bytes = file.info(cf)$size,
652     md5 = file_checksum(cf)
653   ))
654 )
655 jsonlite::write_json(manifest, file.path(chunk_dir, "manifest.json"), pretty = TRUE, auto_unbox = TRUE)
656 if (file.exists(path)) unlink(path)
657 invisible(list(path = path, chunk_dir = chunk_dir, manifest = manifest))
658 }
659
660 read_chunked_rds <- function(path) {
661   chunk_dir <- chunk_dir_for_rds(path)
662   manifest_path <- file.path(chunk_dir, "manifest.json")
663   if (!file.exists(manifest_path)) stop("Chunked RDS manifest not found: ", manifest_path, call. = FALSE)
664   manifest <- jsonlite::read_json(manifest_path, simplifyVector = TRUE)
665   chunk_paths <- file.path(chunk_dir, basename(manifest$chunks$path))
666   if (!all(file.exists(chunk_paths))) {
667     # Fall back to paths relative to the project root when the manifest was moved.
668     root <- dirname(dirname(dirname(path)))
669     chunk_paths <- file.path(root, manifest$chunks$path)
670   }
671   if (!all(file.exists(chunk_paths))) {
672     missing <- chunk_paths[!file.exists(chunk_paths)]
673     stop("Missing chunked RDS files: ", paste(missing, collapse = ", "), call. = FALSE)
674   }
675   compressed <- unlist(Map(function(pth, nbytes) {
676     readBin(pth, what = "raw", n = nbytes)
677   }, chunk_paths, file.info(chunk_paths)$size), use.names = FALSE)
678   unserialize(memDecompress(compressed, type = "xz"))
679 }
680
681 write_model_object <- function(object, path, config = CONFIG) {
682   if (isTRUE(config$save_single_rds)) {
683     saveRDS(object, path, compress = "xz")
684     chunk_dir <- chunk_dir_for_rds(path)
685     if (dir.exists(chunk_dir)) unlink(chunk_dir, recursive = TRUE, force = TRUE)
686     invisible(list(path = path, mode = "single_rds"))
687   } else {
688     out <- write_chunked_rds(object, path, config)
689     invisible(c(out, list(mode = "chunked_rds")))
690   }
691 }
692
693 read_model_object <- function(path) {
694   if (file.exists(path)) return(readRDS(path))
695   read_chunked_rds(path)
696 }
697
698 model_object_reference <- function(path, config = CONFIG) {
699   if (file.exists(path)) {
700     return(list(path = path, md5 = file_checksum(path), storage = "single_rds"))
701   }
702   chunk_dir <- chunk_dir_for_rds(path)
703   manifest_path <- file.path(chunk_dir, "manifest.json")
704   if (file.exists(manifest_path)) {
705     return(list(
706       path = path,
707       chunk_manifest = manifest_path,
708       chunk_dir = chunk_dir,
709       manifest_md5 = file_checksum(manifest_path),
710       storage = "chunked_rds"
711     ))
712   }
713   list(path = path, md5 = NA_character_, storage = "missing")
714 }
715

```

```

716
717 output_file_category <- function(path, config = CONFIG) {
718   rel <- portable_path(path, config)
719   dplyr::case_when(
720     startsWith(rel, "output/diagnostics/") ~ "diagnostics",
721     startsWith(rel, "output/intermediate/") ~ "intermediate",
722     startsWith(rel, "output/tables/") ~ "tables",
723     startsWith(rel, "output/figures/") ~ "figures",
724     TRUE ~ "other"
725   )
726 }
727
728 infer_crosswalk_slug_from_path <- function(path) {
729   base <- basename(path)
730   hit <- stringr::str_match(base, "(m2|m4|m5|m6)(\\.|_|$)")[, 2]
731   ifelse(is.na(hit), NA_character_, hit)
732 }
733
734 write_output_manifest <- function(config = CONFIG) {
735   config <- finalize_config(config)
736   if (!dir.exists(config$output_dir)) {
737     empty <- tibble::tibble(
738       path = character(), category = character(), extension = character(), bytes = numeric(),
739       md5 = character(), crosswalk_slug = character(), modified_time = character()
740     )
741     readr::write_csv(empty, config$output_manifest_csv)
742     jsonlite::write_json(list(generated_at = format(Sys.time(), "%Y-%m-%d %H:%M:%S %Z"), files = empty),
743       config$output_manifest_json, pretty = TRUE, auto_unbox = TRUE)
744     return(invisible(empty))
745   }
746   files <- list.files(config$output_dir, recursive = TRUE, full.names = TRUE, all.files = FALSE, no.. = TRUE)
747   files <- files[file.exists(files) & !dir.exists(files)]
748   # Avoid self-recursive checksum churn by writing the manifest after listing but
749   # excluding the previous copy of itself from checksum-based reproducibility checks.
750   info <- file.info(files)
751   out <- tibble::tibble(
752     path = vapply(files, portable_path, character(1), config = config),
753     category = vapply(files, output_file_category, character(1), config = config),
754     extension = tolower(tools::file_ext(files)),
755     bytes = as.numeric(info$size),
756     md5 = vapply(files, file_checksum, character(1)),
757     crosswalk_slug = vapply(files, infer_crosswalk_slug_from_path, character(1)),
758     modified_time = format(info$mtime, "%Y-%m-%d %H:%M:%S %Z")
759   ) %>%
760     dplyr::arrange(category, path)
761   readr::write_csv(out, config$output_manifest_csv)
762   jsonlite::write_json(
763     list(
764       generated_at = format(Sys.time(), "%Y-%m-%d %H:%M:%S %Z"),
765       n_files = nrow(out),
766       total_bytes = sum(out$bytes, na.rm = TRUE),
767       files = out
768     ),
769     config$output_manifest_json,
770     pretty = TRUE,
771     auto_unbox = TRUE
772   )
773   invisible(out)
774 }
775
776 write_pipeline_manifest <- function(config, checks, stage, sources = list(), extra = list()) {
777   sources <- sanitize_manifest_paths(sources, config)
778   extra <- sanitize_manifest_paths(extra, config)
779   manifest <- list(
780     stage = stage,
781     generated_at = format(Sys.time(), "%Y-%m-%d %H:%M:%S %Z"),
782     config = config[c(
783       "first_election_year", "diagnostic_first_election_year", "last_election_year",
784       "reference_year", "crosswalk_weight", "crosswalk_missing_weight_policy",
785       "renormalize_crosswalk_weights", "weight_sum_tolerance", "min_retained_vote_share",
786       "require_balanced_panel", "require_no_duplicate_county_years",
787       "require_no_vcov_repair_for_main", "allow_vcov_repair", "allow_failed_diagnostics",
788       "max_abs_vote_identity_error", "max_missing_cz_year_share",
789       "conley_cutooffs_km", "main_se_type", "nw_lag", "dk_lag", "interacted_controls",
790       "standardize_interacted_controls", "export_crosswalk_maps", "run_crosswalk_sensitivity",
791       "save_single_rds", "rds_chunk_size_mib", "county1990_shapefile_path",
792       "nhgis_extract_dir", "output_manifest_csv", "output_manifest_json"
793     )],
794     sources = sources,
795     checks = checks,
796     fatal_failure = has_fatal_failures(checks),
797     extra = extra
798   )
799   manifest <- sanitize_manifest_paths(manifest, config)
800   jsonlite::write_json(manifest, config$pipeline_manifest_json, pretty = TRUE, auto_unbox = TRUE, null = "null")
801   # Keep the standalone output inventory current after each manifest write.
802   tryCatch(write_output_manifest(config), error = function(e) warning(
803     "Could not update output manifest after writing pipeline manifest: ",
804     conditionMessage(e), call. = FALSE
805   ))

```

```

806 invisible(manifest)
807 }
808
809 "%||%" <- function(x, y) if (is.null(x)) y else x
810
811
812 standardized_control_name <- function(ctrl) paste0("z_", ctrl)
813
814 standardize_baseline_controls <- function(data, controls) {
815   out <- data
816   summary <- purrr::map_dfr(controls, function(ctrl) {
817     vals <- out[[ctrl]]
818     mu <- mean(vals, na.rm = TRUE)
819     sig <- stats::sd(vals, na.rm = TRUE)
820     zname <- standardized_control_name(ctrl)
821     out[[zname]] <- if (is.finite(sig) && sig > 0) (vals - mu) / sig else NA_real_
822     tibble::tibble(
823       control = ctrl,
824       standardized_control = zname,
825       n_nonmissing = sum(!is.na(vals)),
826       mean = mu,
827       sd = sig,
828       min = suppressWarnings(min(vals, na.rm = TRUE)),
829       max = suppressWarnings(max(vals, na.rm = TRUE))
830     )
831   })
832   list(data = out, summary = summary)
833 }
834
835
836 read_county_bridge_exceptions <- function(config) {
837   empty <- tibble::tibble(
838     year = integer(), county_fips = character(), county_name_pattern = character(),
839     override_source_decade = integer(), override_county_fips_source = character(),
840     issue_class = character(), reason = character(), apply = logical()
841   )
842   path <- file.path(config$src_dir, "county_bridge_exceptions.csv")
843   if (!file.exists(path)) return(empty)
844   out <- readr::read_csv(path, show_col_types = FALSE) %>%
845     dplyr::mutate(
846       year = as.integer(year),
847       county_fips = stringr::str_pad(as.character(county_fips), 5, pad = "0"),
848       override_county_fips_source = stringr::str_pad(as.character(override_county_fips_source), 5, pad = "0"),
849       override_source_decade = as.integer(override_source_decade),
850       apply = as.logical(apply)
851     ) %>%
852     dplyr::filter(apply %in% TRUE)
853   dplyr::bind_rows(empty, out)
854 }
855
856 classify_unmatched_county <- function(state_fips, county_fips, county_name, year) {
857   state_fips <- stringr::str_pad(as.character(state_fips), 2, pad = "0")
858   county_fips <- stringr::str_pad(as.character(county_fips), 5, pad = "0")
859   nm <- tolower(as.character(county_name))
860   dplyr::case_when(
861     state_fips %in% c("02", "15", "72") ~ "excluded_nonmainland",
862     county_fips %in% c("04012", "35006", "08014", "46113", "46102", "51683", "51685", "51735") ~ "county_boundary_change_needs_bridge",
863     state_fips == "51" & stringr::str_detect(nm, "south boston|clifton forge|bedford|manassas|poquoson") ~
864       ↪ "county_boundary_change_needs_bridge",
865     year < 1972L ~ "pre_1972_diagnostic_only",
866     TRUE ~ "unexpected_unmatched"
867   )
868 }
869
870 is_adh_mainland_source <- function(state_fips) {
871   !stringr::str_pad(as.character(state_fips), 2, pad = "0") %in% c("02", "15", "72")
872 }
873
874 find_first_shapefile <- function(path) {
875   if (is.null(path) || !nzchar(path)) return(NA_character_)
876   if (file.exists(path) && grepl("\\.shp$", path, ignore.case = TRUE)) return(path)
877   if (dir.exists(path)) {
878     shps <- list.files(path, pattern = "\\shp$", recursive = TRUE, full.names = TRUE, ignore.case = TRUE)
879     if (length(shps) > 0) return(shps[[1]])
880   }
881   NA_character_
882 }
883
884 infer_county_fips_from_sf <- function(sf_obj) {
885   nm <- names(sf_obj)
886   if ("county_fips_1990" %in% nm) return(stringr::str_pad(as.character(sf_obj$county_fips_1990), 5, pad = "0"))
887   if ("FIPS" %in% nm) return(stringr::str_pad(as.character(sf_obj$FIPS), 5, pad = "0"))
888   if ("GEOID" %in% nm) return(stringr::str_pad(as.character(sf_obj$GEOID), 5, pad = "0"))
889   if ("GISJOIN" %in% nm) return(gisjoin_to_fips(sf_obj$GISJOIN))
890   if (all(c("STATEFP", "COUNTYFP") %in% nm)) return(paste0(stringr::str_pad(sf_obj$STATEFP, 2, pad = "0"),
891     ↪ stringr::str_pad(sf_obj$COUNTYFP, 3, pad = "0")))
892   if (all(c("STATEA", "COUNTYA") %in% nm)) return(paste0(stringr::str_pad(sf_obj$STATEA, 2, pad = "0"), stringr::str_pad(sf_obj$COUNTYA,
893     ↪ 3, pad = "0")))
894   if (all(c("STATE_FIPS", "FIPS") %in% nm)) return(stringr::str_pad(as.character(sf_obj$FIPS), 5, pad = "0"))

```

```

892 stop("Could not infer county FIPS from shapefile. Expected county_fips_1990, FIPS, GEOID, GISJOIN, STATEFP+COUNTYFP, STATEA+COUNTYA, or
      ↪ STATE_FIPS+FIPS.")
893 }
894
895 safe_read_county1990_shapefile <- function(config) {
896   if (!requireNamespace("sf", quietly = TRUE)) {
897     warning("Package 'sf' is not installed; skipping crosswalk support maps.", call. = FALSE)
898     return(NULL)
899   }
900
901   candidate_paths <- unique(c(
902     config$county1990_shapefile_path,
903     config$nhgis_extract_dir,
904     configured_shapefile_defaults(config)
905   ))
906   candidate_paths <- candidate_paths[!is.na(candidate_paths) & nzchar(candidate_paths)]
907   candidate_paths <- candidate_paths[!grepl("^path/to", candidate_paths, ignore.case = TRUE)]
908
909   shp <- NA_character_
910   for (candidate in candidate_paths) {
911     shp <- find_first_shapefile(candidate)
912     if (!is.na(shp)) break
913   }
914
915   if (is.na(shp)) {
916     warning(
917       "No 1990 county shapefile found. Put co1990p020.shp (and companion files) under ",
918       file.path(config$replication_dir, "spatial-data", "counties-1990"),
919       ", or set COUNTY1990_SHAPEFILE to a 1990 county .shp file or folder. ",
920       "Skipping map rendering but writing non-spatial diagnostics.",
921       call. = FALSE
922     )
923     return(NULL)
924   }
925
926   sf_obj <- sf::st_read(shp, quiet = TRUE)
927   sf_obj$county_fips_1990 <- infer_county_fips_from_sf(sf_obj)
928   sf_obj <- sf_obj %>%
929     dplyr::filter(!is.na(county_fips_1990))
930
931   # Many historical county shapefiles store multipart counties/islands as multiple
932   # rows with the same FIPS. Dissolve before any diagnostic joins so maps do not
933   # trigger unexpected many-to-many warnings or over-represent multipart counties.
934   sf_obj <- sf_obj %>%
935     dplyr::group_by(county_fips_1990) %>%
936     dplyr::summarise(geometry = sf::st_union(geometry), .groups = "drop")
937
938   attr(sf_obj, "source_shapefile") <- shp
939   sf_obj
940 }
941
942 haversine_km <- function(lon1, lat1, lon2, lat2) {
943   rad <- pi / 180
944   dlon <- (lon2 - lon1) * rad
945   dlat <- (lat2 - lat1) * rad
946   a <- sin(dlat / 2)^2 + cos(lat1 * rad) * cos(lat2 * rad) * sin(dlon / 2)^2
947   6371.0088 * 2 * atan2(sqrt(a), sqrt(pmax(0, 1 - a)))
948 }
949
950 summarise_conley_neighbors <- function(panel, cutoffs_km) {
951   coords <- panel %>%
952     dplyr::distinct(czone, lon, lat) %>%
953     dplyr::filter(is.finite(lon), is.finite(lat))
954   if (nrow(coords) == 0) return(tibble::tibble())
955   dist_mat <- outer(seq_len(nrow(coords)), seq_len(nrow(coords)), Vectorize(function(i, j) {
956     haversine_km(coords$lon[[i]], coords$lat[[i]], coords$lon[[j]], coords$lat[[j]])
957   }))
958   purrr::map_dfr(cutoffs_km, function(cutoff) {
959     counts <- rowSums(dist_mat <= cutoff, na.rm = TRUE) - 1L
960     tibble::tibble(
961       cutoff_km = cutoff,
962       n_cz = nrow(coords),
963       min_neighbors = min(counts),
964       p25_neighbors = as.numeric(stats::quantile(counts, 0.25)),
965       median_neighbors = stats::median(counts),
966       mean_neighbors = mean(counts),
967       p75_neighbors = as.numeric(stats::quantile(counts, 0.75)),
968       max_neighbors = max(counts),
969       cz_with_zero_neighbors = sum(counts == 0)
970     )
971   })
972 }
973
974 render_table_pdf <- function(fragment_tex, standalone_tex, output_pdf) {
975   input_name <- basename(fragment_tex)
976   lines <- c(
977     "\\documentclass[11pt]{article}",
978     "\\usepackage[letterpaper,landscape,margin=0.7in]{geometry}",
979     "\\usepackage{array}",
980     "\\usepackage{booktabs}",

```

```

981     "\\usepackage{longtable}",
982     "\\usepackage{threeparttablex}",
983     "\\usepackage{caption}",
984     "\\renewcommand{\\arraystretch}{1.08}",
985     "\\newcolumnntype{L}[1]{>{\\raggedright\\arraybackslash}p{#1}}",
986     "\\newcolumnntype{C}[1]{>{\\centering\\arraybackslash}p{#1}}",
987     "\\begin{document}",
988     paste0("\\input{", input_name, ".}"),
989     "\\end{document}"
990 )
991 writeLines(lines, standalone_tex)
992
993 tex_engine <- Sys.which("pdflatex")
994 if (!nzchar(tex_engine)) {
995     warning("No LaTeX engine found (pdflatex not on PATH). Wrote TeX sources but did not render a PDF.")
996     return(invisible(NULL))
997 }
998
999 oldwd <- getwd()
1000 on.exit(setwd(oldwd), add = TRUE)
1001 setwd(dirname(standalone_tex))
1002
1003 out1 <- system2(tex_engine, c("-interaction=nonstopmode", basename(standalone_tex)), stdout = TRUE, stderr = TRUE)
1004 out2 <- system2(tex_engine, c("-interaction=nonstopmode", basename(standalone_tex)), stdout = TRUE, stderr = TRUE)
1005 pdf_path <- file.path(dirname(standalone_tex), paste0(tools::file_path_sans_ext(basename(standalone_tex)), ".pdf"))
1006
1007 if (!file.exists(pdf_path)) {
1008     warning("pdflatex ran but no PDF was produced. Last output:\\n", paste(c(out1, out2), collapse = "\\n"))
1009     return(invisible(NULL))
1010 }
1011
1012 file.copy(pdf_path, output_pdf, overwrite = TRUE)
1013
1014 sidecars <- file.path(
1015     dirname(standalone_tex),
1016     paste0(tools::file_path_sans_ext(basename(standalone_tex)), c(".aux", ".log", ".out", ".toc"))
1017 )
1018 unlink(sidecars[file.exists(sidecars)])
1019
1020 invisible(output_pdf)
1021 }

```

3 02_build_analysis_data.R

```
1 # extension-adh2013/src/02_build_analysis_data.R
2
3 preflight_ftz_builtup_support <- function(zip_path, source_decades, tolerance = 1e-6,
4                                         weight_types = c("m2_weight", "m4_weight", "m5_weight", "m6_weight")) {
5   # 1990 is identity-mapped to itself in this project, so M2/M4/M5/M6 support
6   # is not meaningful for that decade. Omitting it avoids the misleading
7   # n_source_counties = 1 synthetic row seen in earlier diagnostics.
8   source_decades <- setdiff(as.integer(source_decades), 1990L)
9
10  purrr::map_dfr(source_decades, function(source_decade) {
11    file_name <- paste0("Crosswalk_", source_decade, "_1990.csv")
12    source_col <- paste0("gisjoin_", source_decade)
13    raw <- readr::read_csv(
14      unz(zip_path, file_name),
15      col_select = dplyr::all_of(c(source_col, weight_types)),
16      show_col_types = FALSE
17    ) %>%
18    dplyr::mutate(
19      gisjoin_source = as.character(.data[[source_col]])
20    ) %>%
21    dplyr::select(gisjoin_source, dplyr::all_of(weight_types))
22    if (source_decade == 2020L) {
23      raw <- raw %>%
24        dplyr::mutate(gisjoin_source = fips_to_gisjoin(stringr::str_pad(gisjoin_source, 5, pad = "0")))
25    }
26
27    raw %>%
28      tidyr::pivot_longer(
29        cols = dplyr::all_of(weight_types),
30        names_to = "weight_type",
31        values_to = "weight"
32      ) %>%
33      dplyr::group_by(gisjoin_source, weight_type) %>%
34      dplyr::summarise(
35        weight_sum = sum_preserve_all_na(weight),
36        n_parts = dplyr::n(),
37        n_missing_parts = sum(is.na(weight)),
38        .groups = "drop"
39      ) %>%
40      dplyr::transmute(
41        source_decade = as.integer(source_decade),
42        gisjoin_source,
43        county_fips_source = gisjoin_to_fips(gisjoin_source),
44        weight_type,
45        weight_sum,
46        n_parts,
47        n_missing_parts,
48        available = !is.na(weight_sum) & weight_sum > tolerance,
49        zero_or_undefined_support = is.na(weight_sum) | weight_sum <= tolerance
50      )
51    })
52  }
53
54  read_ftz_raw_target_rows <- function(source_decade, zip_path, weight_types = c("m2_weight", "m4_weight", "m5_weight", "m6_weight")) {
55    if (source_decade == 1990L) return(tibble::tibble())
56    file_name <- paste0("Crosswalk_", source_decade, "_1990.csv")
57    source_col <- paste0("gisjoin_", source_decade)
58    raw <- readr::read_csv(
59      unz(zip_path, file_name),
60      col_select = dplyr::all_of(c(source_col, "gisjoin_1990", weight_types)),
61      show_col_types = FALSE
62    ) %>%
63    dplyr::mutate(
64      source_decade = as.integer(source_decade),
65      gisjoin_source = as.character(.data[[source_col]]),
66      gisjoin_source = dplyr::if_else(
67        source_decade == 2020L,
68        fips_to_gisjoin(stringr::str_pad(gisjoin_source, 5, pad = "0")),
69        gisjoin_source
70      ),
71      county_fips_source = gisjoin_to_fips(gisjoin_source),
72      county_fips_1990 = gisjoin_to_fips(gisjoin_1990)
73    )
74    raw %>%
75      dplyr::select(source_decade, gisjoin_source, county_fips_source, gisjoin_1990, county_fips_1990, dplyr::all_of(weight_types))
76  }
77
78  build_target_weight_support_diagnostics <- function(zip_path, source_decades, county_pres, cz_lookup,
79                                                    selected_crosswalks, tolerance = 1e-6,
80                                                    adh_czones = NULL) {
81    weight_types <- c("m2_weight", "m4_weight", "m5_weight", "m6_weight")
82    source_support <- preflight_ftz_builtup_support(zip_path, source_decades, tolerance, weight_types)
83
84    raw_targets <- purrr::map_dfr(setdiff(as.integer(source_decades), 1990L), read_ftz_raw_target_rows,
85                                zip_path = zip_path, weight_types = weight_types)
86    support_long <- source_support %>%
87      dplyr::select(source_decade, gisjoin_source, county_fips_source, weight_type, weight_sum, available)
```

```

88
89 # Attach election-year vote mass so the map can indicate which 1990 counties/CZs
90 # actually receive votes from source counties whose M2/M4/M5/M6 support is unavailable.
91 source_votes <- county_pres %>%
92   dplyr::select(year, source_decade, gisjoin_source, county_fips, state_fips, state, county_name, twoparty_votes)
93
94 # Use the selected/fallback crosswalk weights as the vote-mass allocator for the
95 # demonstration map. A target is flagged only if the selected/fallback crosswalk
96 # sends positive vote mass to that 1990 target from a source county where the
97 # diagnostic weight type is unavailable. This avoids over-flagging ordinary
98 # zero-weight target rows in the raw many-to-many FTZ crosswalk.
99 selected_targets <- selected_crosswalks %>%
100   dplyr::filter(!is.na(crosswalk_weight), crosswalk_weight > tolerance) %>%
101   dplyr::select(source_decade, gisjoin_source, county_fips_1990,
102     selected_crosswalk_weight = crosswalk_weight) %>%
103   dplyr::distinct()
104
105 target_long <- raw_targets %>%
106   tidyr::pivot_longer(
107     cols = dplyr::all_of(weight_types),
108     names_to = "weight_type",
109     values_to = "raw_weight_for_type"
110   ) %>%
111   dplyr::left_join(support_long, by = c("source_decade", "gisjoin_source", "county_fips_source", "weight_type")) %>%
112   dplyr::left_join(source_votes, by = c("source_decade", "gisjoin_source"), relationship = "many-to-many") %>%
113   dplyr::left_join(selected_targets, by = c("source_decade", "gisjoin_source", "county_fips_1990"), relationship = "many-to-many") %>%
114   dplyr::mutate(
115     unavailable = !dplyr::coalesce(available, FALSE),
116     selected_crosswalk_weight = dplyr::coalesce(selected_crosswalk_weight, 0),
117     received_vote_mass_proxy = twoparty_votes * selected_crosswalk_weight,
118     receives_positive_vote_mass = !is.na(received_vote_mass_proxy) & received_vote_mass_proxy > 0,
119     affected_received_vote_mass_proxy = dplyr::if_else(
120       unavailable & receives_positive_vote_mass,
121       received_vote_mass_proxy,
122       0
123     )
124   )
125
126 county_diag <- target_long %>%
127   dplyr::filter(!is.na(county_fips_1990), !is.na(year)) %>%
128   dplyr::group_by(county_fips_1990, weight_type) %>%
129   dplyr::summarise(
130     receives_unavailable_source = any(affected_received_vote_mass_proxy > 0, na.rm = TRUE),
131     n_unavailable_source_counties = dplyr::n_distinct(county_fips_source[affected_received_vote_mass_proxy > 0]),
132     unavailable_source_vote_mass_proxy = sum(affected_received_vote_mass_proxy, na.rm = TRUE),
133     total_received_vote_mass_proxy = sum(received_vote_mass_proxy, na.rm = TRUE),
134     share_received_from_unavailable_sources = dplyr::if_else(
135       total_received_vote_mass_proxy > 0,
136       unavailable_source_vote_mass_proxy / total_received_vote_mass_proxy,
137       NA_real_
138     ),
139     .groups = "drop"
140   ) %>%
141   dplyr::left_join(cz_lookup %>% dplyr::select(county_fips_1990, czzone), by = "county_fips_1990")
142
143 if (!is.null(adh_czones)) {
144   adh_czones <- unique(as.integer(adh_czones))
145   county_diag <- county_diag %>% dplyr::filter(czzone %in% adh_czones)
146 }
147
148 cz_diag <- county_diag %>%
149   dplyr::filter(!is.na(czzone)) %>%
150   dplyr::group_by(czzone, weight_type) %>%
151   dplyr::summarise(
152     receives_unavailable_source = any(receives_unavailable_source %in% TRUE, na.rm = TRUE),
153     n_1990_counties_receiving_unavailable_source = sum(receives_unavailable_source %in% TRUE, na.rm = TRUE),
154     n_unavailable_source_counties = sum(n_unavailable_source_counties, na.rm = TRUE),
155     unavailable_source_vote_mass_proxy = sum(unavailable_source_vote_mass_proxy, na.rm = TRUE),
156     total_received_vote_mass_proxy = sum(total_received_vote_mass_proxy, na.rm = TRUE),
157     share_received_from_unavailable_sources = dplyr::if_else(
158       total_received_vote_mass_proxy > 0,
159       unavailable_source_vote_mass_proxy / total_received_vote_mass_proxy,
160       NA_real_
161     ),
162     .groups = "drop"
163   )
164
165 list(source_support = source_support, county_diag = county_diag, cz_diag = cz_diag)
166 }
167
168 build_analysis_data <- function(config = CONFIG, stop_on_fatal = TRUE) {
169   load_required_packages(config)
170   config <- finalize_config(config)
171
172   adh_stack_path <- file.path(config$replication_dir, "adh2013", "dta", "workfile_china.dta")
173   adh_long_path <- file.path(config$replication_dir, "adh2013", "dta", "workfile_china_long.dta")
174   cz_path <- file.path(config$replication_dir, "cz-data", "cz-198090.xls")
175   centroid_path <- file.path(
176     config$replication_dir, "ftz2024", "crosswalks", "County-CD-centroid-lat-lon",
177     "lat_lon_coordinates_county_csv", "counties_1990_xy.csv"

```



```

178 )
179 ftz_zip <- file.path(
180   config$replication_dir, "ftz2024", "crosswalks", "CountyToCounty", "1990", "1990_csv.zip"
181 )
182 aa_path <- file.path(
183   config$replication_dir, "outcomes-data", "aa2021-nospacial",
184   "County_Level_US_Elections_Data",
185   "dataverse_shareable_presidential_county_returns_1868_2020.Rdata"
186 )
187
188 message("Reading ADH baseline and exposure data...")
189 adh_stack <- haven::read_dta(adh_stack_path)
190 adh_long <- haven::read_dta(adh_long_path)
191
192 adh_baseline <- adh_stack %>%
193   dplyr::filter(yr == 1990) %>%
194   dplyr::transmute(
195     czone = as.integer(czone),
196     statefip = as.integer(statefip),
197     dplyr::across(dplyr::all_of(config$baseline_controls), as.numeric)
198   ) %>%
199   dplyr::distinct()
200
201 adh_exposure <- adh_long %>%
202   dplyr::transmute(
203     czone = as.integer(czone),
204     statefip = as.integer(statefip),
205     exposure = as.numeric(d_tradeusch_pw),
206     adh_weight = as.numeric(timepwt48)
207   ) %>%
208   dplyr::distinct()
209
210 adh_sample <- adh_baseline %>%
211   dplyr::inner_join(adh_exposure, by = c("czone", "statefip"))
212
213 adh_duplicate_czones <- adh_sample %>%
214   dplyr::count(czone, name = "rows_per_czone") %>%
215   dplyr::filter(rows_per_czone > 1)
216
217 std_controls <- standardize_baseline_controls(adh_sample, config$baseline_controls)
218 adh_sample <- std_controls$data
219 readr::write_csv(std_controls$summary, config$baseline_control_summary_csv)
220
221 control_corr <- stats::cor(
222   adh_sample[, config$baseline_controls, drop = FALSE],
223   use = "pairwise.complete.obs"
224 )
225
226 control_corr_long <- as.data.frame(as.table(control_corr), stringsAsFactors = FALSE) %>%
227   tibble::as_tibble() %>%
228   dplyr::rename(control_1 = Var1, control_2 = Var2, correlation = Freq)
229 readr::write_csv(control_corr_long, config$baseline_control_correlation_csv)
230
231 message("Reading 1990 county-to-CZ lookup and county centroids...")
232 cz_lookup <- readxl::read_xls(cz_path) %>%
233   dplyr::transmute(
234     county_fips_1990 = stringr::str_pad(as.character(`County FIPS Code`), width = 5, side = "left", pad = "0"),
235     czone = as.integer(CZ90),
236     county_name_1990 = as.character(`County name`),
237     pop1990 = as.numeric(`Population 1990`)
238   )
239
240 cz_lookup_duplicate_count <- cz_lookup %>%
241   dplyr::count(county_fips_1990, name = "rows_per_1990_county") %>%
242   dplyr::filter(rows_per_1990_county > 1) %>%
243   nrow()
244
245 cz_lookup_multi_czone_count <- cz_lookup %>%
246   dplyr::distinct(county_fips_1990, czone) %>%
247   dplyr::count(county_fips_1990, name = "czones_per_1990_county") %>%
248   dplyr::filter(czones_per_1990_county > 1) %>%
249   nrow()
250
251 county_centroids <- readr::read_csv(centroid_path, show_col_types = FALSE) %>%
252   dplyr::transmute(
253     county_fips_1990 = gisjoin_to_fips(gisjoin_1990),
254     lon = as.numeric(centroid_x),
255     lat = as.numeric(centroid_y)
256   )
257
258 cz_centroids <- cz_lookup %>%
259   dplyr::inner_join(county_centroids, by = "county_fips_1990") %>%
260   dplyr::group_by(czone) %>%
261   dplyr::mutate(pop_w = pop1990 / sum(pop1990, na.rm = TRUE)) %>%
262   dplyr::summarise(
263     lon = stats::weighted.mean(lon, pop_w, na.rm = TRUE),
264     lat = stats::weighted.mean(lat, pop_w, na.rm = TRUE),
265     pop1990 = sum(pop1990, na.rm = TRUE),
266     counties_in_cz = dplyr::n(),
267     .groups = "drop"
268   ) %>%
269   dplyr::inner_join(adh_sample %>% dplyr::distinct(czone), by = "czone")

```

```

268 readr::write_csv(cz_centroids, config$cz_centroids_csv)
269
270 preflight_support <- preflight_ftz_builtup_support(
271   zip_path = ftz_zip,
272   source_decades = config$source_decades,
273   tolerance = config$weight_sum_tolerance
274 )
275 preflight_summary <- preflight_support %>%
276   dplyr::group_by(source_decade, weight_type) %>%
277   dplyr::summarise(
278     n_source_counties = dplyr::n(),
279     n_zero_or_undefined_support = sum(zero_or_undefined_support),
280     .groups = "drop"
281   )
282 readr::write_csv(preflight_summary, config$crosswalk_preflight_summary_csv)
283 readr::write_csv(
284   preflight_support %>% dplyr::filter(zero_or_undefined_support),
285   config$crosswalk_preflight_counties_csv
286 )
287
288 builtup_preflight <- preflight_support %>%
289   dplyr::filter(weight_type %in% c("m5_weight", "m6_weight"))
290 if (any(builtup_preflight$zero_or_undefined_support)) {
291   summary_msg <- preflight_summary %>%
292     dplyr::filter(weight_type %in% c("m5_weight", "m6_weight"), n_zero_or_undefined_support > 0) %>%
293     dplyr::mutate(piece = paste0(weight_type, " ", source_decade, ": ", n_zero_or_undefined_support)) %>%
294     dplyr::pull(piece) %>%
295     paste(collapse = "; ")
296   warning(
297     "FTZ built-up-support preflight: strict M5/M6 weighting leaves some source counties ",
298     "with zero or undefined built-up support, so a strict M5/M6 run cannot produce the balanced ",
299     "ADH mainland CZ panel. Current policy is '", config$crosswalk_missing_weight_policy, "'. ",
300     "Zero/undefined support counts: ", summary_msg, ". See ",
301     config$crosswalk_preflight_summary_csv,
302     call. = FALSE
303   )
304 }
305
306 message("Reading FTZ county crosswalks with ", config$crosswalk_weight, "...")
307 crosswalks <- purrr::map_dfr(
308   config$source_decades,
309   ~ read_ftz_crosswalk(
310     source_decade = .x,
311     zip_path = ftz_zip,
312     county_fips_1990_identity = cz_lookup$county_fips_1990,
313     weight_col = config$crosswalk_weight,
314     missing_weight_policy = config$crosswalk_missing_weight_policy,
315     renormalize_weights = config$renormalize_crosswalk_weights,
316     tolerance = config$weight_sum_tolerance
317   )
318 )
319
320 crosswalk_source_diagnostics <- crosswalks %>%
321   dplyr::group_by(source_decade, gisjoin_source, county_fips_source) %>%
322   dplyr::summarise(
323     weight_sum_raw = dplyr::first(weight_sum_raw),
324     selected_weight_sum_raw = dplyr::first(selected_weight_sum_raw),
325     crosswalk_weight_sum = dplyr::first(crosswalk_weight_sum),
326     raw_weight_undefined = dplyr::first(raw_weight_undefined),
327     raw_weight_sum_valid = dplyr::first(raw_weight_sum_valid),
328     raw_weight_requires_policy = dplyr::first(raw_weight_requires_policy),
329     fallback_used = any(fallback_used),
330     fallback_policy = if (any(fallback_policy != "none")) {
331       dplyr::first(fallback_policy[fallback_policy != "none"])
332     } else {
333       "none"
334     },
335     n_targets = dplyr::n(),
336     positive_targets = sum(!is.na(crosswalk_weight) & crosswalk_weight > 0),
337     missing_raw_weight_parts = sum(raw_weight_missing_parts),
338     final_weight_sum_ok = !is.na(dplyr::first(crosswalk_weight_sum)) &
339       abs(dplyr::first(crosswalk_weight_sum) - 1) <= config$weight_sum_tolerance,
340     .groups = "drop"
341   )
342
343 readr::write_csv(crosswalk_source_diagnostics, config$crosswalk_diagnostics_csv)
344 readr::write_csv(
345   crosswalk_source_diagnostics %>%
346     dplyr::filter(raw_weight_undefined | !final_weight_sum_ok),
347   config$crosswalk_zero_support_csv
348 )
349 readr::write_csv(
350   crosswalks %>%
351     dplyr::select(
352       source_decade, county_fips_source, county_fips_1990, raw_weight,
353       weight_sum_raw, crosswalk_weight, crosswalk_weight_sum,
354       raw_weight_undefined, raw_weight_sum_valid, raw_weight_requires_policy,
355       fallback_used, fallback_policy, weight_source
356     ),
357   config$crosswalk_row_diagnostics_csv

```

```

358 )
359
360 message("Reading and validating Amlani and Algara (2021) presidential county returns...")
361 aa_pres <- read_aa_rdata_object(aa_path, object_name = "pres_elections_release")
362
363 county_pres_raw <- aa_pres %>%
364   dplyr::transmute(
365     year = as.integer(election_year),
366     county_fips = stringr::str_pad(as.character(fips), width = 5, side = "left", pad = "0"),
367     state_fips = stringr::str_pad(as.character(sfips), width = 2, side = "left", pad = "0"),
368     state = as.character(state),
369     county_name = as.character(county_name),
370     rep_votes = as.numeric(republican_raw_votes),
371     dem_votes = as.numeric(democratic_raw_votes),
372     twoparty_votes = as.numeric(pres_raw_county_vote_totals_two_party),
373     total_votes = as.numeric(raw_county_vote_totals),
374     complete_case = as.integer(complete_county_cases)
375   ) %>%
376   dplyr::filter(year %in% config$diagnostic_event_years)
377
378 exact_duplicate_rows <- nrow(county_pres_raw) - nrow(dplyr::distinct(county_pres_raw))
379 county_pres_distinct <- county_pres_raw %>% dplyr::distinct()
380
381 duplicate_county_years <- county_pres_distinct %>%
382   dplyr::count(year, county_fips, name = "rows_per_county_year") %>%
383   dplyr::filter(rows_per_county_year > 1) %>%
384   dplyr::left_join(county_pres_distinct, by = c("year", "county_fips"))
385 readr::write_csv(duplicate_county_years, config$outcome_duplicate_report_csv)
386
387 county_pres_validation <- county_pres_distinct %>%
388   dplyr::mutate(
389     valid_fips = !is.na(county_fips) & stringr::str_detect(county_fips, "[0-9]{5}$"),
390     nonmissing_votes = !is.na(rep_votes) & !is.na(dem_votes) & !is.na(twoparty_votes),
391     nonnegative_votes = rep_votes >= 0 & dem_votes >= 0 & twoparty_votes >= 0 &
392       (is.na(total_votes) | total_votes >= 0),
393     twoparty_matches = abs((rep_votes + dem_votes) - twoparty_votes) <= 1e-6,
394     usable_for_bridge = valid_fips & nonmissing_votes & nonnegative_votes &
395       twoparty_matches & twoparty_votes > 0 & complete_case == 1L
396   )
397 readr::write_csv(county_pres_validation, config$outcome_validation_csv)
398
399 invalid_fips_count <- county_pres_validation %>%
400   dplyr::filter(!valid_fips) %>%
401   nrow()
402 max_vote_identity_error <- county_pres_validation %>%
403   dplyr::filter(nonmissing_votes) %>%
404   dplyr::mutate(abs_vote_identity_error = abs((rep_votes + dem_votes) - twoparty_votes)) %>%
405   dplyr::pull(abs_vote_identity_error) %>%
406   max(na.rm = TRUE)
407
408 bridge_exceptions <- read_county_bridge_exceptions(config)
409 if (nrow(bridge_exceptions) > 0) {
410   readr::write_csv(bridge_exceptions, file.path(config$diagnostic_dir, "county_bridge_exceptions_applied.csv"))
411 }
412
413 county_pres <- county_pres_validation %>%
414   dplyr::filter(usable_for_bridge) %>%
415   dplyr::mutate(source_decade_default = year_to_source_decade(year)) %>%
416   dplyr::left_join(
417     bridge_exceptions %>%
418       dplyr::select(year, county_fips, override_source_decade, override_county_fips_source, exception_issue_class = issue_class,
419         ↪ exception_reason = reason),
420     by = c("year", "county_fips")
421   ) %>%
422   dplyr::mutate(
423     source_decade = dplyr::coalesce(override_source_decade, source_decade_default),
424     county_fips_for_crosswalk = dplyr::coalesce(override_county_fips_source, county_fips),
425     gisjoin_source = fips_to_gisjoin(county_fips_for_crosswalk),
426     source_scope = dplyr::if_else(is_adh_mainland_source(state_fips), "adh_mainland_eligible", "excluded_nonmainland"),
427     bridge_exception_applied = !is.na(override_county_fips_source) | !is.na(override_source_decade)
428   ) %>%
429   dplyr::select(
430     year, county_fips, county_fips_for_crosswalk, state_fips, state, county_name, rep_votes, dem_votes,
431     twoparty_votes, total_votes, source_decade, source_decade_default, gisjoin_source, source_scope,
432     bridge_exception_applied, exception_issue_class, exception_reason
433   )
434
435 target_support <- build_target_weight_support_diagnostics(
436   zip_path = ftz_zip,
437   source_decades = config$source_decades,
438   county_pres = county_pres,
439   cz_lookup = cz_lookup,
440   selected_crosswalks = crosswalks,
441   tolerance = config$weight_sum_tolerance,
442   adh_czones = adh_sample$czone
443 )
444 readr::write_csv(target_support$source_support, config$all_weight_support_csv)
445 readr::write_csv(target_support$county_diag, config$target_weight_support_county_csv)
446 readr::write_csv(target_support$cz_diag, config$target_weight_support_cz_csv)

```

```

447 source_vote_denominators <- county_pres %>%
448   dplyr::group_by(year) %>%
449   dplyr::summarise(source_two_party_votes_prejoin = sum(twoparty_votes, na.rm = TRUE), .groups = "drop")
450
451 positive_target_summary <- crosswalks %>%
452   dplyr::filter(!is.na(crosswalk_weight), crosswalk_weight > config$weight_sum_tolerance) %>%
453   dplyr::left_join(cz_lookup %>% dplyr::select(county_fips_1990, czzone), by = "county_fips_1990") %>%
454   dplyr::group_by(source_decade, gisjoin_source) %>%
455   dplyr::summarise(
456     positive_target_1990_counties = paste(sort(unique(county_fips_1990)), collapse = ";"),
457     positive_target_czones = paste(sort(unique(stats::na.omit(czzone))), collapse = ";"),
458     positive_target_count = dplyr::n_distinct(county_fips_1990),
459     positive_target_cz_count = dplyr::n_distinct(czzone, na.rm = TRUE),
460     .groups = "drop"
461   )
462
463 # Source-level issue report. Do not filter on row-level crosswalk_weight <= 0:
464 # valid source counties naturally have many zero-weight target rows in the raw
465 # many-to-many FTZ crosswalk. A source is an issue only if its selected raw
466 # weight is undefined/invalid, its final weight sum is invalid, or it has no
467 # positive final target weight after the chosen fallback policy.
468 crosswalk_source_issue_rows <- crosswalk_source_diagnostics %>%
469   dplyr::mutate(
470     no_positive_final_target = positive_targets == 0,
471     final_weight_invalid = is.na(crosswalk_weight_sum) |
472       abs(crosswalk_weight_sum - 1) > config$weight_sum_tolerance
473   ) %>%
474   dplyr::filter(raw_weight_undefined | raw_weight_requires_policy | final_weight_invalid | no_positive_final_target) %>%
475   dplyr::left_join(positive_target_summary, by = c("source_decade", "gisjoin_source"))
476
477 crosswalk_issue_vote_report <- county_pres %>%
478   dplyr::left_join(
479     crosswalk_source_issue_rows,
480     by = c("source_decade", "gisjoin_source")
481   ) %>%
482   dplyr::filter(
483     raw_weight_undefined %in% TRUE |
484     raw_weight_requires_policy %in% TRUE |
485     final_weight_invalid %in% TRUE |
486     no_positive_final_target %in% TRUE
487   ) %>%
488   dplyr::left_join(source_vote_denominators, by = "year") %>%
489   dplyr::mutate(
490     vote_share_of_year = twoparty_votes / source_two_party_votes_prejoin,
491     source_issue_class = dplyr::case_when(
492       no_positive_final_target %in% TRUE & fallback_used %in% TRUE ~ paste0(
493         "selected_", weight_slug(config$crosswalk_weight), "_unavailable_", fallback_policy, "_used_but_no_positive_target"
494       ),
495       no_positive_final_target %in% TRUE & raw_weight_requires_policy %in% TRUE ~ paste0(
496         "selected_", weight_slug(config$crosswalk_weight), "_unavailable_no_fallback"
497       ),
498       no_positive_final_target %in% TRUE ~ classify_unmatched_county(state_fips, county_fips, county_name, year),
499       raw_weight_requires_policy %in% TRUE & fallback_used %in% TRUE ~ paste0(
500         "selected_", weight_slug(config$crosswalk_weight), "_unavailable_", fallback_policy, "_used"
501       ),
502       raw_weight_requires_policy %in% TRUE & !(fallback_used %in% TRUE) ~ paste0(
503         "selected_", weight_slug(config$crosswalk_weight), "_unavailable_no_fallback"
504       ),
505       final_weight_invalid %in% TRUE ~ "final_weight_invalid",
506       TRUE ~ "reported_crosswalk_issue"
507     )
508   ) %>%
509   dplyr::select(
510     year, source_decade, source_decade_default, source_scope, source_issue_class,
511     state_fips, state, county_fips, county_fips_for_crosswalk, county_name,
512     bridge_exception_applied, exception_issue_class, exception_reason,
513     twoparty_votes, source_two_party_votes_prejoin, vote_share_of_year,
514     weight_sum_raw, selected_weight_sum_raw, crosswalk_weight_sum,
515     raw_weight_undefined, raw_weight_sum_valid, raw_weight_requires_policy,
516     fallback_used, fallback_policy, no_positive_final_target, final_weight_invalid,
517     n_targets, positive_targets, positive_target_1990_counties, positive_target_czones,
518     positive_target_count, positive_target_cz_count
519   ) %>%
520   dplyr::arrange(year, source_issue_class, state_fips, county_fips)
521 readr::write_csv(crosswalk_issue_vote_report, config$crosswalk_issue_vote_report_csv)
522
523 national_pre <- county_pres %>%
524   dplyr::group_by(year) %>%
525   dplyr::summarise(
526     scope = "pre_bridge",
527     rep_votes = sum(rep_votes, na.rm = TRUE),
528     dem_votes = sum(dem_votes, na.rm = TRUE),
529     twoparty_votes = sum(twoparty_votes, na.rm = TRUE),
530     total_votes = sum(total_votes, na.rm = TRUE),
531     source_counties = dplyr::n_distinct(county_fips),
532     .groups = "drop"
533   )
534
535 state_pre <- county_pres %>%
536   dplyr::group_by(year, state_fips, state) %>%

```

```

537   dplyr::summarise(
538     scope = "pre_bridge",
539     rep_votes = sum(rep_votes, na.rm = TRUE),
540     dem_votes = sum(dem_votes, na.rm = TRUE),
541     twoparty_votes = sum(twoparty_votes, na.rm = TRUE),
542     total_votes = sum(total_votes, na.rm = TRUE),
543     source_counties = dplyr::n_distinct(county_fips),
544     .groups = "drop"
545   )
546
547   message("Bridging county returns to 1990 counties...")
548   county_bridged <- county_pres %>%
549     dplyr::left_join(
550       crosswalks %>%
551         dplyr::select(
552           source_decade, gisjoin_source, county_fips_1990, raw_weight,
553           crosswalk_weight, raw_weight_undefined, raw_weight_sum_valid,
554           raw_weight_requires_policy, fallback_used, fallback_policy, weight_source
555         ),
556       by = c("source_decade", "gisjoin_source"),
557       relationship = "many-to-many"
558     )
559
560   unmatched_source_counties <- county_bridged %>%
561     dplyr::group_by(year, source_decade, source_decade_default, county_fips, county_fips_for_crosswalk, state_fips, state, county_name,
562       ↪ source_scope, bridge_exception_applied, exception_issue_class, exception_reason) %>%
563     dplyr::summarise(
564       twoparty_votes = dplyr::first(twoparty_votes),
565       has_positive_crosswalk_weight = any(!is.na(crosswalk_weight) & crosswalk_weight > 0),
566       raw_weight_undefined = any(raw_weight_undefined %in% TRUE, na.rm = TRUE),
567       fallback_used = any(fallback_used %in% TRUE, na.rm = TRUE),
568       .groups = "drop"
569     ) %>%
570     dplyr::left_join(source_vote_denominators, by = "year") %>%
571     dplyr::mutate(
572       vote_share_of_year = twoparty_votes / source_two_party_votes_prejoin,
573       source_issue_class = classify_unmatched_county(state_fips, county_fips, county_name, year)
574     ) %>%
575     dplyr::filter(!has_positive_crosswalk_weight)
576   readr::write_csv(unmatched_source_counties, config$unmatched_source_counties_csv)
577   readr::write_csv(
578     unmatched_source_counties %>%
579       dplyr::count(year, source_scope, source_issue_class, name = "unmatched_source_counties") %>%
580       dplyr::left_join(
581         unmatched_source_counties %>%
582           dplyr::group_by(year, source_scope, source_issue_class) %>%
583           dplyr::summarise(unmatched_two_party_votes = sum(twoparty_votes, na.rm = TRUE), .groups = "drop"),
584         by = c("year", "source_scope", "source_issue_class")
585       ),
586     config$county_bridge_classification_csv
587   )
588
589   original_by_year <- county_pres %>%
590     dplyr::group_by(year) %>%
591     dplyr::summarise(
592       source_counties = dplyr::n_distinct(county_fips),
593       source_two_party_votes_prejoin = sum(twoparty_votes, na.rm = TRUE),
594       source_rep_votes_prejoin = sum(rep_votes, na.rm = TRUE),
595       source_dem_votes_prejoin = sum(dem_votes, na.rm = TRUE),
596       .groups = "drop"
597     )
598
599   naive_postjoin_denominator <- county_bridged %>%
600     dplyr::group_by(year) %>%
601     dplyr::summarise(
602       source_two_party_votes_after_join_naive_for_diagnostic_only = sum(twoparty_votes, na.rm = TRUE),
603       .groups = "drop"
604     )
605
606   bridged_by_year <- county_bridged %>%
607     dplyr::filter(!is.na(crosswalk_weight), crosswalk_weight > 0) %>%
608     dplyr::group_by(year) %>%
609     dplyr::summarise(
610       matched_source_counties = dplyr::n_distinct(county_fips),
611       bridged_two_party_votes = sum(twoparty_votes * crosswalk_weight, na.rm = TRUE),
612       bridged_rep_votes = sum(rep_votes * crosswalk_weight, na.rm = TRUE),
613       bridged_dem_votes = sum(dem_votes * crosswalk_weight, na.rm = TRUE),
614       fallback_source_counties = dplyr::n_distinct(county_fips[fallback_used %in% TRUE]),
615       .groups = "drop"
616     )
617
618   original_by_scope <- county_pres %>%
619     dplyr::group_by(year, source_scope) %>%
620     dplyr::summarise(
621       source_counties = dplyr::n_distinct(county_fips),
622       source_two_party_votes_prejoin = sum(twoparty_votes, na.rm = TRUE),
623       .groups = "drop"
624     )
625
626   bridged_by_scope <- county_bridged %>%
627     dplyr::filter(!is.na(crosswalk_weight), crosswalk_weight > 0) %>%

```

```

626   dplyr::group_by(year, source_scope) %>%
627   dplyr::summarise(
628     matched_source_counties = dplyr::n_distinct(county_fips),
629     bridged_two_party_votes = sum(twoparty_votes * crosswalk_weight, na.rm = TRUE),
630     fallback_source_counties = dplyr::n_distinct(county_fips[fallback_used %in% TRUE]),
631     .groups = "drop"
632   )
633   retention_by_scope <- original_by_scope %>%
634   dplyr::left_join(bridged_by_scope, by = c("year", "source_scope")) %>%
635   dplyr::mutate(
636     matched_source_counties = dplyr::coalesce(matched_source_counties, 0L),
637     bridged_two_party_votes = dplyr::coalesce(bridged_two_party_votes, 0),
638     fallback_source_counties = dplyr::coalesce(fallback_source_counties, 0L),
639     unmatched_source_counties = source_counties - matched_source_counties,
640     retained_vote_share = bridged_two_party_votes / source_two_party_votes_prejoin
641   )
642   readr::write_csv(retention_by_scope, config$mainland_retention_csv)
643
644   mainland_retention_wide <- retention_by_scope %>%
645   dplyr::select(year, source_scope, source_two_party_votes_prejoin, bridged_two_party_votes, retained_vote_share) %>%
646   tidyr::pivot_wider(
647     names_from = source_scope,
648     values_from = c(source_two_party_votes_prejoin, bridged_two_party_votes, retained_vote_share),
649     names_sep = "_"
650   )
651
652   bridge_diagnostics <- original_by_year %>%
653   dplyr::left_join(naive_postjoin_denominator, by = "year") %>%
654   dplyr::left_join(bridged_by_year, by = "year") %>%
655   dplyr::left_join(mainland_retention_wide, by = "year") %>%
656   dplyr::mutate(
657     matched_source_counties = dplyr::coalesce(matched_source_counties, 0L),
658     unmatched_source_counties = source_counties - matched_source_counties,
659     retained_vote_share_correct = bridged_two_party_votes / source_two_party_votes_prejoin,
660     retained_vote_share = retained_vote_share_correct,
661     retained_vote_share_adh_mainland_eligible = retained_vote_share_adh_mainland_eligible
662   )
663   readr::write_csv(bridge_diagnostics, config$bridge_diagnostics_csv)
664
665   national_post <- county_bridged %>%
666   dplyr::filter(!is.na(crosswalk_weight), crosswalk_weight > 0) %>%
667   dplyr::group_by(year) %>%
668   dplyr::summarise(
669     scope = "post_bridge",
670     rep_votes = sum(rep_votes * crosswalk_weight, na.rm = TRUE),
671     dem_votes = sum(dem_votes * crosswalk_weight, na.rm = TRUE),
672     twoparty_votes = sum(twoparty_votes * crosswalk_weight, na.rm = TRUE),
673     total_votes = sum(total_votes * crosswalk_weight, na.rm = TRUE),
674     source_counties = dplyr::n_distinct(county_fips),
675     .groups = "drop"
676   )
677   readr::write_csv(dplyr::bind_rows(national_pre, national_post), config$national_vote_totals_csv)
678
679   state_post <- county_bridged %>%
680   dplyr::filter(!is.na(crosswalk_weight), crosswalk_weight > 0) %>%
681   dplyr::group_by(year, state_fips, state) %>%
682   dplyr::summarise(
683     scope = "post_bridge",
684     rep_votes = sum(rep_votes * crosswalk_weight, na.rm = TRUE),
685     dem_votes = sum(dem_votes * crosswalk_weight, na.rm = TRUE),
686     twoparty_votes = sum(twoparty_votes * crosswalk_weight, na.rm = TRUE),
687     total_votes = sum(total_votes * crosswalk_weight, na.rm = TRUE),
688     source_counties = dplyr::n_distinct(county_fips),
689     .groups = "drop"
690   )
691   readr::write_csv(dplyr::bind_rows(state_pre, state_post), config$state_vote_totals_csv)
692
693   county90_panel <- county_bridged %>%
694   dplyr::filter(!is.na(county_fips_1990), !is.na(crosswalk_weight), crosswalk_weight > 0) %>%
695   dplyr::mutate(
696     rep_votes = rep_votes * crosswalk_weight,
697     dem_votes = dem_votes * crosswalk_weight,
698     twoparty_votes = twoparty_votes * crosswalk_weight,
699     total_votes = total_votes * crosswalk_weight
700   ) %>%
701   dplyr::group_by(year, county_fips_1990) %>%
702   dplyr::summarise(
703     rep_votes = sum(rep_votes, na.rm = TRUE),
704     dem_votes = sum(dem_votes, na.rm = TRUE),
705     twoparty_votes = sum(twoparty_votes, na.rm = TRUE),
706     total_votes = sum(total_votes, na.rm = TRUE),
707     .groups = "drop"
708   )
709
710   message("Aggregating bridged county votes to 1990 commuting zones...")
711   cz_panel <- county90_panel %>%
712   dplyr::inner_join(cz_lookup, by = "county_fips_1990") %>%
713   dplyr::group_by(czone, year) %>%
714   dplyr::summarise(
715     rep_votes = sum(rep_votes, na.rm = TRUE),

```

```

716     dem_votes = sum(dem_votes, na.rm = TRUE),
717     twoparty_votes = sum(twoparty_votes, na.rm = TRUE),
718     total_votes = sum(total_votes, na.rm = TRUE),
719     counties_contributing = dplyr::n(),
720     .groups = "drop"
721 ) %>%
722 dplyr::mutate(rep_margin = (rep_votes - dem_votes) / twoparty_votes)
723
724 make_panel <- function(years) {
725   tidy::expand_grid(czone = sort(unique(adh_sample$czone)), year = years) %>%
726   dplyr::mutate(election_index = match(year, sort(unique(years)))) %>%
727   dplyr::left_join(cz_panel, by = c("czone", "year")) %>%
728   dplyr::inner_join(adh_sample, by = "czone") %>%
729   dplyr::inner_join(cz_centroids, by = "czone") %>%
730   dplyr::arrange(czone, year)
731 }
732
733 analysis_panel_full <- make_panel(config$event_years)
734 diagnostic_panel_full <- make_panel(config$diagnostic_event_years)
735
736 analysis_panel <- analysis_panel_full %>%
737   dplyr::filter(!is.na(.data[[config$outcome_var]]))
738 diagnostic_panel <- diagnostic_panel_full %>%
739   dplyr::filter(!is.na(.data[[config$outcome_var]]))
740
741 missing_cz_years <- dplyr::bind_rows(
742   analysis_panel_full %>%
743     dplyr::filter(is.na(.data[[config$outcome_var]])) %>%
744     dplyr::transmute(sample = "main_1972_start", czone, year),
745   diagnostic_panel_full %>%
746     dplyr::filter(is.na(.data[[config$outcome_var]])) %>%
747     dplyr::transmute(sample = "diagnostic_1952_start", czone, year)
748 )
749 readr::write_csv(missing_cz_years, config$missing_cz_years_csv)
750
751 panel_coverage <- dplyr::bind_rows(
752   analysis_panel_full %>% dplyr::mutate(sample = "main_1972_start"),
753   diagnostic_panel_full %>% dplyr::mutate(sample = "diagnostic_1952_start")
754 ) %>%
755   dplyr::mutate(has_outcome = !is.na(.data[[config$outcome_var]])) %>%
756   dplyr::group_by(sample, year) %>%
757   dplyr::summarise(
758     n_cz_total = dplyr::n(),
759     n_cz_with_outcome = sum(has_outcome),
760     missing_cz_years = n_cz_total - n_cz_with_outcome,
761     share_cz_with_outcome = mean(has_outcome),
762     .groups = "drop"
763   )
764 readr::write_csv(panel_coverage, config$panel_coverage_csv)
765
766 saveRDS(analysis_panel, config$analysis_panel_rds)
767 saveRDS(analysis_panel_full, config$analysis_panel_full_rds)
768 saveRDS(diagnostic_panel, config$diagnostic_panel_rds)
769 saveRDS(diagnostic_panel_full, config$diagnostic_panel_full_rds)
770
771 main_expected_n <- length(unique(adh_sample$czone)) * length(config$event_years)
772 main_missing_n <- nrow(analysis_panel_full) - nrow(analysis_panel)
773 main_missing_share <- if (main_expected_n > 0) main_missing_n / main_expected_n else NA_real_
774 adh_duplicate_czone_count <- nrow(adh_duplicate_czones)
775 min_main_retained <- bridge_diagnostics %>%
776   dplyr::filter(year %in% config$event_years) %>%
777   dplyr::pull(retained_vote_share) %>%
778   min(na.rm = TRUE)
779 duplicate_count <- duplicate_county_years %>%
780   dplyr::distinct(year, county_fips) %>%
781   nrow()
782 twoparty_mismatch_count <- county_pres_validation %>%
783   dplyr::filter(complete_case == 1L, valid_fips, nonmissing_votes, !twoparty_matches) %>%
784   nrow()
785 nonnegative_failure_count <- county_pres_validation %>%
786   dplyr::filter(complete_case == 1L, valid_fips, nonmissing_votes, !nonnegative_votes) %>%
787   nrow()
788 invalid_final_weight_count <- crosswalk_source_diagnostics %>%
789   dplyr::filter(!final_weight_sum_ok) %>%
790   nrow()
791 invalid_positive_final_weight_count <- crosswalk_source_diagnostics %>%
792   dplyr::filter(positive_targets > 0, !final_weight_sum_ok) %>%
793   nrow()
794 undefined_selected_weight_count <- crosswalk_source_diagnostics %>%
795   dplyr::filter(raw_weight_undefined) %>%
796   nrow()
797
798 checks <- dplyr::bind_rows(
799   validation_check(
800     "adh_one_row_per_czone",
801     adh_duplicate_czone_count == 0,
802     fatal = TRUE,
803     n_failed = adh_duplicate_czone_count,
804     details = "Requires one ADH baseline/exposure row per commuting zone."
805   ),

```

```

806 validation_check(
807   "cz_lookup_unique_1990_counties",
808   cz_lookup_duplicate_count == 0 && cz_lookup_multi_czone_count == 0,
809   fatal = TRUE,
810   n_failed = cz_lookup_duplicate_count + cz_lookup_multi_czone_count,
811   details = "Requires each 1990 county FIPS to appear once and map to one CZ."
812 ),
813 validation_check(
814   "aa_all_county_fips_valid",
815   invalid_fips_count == 0,
816   fatal = TRUE,
817   n_failed = invalid_fips_count,
818   details = "Requires all county FIPS to be five numeric digits before filtering."
819 ),
820 validation_check(
821   "aa_vote_identity_error_within_tolerance",
822   is.finite(max_vote_identity_error) && max_vote_identity_error <= config$max_abs_vote_identity_error,
823   fatal = TRUE,
824   n_failed = sum(!county_pres_validation$twoparty_matches, na.rm = TRUE),
825   details = paste0("Max absolute vote identity error: ", signif(max_vote_identity_error, 6))
826 ),
827 validation_check(
828   "crosswalk_positive_final_weight_sums_equal_one",
829   invalid_positive_final_weight_count == 0,
830   fatal = TRUE,
831   n_failed = invalid_positive_final_weight_count,
832   details = paste0("Tolerance: ", config$weight_sum_tolerance)
833 ),
834 reported_check(
835   "crosswalk_all_source_weight_sums_equal_one_or_reported",
836   n_reported = invalid_final_weight_count,
837   details = paste0(
838     "Global non-ADH or unsupported source-county rows are reported rather than treated as validation failures. ",
839     "Estimation is gated by positive final weights, retained vote share, and panel balance. Tolerance: ",
840     config$weight_sum_tolerance
841   )
842 ),
843 validation_check(
844   "crosswalk_undefined_selected_weights_handled",
845   config$crosswalk_missing_weight_policy != "fail" || undefined_selected_weight_count == 0,
846   fatal = TRUE,
847   n_failed = undefined_selected_weight_count,
848   details = paste0("Policy: ", config$crosswalk_missing_weight_policy)
849 ),
850 validation_check(
851   "aa_no_duplicate_county_years_after_exact_distinct",
852   duplicate_count == 0 || !isTRUE(config$require_no_duplicate_county_years),
853   fatal = isTRUE(config$require_no_duplicate_county_years),
854   n_failed = duplicate_count,
855   details = paste0("Exact duplicate rows removed: ", exact_duplicate_rows)
856 ),
857 validation_check(
858   "aa_two_party_votes_match_party_votes",
859   twoparty_mismatch_count == 0,
860   fatal = TRUE,
861   n_failed = twoparty_mismatch_count,
862   details = "Requires republican + democratic votes to equal two-party votes."
863 ),
864 validation_check(
865   "aa_votes_nonnegative",
866   nonnegative_failure_count == 0,
867   fatal = TRUE,
868   n_failed = nonnegative_failure_count,
869   details = "Requires nonnegative Republican, Democratic, two-party, and total votes."
870 ),
871 validation_check(
872   "bridged_vote_retention_above_threshold",
873   is.finite(min_main_retained) && min_main_retained >= config$min_retained_vote_share,
874   fatal = TRUE,
875   n_failed = sum(bridge_diagnostics$retained_vote_share < config$min_retained_vote_share, na.rm = TRUE),
876   details = paste0("Minimum retained share in main years: ", signif(min_main_retained, 6))
877 ),
878 validation_check(
879   "main_panel_balanced",
880   main_missing_n == 0 || (
881     !isTRUE(config$require_balanced_panel) &&
882     is.finite(main_missing_share) &&
883     main_missing_share <= config$max_missing_cz_year_share
884   ),
885   fatal = isTRUE(config$require_balanced_panel) || config$max_missing_cz_year_share <= 0,
886   n_failed = main_missing_n,
887   details = paste0(
888     "Observed complete rows: ", nrow(analysis_panel), "; expected rows: ", main_expected_n,
889     "; missing share: ", signif(main_missing_share, 6)
890   )
891 )
892 )
893 readr::write_csv(checks, config$validation_checks_csv)
894
895 sources <- list(

```



```

896 adh_stack = list(path = adh_stack_path, md5 = file_checksum(adh_stack_path)),
897 adh_long = list(path = adh_long_path, md5 = file_checksum(adh_long_path)),
898 cz_lookup = list(path = cz_path, md5 = file_checksum(cz_path)),
899 ftz_zip = list(path = ftz_zip, md5 = file_checksum(ftz_zip)),
900 aa_presidential = list(path = aa_path, md5 = file_checksum(aa_path))
901 )
902 manifest <- write_pipeline_manifest(
903   config = config,
904   checks = checks,
905   stage = "build_analysis_data",
906   sources = sources,
907   extra = list(
908     main_expected_rows = main_expected_n,
909     main_observed_rows = nrow(analysis_panel),
910     main_missing_rows = main_missing_n,
911     main_missing_share = main_missing_share,
912     min_main_retained_vote_share = min_main_retained
913   )
914 )
915
916 failed <- checks %>%
917   dplyr::filter(fatal, status == "fail")
918
919 if (nrow(failed) > 0 && isTRUE(stop_on_fatal) && !isTRUE(config$allow_failed_diagnostics)) {
920   failed_msg <- paste(
921     paste0(
922       failed$check,
923       " (n_failed = ", failed$n_failed, "): ",
924       failed$details
925     ),
926     collapse = "\n"
927   )
928
929   detail_parts <- character(0)
930   if ("bridged_vote_retention_above_threshold" %in% failed$check) {
931     failing_years <- bridge_diagnostics %>%
932       dplyr::filter(year %in% config$event_years, retained_vote_share < config$min_retained_vote_share) %>%
933       dplyr::transmute(piece = paste0(year, " retained_vote_share=", signif(retained_vote_share, 6))) %>%
934       dplyr::pull(piece)
935     if (length(failing_years) > 0) {
936       detail_parts <- c(detail_parts, paste0("Failing retention years: ", paste(failing_years, collapse = "; ")))
937     }
938   }
939   if ("main_panel_balanced" %in% failed$check) {
940     missing_main <- missing_cz_years %>%
941       dplyr::filter(sample == "main_1972_start") %>%
942       dplyr::mutate(piece = paste0("CZ ", czone, " in ", year)) %>%
943       dplyr::pull(piece)
944     if (length(missing_main) > 0) {
945       shown <- head(missing_main, 30)
946       suffix <- if (length(missing_main) > length(shown)) {
947         paste0("; ... +", length(missing_main) - length(shown), " more")
948       } else {
949         ""
950       }
951       detail_parts <- c(detail_parts, paste0("Missing main-panel CZ-years: ", paste(shown, collapse = "; "), suffix))
952     }
953   }
954   if ("crosswalk_positive_final_weight_sums_equal_one" %in% failed$check) {
955     bad_crosswalks <- crosswalk_source_diagnostics %>%
956       dplyr::filter(positive_targets > 0, !final_weight_sum_ok) %>%
957       dplyr::mutate(piece = paste0(
958         source_decade, " county ", county_fips_source,
959         " final_sum=", signif(crosswalk_weight_sum, 6)
960       )) %>%
961       dplyr::pull(piece)
962     if (length(bad_crosswalks) > 0) {
963       shown <- head(bad_crosswalks, 30)
964       suffix <- if (length(bad_crosswalks) > length(shown)) {
965         paste0("; ... +", length(bad_crosswalks) - length(shown), " more")
966       } else {
967         ""
968       }
969       detail_parts <- c(detail_parts, paste0("Crosswalk source counties with invalid positive final sums: ", paste(shown, collapse = "; "
970         ↵ " ), suffix))
971     }
972   }
973   detail_msg <- if (length(detail_parts) > 0) {
974     paste0("\n\nAdditional failing rows:\n", paste(detail_parts, collapse = "\n"))
975   } else {
976     ""
977   }
978
979   stop(
980     "Fatal validation checks failed before estimation:\n",
981     failed_msg,
982     detail_msg,
983     "\nSee ", config$pipeline_manifest_json,
984     call. = FALSE

```

```
985     )
986   }
987
988   invisible(list(
989     config = config,
990     checks = checks,
991     manifest = manifest,
992     analysis_panel = analysis_panel,
993     analysis_panel_full = analysis_panel_full,
994     diagnostic_panel = diagnostic_panel,
995     diagnostic_panel_full = diagnostic_panel_full,
996     crosswalks = crosswalks
997   ))
998 }
```

4 03_estimate_event_study.R

```

1 # extension-adh2013/src/03_estimate_event_study.R
2
3 add_event_study_terms <- function(data, controls, config) {
4   years <- sort(unique(data$year))
5   years_est <- setdiff(years, config$reference_year)
6   event_vars <- paste0("es_", years_est)
7   for (i in seq_along(years_est)) {
8     data[[event_vars[[i]]]] <- data[[config$exposure_var]] * as.integer(data$year == years_est[[i]])
9   }
10  ctrl_build <- make_interacted_control_vars(data, controls, years, config$reference_year)
11  list(data = ctrl_build$data, event_vars = event_vars, control_vars = ctrl_build$vars)
12 }
13
14 model_control_names <- function(controls, standardize = FALSE) {
15   if (isTRUE(standardize)) standardized_control_name(controls) else controls
16 }
17
18 interacted_control_variance_diagnostics <- function(prepared_data, control_vars, sample_name, spec_name) {
19   if (length(control_vars) == 0) return(tibble::tibble())
20   purrr::map_dfr(control_vars, function(v) {
21     vals <- prepared_data[[v]]
22     tibble::tibble(
23       sample = sample_name,
24       spec = spec_name,
25       interacted_control = v,
26       n_nonmissing = sum(!is.na(vals)),
27       variance = stats::var(vals, na.rm = TRUE),
28       min = suppressWarnings(min(vals, na.rm = TRUE)),
29       max = suppressWarnings(max(vals, na.rm = TRUE)),
30       share_zero = mean(vals == 0, na.rm = TRUE)
31     )
32   })
33 }
34
35 vcov_rank_summary <- function(vcov_mat, sample_name, spec_name, se_type) {
36   if (is.null(vcov_mat) || any(!is.finite(vcov_mat))) {
37     return(tibble::tibble(
38       sample = sample_name, spec = spec_name, se_type = se_type,
39       vcov_dim = NA_integer_, vcov_rank = NA_integer_, vcov_rank_deficiency = NA_integer_,
40       min_diag = NA_real_, max_diag = NA_real_, median_diag = NA_real_
41     ))
42   }
43   q <- qr(vcov_mat, tol = 1e-10)
44   d <- diag(vcov_mat)
45   tibble::tibble(
46     sample = sample_name, spec = spec_name, se_type = se_type,
47     vcov_dim = nrow(vcov_mat), vcov_rank = q$rank,
48     vcov_rank_deficiency = nrow(vcov_mat) - q$rank,
49     min_diag = suppressWarnings(min(d, na.rm = TRUE)),
50     max_diag = suppressWarnings(max(d, na.rm = TRUE)),
51     median_diag = suppressWarnings(stats::median(d, na.rm = TRUE))
52   )
53 }
54
55 residualized_design_diagnostics <- function(data, rhs_terms, config, sample_name, spec_name) {
56   x <- as.matrix(data[, rhs_terms, drop = FALSE])
57   fe <- stats::model.matrix(~ factor(czone) + factor(year) - 1, data = data)
58   w <- data[[config$weight_var]]
59   w <- ifelse(is.finite(w) & w > 0, sqrt(w), 1)
60   xw <- x * w
61   few <- fe * w
62   fit <- stats::lm.fit(few, xw)
63   resid_x <- fit$residuals
64   qr_obj <- qr(resid_x, tol = 1e-7)
65   singular_values <- tryCatch(svd(resid_x, nu = 0, nv = 0)$d, error = function(e) numeric(0))
66   positive_sv <- singular_values[singular_values > 1e-10]
67   condition_number <- if (length(positive_sv) == 0) NA_real_ else max(positive_sv) / min(positive_sv)
68   tibble::tibble(
69     sample = sample_name,
70     spec = spec_name,
71     n_obs = nrow(data),
72     n_cz = dplyr::n_distinct(data$czone),
73     n_years = dplyr::n_distinct(data$year),
74     n_rhs_terms = length(rhs_terms),
75     residualized_rank = qr_obj$rank,
76     residualized_rank_deficiency = length(rhs_terms) - qr_obj$rank,
77     condition_number = condition_number,
78     min_singular_value = if (length(singular_values) == 0) NA_real_ else min(singular_values),
79     max_singular_value = if (length(singular_values) == 0) NA_real_ else max(singular_values)
80   )
81 }
82
83 vcov_eigen_summary <- function(vcov_mat) {
84   if (is.null(vcov_mat) || any(!is.finite(vcov_mat))) {
85     return(list(
86       eigenvalues = NA_real_, min_eigenvalue = NA_real_,
87       max_eigenvalue = NA_real_, repair_required = TRUE,

```

```

88     n_negative_eigenvalues = NA_integer_,
89     eigenvalue_ratio_abs_min_to_max = NA_real_,
90     nonfinite = TRUE
91   ))
92 }
93 ev <- eigen((vcov_mat + t(vcov_mat)) / 2, symmetric = TRUE, only.values = TRUE)$values
94 min_ev <- min(ev)
95 max_ev <- max(ev)
96 list(
97   eigenvalues = ev,
98   min_eigenvalue = min_ev,
99   max_eigenvalue = max_ev,
100   repair_required = min_ev < -1e-8,
101   n_negative_eigenvalues = sum(ev < 0),
102   eigenvalue_ratio_abs_min_to_max = if (is.finite(max_ev) && max_ev > 0) abs(min_ev) / max_ev else NA_real_,
103   nonfinite = FALSE
104 )
105 }
106
107 extract_event_coefficients <- function(model, vcov_mat, spec_name, sample_name, se_type,
108                                       repair_required, vcov_repaired) {
109   ct <- fixest::coefstable(model, vcov = vcov_mat)
110   ct_df <- data.frame(term = rownames(ct), ct, row.names = NULL, check.names = FALSE)
111   names(ct_df)[names(ct_df) == "Estimate"] <- "estimate"
112   names(ct_df)[names(ct_df) == "Std. Error"] <- "std.error"
113   if ("Pr(>|t|)" %in% names(ct_df)) names(ct_df)[names(ct_df) == "Pr(>|t|)"] <- "p.value"
114   if ("Pr(>|z|)" %in% names(ct_df)) names(ct_df)[names(ct_df) == "Pr(>|z|)"] <- "p.value"
115   if ("t value" %in% names(ct_df)) names(ct_df)[names(ct_df) == "t value"] <- "statistic"
116   if ("z value" %in% names(ct_df)) names(ct_df)[names(ct_df) == "z value"] <- "statistic"
117   if (!("p.value" %in% names(ct_df))) ct_df$p.value <- NA_real_
118   if (!("statistic" %in% names(ct_df))) ct_df$statistic <- NA_real_
119
120   tibble::as_tibble(ct_df) %>%
121     dplyr::filter(grepl("^es_", term)) %>%
122     dplyr::mutate(
123       sample = sample_name,
124       spec = spec_name,
125       se_type = se_type,
126       year = as.integer(sub("^es_", "", term)),
127       conf.low = estimate - stats::qnorm(0.975) * std.error,
128       conf.high = estimate + stats::qnorm(0.975) * std.error,
129       vcov_repair_required = repair_required,
130       vcov_repaired = vcov_repaired
131     ) %>%
132     dplyr::select(
133       sample, spec, se_type, term, year, estimate, std.error, conf.low, conf.high,
134       statistic, p.value, vcov_repair_required, vcov_repaired
135     ) %>%
136     dplyr::arrange(sample, spec, se_type, year)
137 }
138
139 vcov_specifications <- function(config) {
140   conley_specs <- purrr::map(config$conley_cutoffs_km, function(cutoff) {
141     list(
142       se_type = paste0("conley_", cutoff, "km"),
143       compute = function(model, repair) {
144         fixest::vcov_conley(
145           model,
146           lat = "lat",
147           lon = "lon",
148           cutoff = cutoff,
149           distance = config$conley_distance,
150           vcov_fix = repair
151         )
152       }
153     )
154   })
155   c(
156     list(
157       list(
158         se_type = "heteroskedastic",
159         compute = function(model, repair) fixest::vcov_hetero(model, vcov_fix = repair)
160       ),
161       list(
162         se_type = "cluster_cz",
163         compute = function(model, repair) {
164           fixest::vcov_cluster(model, cluster = stats::as.formula("~czone"), vcov_fix = repair)
165         }
166       ),
167       list(
168         se_type = "cluster_state",
169         compute = function(model, repair) {
170           fixest::vcov_cluster(model, cluster = stats::as.formula("~statefip"), vcov_fix = repair)
171         }
172       ),
173       list(
174         se_type = "cluster_cz_year",
175         compute = function(model, repair) {
176           fixest::vcov_cluster(model, cluster = stats::as.formula("~czone + year"), vcov_fix = repair)
177         }
178       )
179     )
180   )

```

```

178     ),
179     list(
180       se_type = paste0("newey_west_lag", config$nw_lag),
181       compute = function(model, repair) {
182         fixest::vcov_NW(
183           model, unit = "czone", time = "election_index",
184           lag = config$nw_lag, vcov_fix = repair
185         )
186       }
187     ),
188     list(
189       se_type = paste0("driscoll_kraay_lag", config$dk_lag),
190       compute = function(model, repair) {
191         fixest::vcov_DK(model, time = "election_index", lag = config$dk_lag, vcov_fix = repair)
192       }
193     )
194   ),
195   conley_specs
196 )
197 }
198
199 compute_vcov_outputs <- function(model, spec_name, sample_name, config) {
200   specs <- vcov_specifications(config)
201   coefficients <- list()
202   diagnostics <- list()
203   eigenvalues <- list()
204   vcov_mats <- list()
205
206   for (s in specs) {
207     se_type <- s$se_type
208     raw <- tryCatch(s$compute(model, FALSE), error = function(e) e)
209     if (inherits(raw, "error")) {
210       diagnostics[[se_type]] <- tibble::tibble(
211         sample = sample_name, spec = spec_name, se_type = se_type,
212         status = "failed", error_message = conditionMessage(raw),
213         min_eigenvalue = NA_real_, max_eigenvalue = NA_real_,
214         n_negative_eigenvalues = NA_integer_,
215         eigenvalue_ratio_abs_min_to_max = NA_real_,
216         repair_required = TRUE, vcov_repaired = FALSE
217       )
218     }
219     next
220   }
221
222   eig <- vcov_eigen_summary(raw)
223   eigenvalues[[se_type]] <- tibble::tibble(
224     sample = sample_name,
225     spec = spec_name,
226     se_type = se_type,
227     eigen_index = seq_along(eig$eigenvalues),
228     eigenvalue = eig$eigenvalues
229   )
230
231   use_vcov <- raw
232   repaired <- FALSE
233   status <- "ok"
234   err <- NA_character_
235
236   if (isTRUE(eig$repair_required)) {
237     status <- "repair_required"
238     err <- "Unrepaired VCOV is not positive semidefinite."
239     if (isTRUE(config$allow_vcov_repair)) {
240       repaired_try <- tryCatch(s$compute(model, TRUE), error = function(e) e)
241       if (inherits(repaired_try, "error")) {
242         status <- "failed"
243         err <- conditionMessage(repaired_try)
244       } else {
245         use_vcov <- repaired_try
246         repaired <- TRUE
247         status <- "repaired"
248       }
249     }
250   }
251
252   diagnostics[[se_type]] <- tibble::tibble(
253     sample = sample_name, spec = spec_name, se_type = se_type,
254     status = status, error_message = err,
255     min_eigenvalue = eig$min_eigenvalue, max_eigenvalue = eig$max_eigenvalue,
256     n_negative_eigenvalues = eig$n_negative_eigenvalues,
257     eigenvalue_ratio_abs_min_to_max = eig$eigenvalue_ratio_abs_min_to_max,
258     repair_required = eig$repair_required, vcov_repaired = repaired
259   )
260
261   # Write coefficient rows for every successfully computed VCOV, including
262   # unrepaired VCOVs flagged as repair_required. This keeps the all-spec CSV
263   # complete for diagnostics (especially all Conley cutoffs/specifications),
264   # while the vcov_repair_required/vcov_repaired flags make clear which rows
265   # should not be treated as publication-ready inference.
266   if (status %in% c("ok", "repaired", "repair_required")) {
267     coef_tbl <- tryCatch(
268       extract_event_coefficients(

```

```

268     model = model, vcov_mat = use_vcov, spec_name = spec_name,
269     sample_name = sample_name, se_type = se_type,
270     repair_required = eig$repair_required, vcov_repaired = repaired
271   ),
272   error = function(e) e
273 )
274 if (inherits(coef_tbl, "error")) {
275   diagnostics[[se_type]]$status <- "coeftable_failed"
276   diagnostics[[se_type]]$error_message <- conditionMessage(coef_tbl)
277 } else {
278   coefficients[[se_type]] <- coef_tbl
279   vcov_mats[[se_type]] <- use_vcov
280 }
281 }
282 }
283
284 list(
285   coefficients = dplyr::bind_rows(coefficients),
286   diagnostics = dplyr::bind_rows(diagnostics),
287   eigenvalues = dplyr::bind_rows(eigenvalues),
288   vcov = vcov_mats
289 )
290 }
291
292 controls_manifest_for_spec <- function(sample_name, spec_name, controls, model_controls, standardize_controls, config, model) {
293   dropped <- model$collin.var %||% character(0)
294   tibble::tibble(control = config$baseline_controls) %>%
295     dplyr::mutate(
296       sample = sample_name,
297       spec = spec_name,
298       requested = control %in% controls,
299       standardize_controls = isTRUE(standardize_controls),
300       model_control = dplyr::if_else(requested, model_control_names(control, standardize_controls), NA_character_),
301       status = dplyr::case_when(
302         !requested ~ "omitted_from_spec",
303         vapply(model_control, function(ctrl) !is.na(ctrl) && any(startsWith(dropped, paste0("ctrl_", ctrl, "_"))), logical(1)) ~
304           "dropped_by_fixest",
305         TRUE ~ "included"
306       )
307     ) %>%
308     dplyr::select(sample, spec, control, requested, standardize_controls, model_control, status)
309 }
310
311 estimate_one_spec <- function(data, sample_name, spec_name, controls, controls_label, config, standardize_controls = FALSE) {
312   model_controls <- model_control_names(controls, standardize_controls)
313   missing_model_controls <- setdiff(model_controls, names(data))
314   if (length(missing_model_controls) > 0) {
315     stop("Missing model control variables: ", paste(missing_model_controls, collapse = ", "))
316   }
317   prepared <- add_event_study_terms(data, model_controls, config)
318   rhs_terms <- c(prepared$event_vars, prepared$control_vars)
319   formula_chr <- paste0(config$outcome_var, " ~ ", paste(rhs_terms, collapse = " + "), " | czone + year")
320   design_diag <- residualized_design_diagnostics(prepared$data, rhs_terms, config, sample_name, spec_name)
321
322   tryCatch({
323     message("Estimating ", sample_name, " / ", spec_name)
324     model <- fixest::feols(
325       fml = stats::as.formula(formula_chr),
326       data = prepared$data,
327       weights = stats::as.formula(paste0("~", config$weight_var)),
328       panel.id = stats::as.formula("~czone + election_index"),
329       warn = TRUE,
330       notes = FALSE
331     )
332     vcovs <- compute_vcov_outputs(model, spec_name, sample_name, config)
333     main_vcov_diag <- vcovs$diagnostics %>% dplyr::filter(se_type == config$main_se_type)
334     main_vcov_failed <- nrow(main_vcov_diag) == 0 ||
335       main_vcov_diag$status %in% c("failed", "repair_required") ||
336       (isTRUE(config$require_no_vcov_repair_for_main) && isTRUE(main_vcov_diag$repair_required[[1]]))
337     status <- if (main_vcov_failed) "failed" else "ok"
338     error_message <- if (main_vcov_failed) {
339       paste0("Main SE ", config$main_se_type, " failed or required VCOV repair.")
340     } else {
341       NA_character_
342     }
343
344     model_stats <- list(
345       "Observations" = format(nrow(prepared$data), big.mark = ","),
346       "Commuting zones" = format(dplyr::n_distinct(prepared$data$czone), big.mark = ","),
347       "Election years" = paste0(min(prepared$data$year), "--", max(prepared$data$year)),
348       "Reference election year" = as.character(config$reference_year),
349       "CZ fixed effects" = "Yes",
350       "Election-year fixed effects" = "Yes",
351       "Controls" = controls_label,
352       "Controls standardized" = if (isTRUE(standardize_controls)) "Yes" else "No",
353       "Sample" = sample_name,
354       "ADH weights" = "timepwt48",
355       "Main SE type" = config$main_se_type
356     )
357

```

```

358   list(
359     sample = sample_name, spec = spec_name, status = status, error_message = error_message,
360     formula_chr = formula_chr, model = model, vcov = vcovs$vcov,
361     coefficients = vcovs$coefficients, vcov_diagnostics = vcovs$diagnostics,
362     vcov_eigenvalues = vcovs$eigenvalues, model_diagnostics = design_diag,
363     interacted_control_variance = interacted_control_variance_diagnostics(prepared$data, prepared$control_vars, sample_name, spec_name),
364     controls_manifest = controls_manifest_for_spec(sample_name, spec_name, controls, model_controls, standardize_controls, config,
365       ↪ model),
366     model_stats = model_stats, n_obs = nrow(prepared$data),
367     n_cz = dplyr::n_distinct(prepared$data$czone), min_year = min(prepared$data$year),
368     max_year = max(prepared$data$year)
369   ), error = function(e) {
370     list(
371       sample = sample_name, spec = spec_name, status = "failed",
372       error_message = conditionMessage(e), formula_chr = formula_chr,
373       model = NULL, vcov = list(), coefficients = tibble::tibble(),
374       vcov_diagnostics = tibble::tibble(), vcov_eigenvalues = tibble::tibble(),
375       model_diagnostics = design_diag, interacted_control_variance = tibble::tibble(), controls_manifest = tibble::tibble(),
376       model_stats = list(
377         "Observations" = format(nrow(prepared$data), big.mark = ","),
378         "Commuting zones" = format(dplyr::n_distinct(prepared$data$czone), big.mark = ","),
379         "Election years" = paste0(min(prepared$data$year), "--", max(prepared$data$year)),
380         "Reference election year" = as.character(config$reference_year),
381         "CZ fixed effects" = "Yes",
382         "Election-year fixed effects" = "Yes",
383         "Controls" = controls_label,
384         "Controls standardized" = if (isTRUE(standardize_controls)) "Yes" else "No",
385         "Sample" = sample_name,
386         "ADH weights" = "timepwt48",
387         "Main SE type" = "Failed"
388       ),
389       n_obs = nrow(prepared$data), n_cz = dplyr::n_distinct(prepared$data$czone),
390       min_year = min(prepared$data$year), max_year = max(prepared$data$year)
391     )
392   })
393 }
394
395 run_pretrend_tests <- function(results, config) {
396   purrr::map_dfr(results, function(res) {
397     if (is.null(res$model) || res$sample != "main_1972_start") return(tibble::tibble())
398     main_vcov <- res$vcov[[config$main_se_type]]
399     if (is.null(main_vcov)) return(tibble::tibble())
400     pre_years <- sort(unique(res$coefficients$year[res$coefficients$year < config$reference_year]))
401     if (length(pre_years) == 0) return(tibble::tibble())
402     keep_pattern <- paste0("~es(", paste(pre_years, collapse = "|"), ")$")
403     wt <- tryCatch(
404       fixest::wald(res$model, keep = keep_pattern, vcov = main_vcov, print = FALSE),
405       error = function(e) e
406     )
407     if (inherits(wt, "error")) {
408       tibble::tibble(
409         sample = res$sample, spec = res$spec, se_type = config$main_se_type,
410         pre_years = paste(pre_years, collapse = ","),
411         statistic = NA_real_, p.value = NA_real_, df1 = NA_real_, df2 = NA_real_,
412         status = "failed", error_message = conditionMessage(wt)
413       )
414     } else {
415       tibble::tibble(
416         sample = res$sample, spec = res$spec, se_type = config$main_se_type,
417         pre_years = paste(pre_years, collapse = ","),
418         statistic = wt$stat, p.value = wt$p, df1 = wt$df1, df2 = wt$df2,
419         status = "ok", error_message = NA_character_
420       )
421     }
422   })
423 }
424
425 estimate_event_study <- function(config = CONFIG, stop_on_fatal = TRUE) {
426   load_required_packages(config)
427   config <- finalize_config(config)
428
429   existing_checks <- if (file.exists(config$validation_checks_csv)) {
430     readr::read_csv(config$validation_checks_csv, show_col_types = FALSE)
431   } else {
432     tibble::tibble()
433   }
434   if (nrow(existing_checks) > 0 && has_fatal_failures(existing_checks) && isTRUE(stop_on_fatal)) {
435     stop("Fatal build-stage validation checks are present. Rebuild with a nonfatal policy before estimating.", call. = FALSE)
436   }
437
438   main_panel <- readRDS(config$analysis_panel_rds)
439   diagnostic_panel <- readRDS(config$diagnostic_panel_rds)
440
441   spec_definitions <- list(
442     minimal_full_panel = list(
443       controls = character(0),
444       standardize_controls = FALSE,
445       label = "None"
446     ),

```

```

447 interacted_core_controls_diagnostic = list(
448   controls = config$core_interacted_controls,
449   standardize_controls = isTRUE(config$standardize_interacted_controls),
450   label = "Three 1990 ADH controls interacted with election-year indicators"
451 ),
452 interacted_controls_full_panel = list(
453   controls = config$interacted_controls,
454   standardize_controls = isTRUE(config$standardize_interacted_controls),
455   label = "All configured 1990 ADH baseline controls interacted with election-year indicators"
456 ),
457 interacted_controls_full_panel_raw_controls_diagnostic = list(
458   controls = config$interacted_controls,
459   standardize_controls = FALSE,
460   label = "All configured 1990 ADH baseline controls interacted with election-year indicators (raw-control diagnostic)"
461 )
462 )
463
464 samples <- list(
465   main_1972_start = main_panel,
466   diagnostic_1952_start = diagnostic_panel
467 )
468
469 all_results <- list()
470 for (sample_name in names(samples)) {
471   for (spec_name in names(spec_definitions)) {
472     def <- spec_definitions[[spec_name]]
473     key <- paste(sample_name, spec_name, sep = "__")
474     all_results[[key]] <- estimate_one_spec(
475       data = samples[[sample_name]],
476       sample_name = sample_name,
477       spec_name = spec_name,
478       controls = def$controls,
479       controls_label = def$label,
480       config = config,
481       standardize_controls = isTRUE(def$standardize_controls)
482     )
483   }
484 }
485
486 status_tbl <- purrr::map_dfr(all_results, ~ tibble::tibble(
487   sample = .x$sample, spec = .x$spec, status = .x$status, error_message = .x$error_message,
488   n_obs = .x$n_obs, n_cz = .x$n_cz, min_year = .x$min_year, max_year = .x$max_year,
489   formula_chr = .x$formula_chr
490 ))
491 coef_tbl_all <- purrr::map_dfr(all_results, ~ .x$coefficients)
492 model_diagnostics <- purrr::map_dfr(all_results, ~ {
493   dropped <- if (is.null(.x$model)) character(0) else (.x$model$collin.var %||% character(0))
494   .x$model_diagnostics %>%
495     dplyr::mutate(
496       fixest_dropped_variables = paste(dropped, collapse = "; "),
497       fixest_dropped_count = length(dropped)
498     )
499 })
500 controls_manifest <- purrr::map_dfr(all_results, ~ .x$controls_manifest)
501 interacted_control_variance <- purrr::map_dfr(all_results, ~ .x$interacted_control_variance %||% tibble::tibble())
502 vcov_diagnostics <- purrr::map_dfr(all_results, ~ .x$vcov_diagnostics)
503 vcov_eigenvalues <- purrr::map_dfr(all_results, ~ .x$vcov_eigenvalues)
504 vcov_rank_diagnostics <- purrr::imap_dfr(all_results, function(res, nm) {
505   if (is.null(res$vcov) || length(res$vcov) == 0) return(tibble::tibble())
506   purrr::imap_dfr(res$vcov, ~ vcov_rank_summary(.x, res$sample, res$spec, .y))
507 })
508 se_warning_diagnostics <- coef_tbl_all %>%
509   dplyr::group_by(sample, spec, se_type) %>%
510   dplyr::summarise(
511     n_event_coefficients = dplyr::n(),
512     min_std_error = min(std.error, na.rm = TRUE),
513     median_std_error = stats::median(std.error, na.rm = TRUE),
514     max_std_error = max(std.error, na.rm = TRUE),
515     near_zero_se_count = sum(abs(std.error) < 1e-10, na.rm = TRUE),
516     nonfinite_se_count = sum(!is.finite(std.error)),
517     groups = "drop"
518   ) %>%
519   dplyr::left_join(
520     vcov_rank_diagnostics %>%
521     dplyr::select(sample, spec, se_type, vcov_dim, vcov_rank, vcov_rank_deficiency),
522     by = c("sample", "spec", "se_type")
523   ) %>%
524   dplyr::left_join(
525     vcov_diagnostics %>%
526     dplyr::select(sample, spec, se_type, vcov_status = status, repair_required, vcov_repaired, min_eigenvalue),
527     by = c("sample", "spec", "se_type")
528   ) %>%
529   dplyr::mutate(
530     diagnostic_note = dplyr::case_when(
531       grepl("driscoll", se_type) & near_zero_se_count > 0 & dplyr::coalesce(vcov_rank_deficiency, 0L) > 0 ~
532         "Suspicious near-zero Driscoll-Kraay SEs and rank-deficient VCOV; do not report without further diagnosis.",
533       grepl("driscoll", se_type) & near_zero_se_count > 0 ~
534         "Suspicious near-zero Driscoll-Kraay SEs; do not report without further diagnosis.",
535       grepl("conley", se_type) & (dplyr::coalesce(repair_required, FALSE) | dplyr::coalesce(vcov_repaired, FALSE)) ~
536         "Conley VCOV is non-positive-definite before repair or requires numerical repair; diagnostic only unless separately justified.",

```



```

537     dplyr::coalesce(vcov_repaired, FALSE) ~
538     "VCOV required numerical repair; diagnostic only unless separately justified.",
539     dplyr::coalesce(vcov_rank_deficiency, 0L) > 0 ~
540     "VCOV is rank deficient; use as diagnostic only unless justified.",
541     grepl("cluster_cz_year", se_type) ~
542     "Two-way clustering includes very few year clusters; use as diagnostic only.",
543     TRUE ~ NA_character_
544   )
545 )
546 pretrend_tests <- run_pretrend_tests(all_results, config)
547 pretrend_coefficients <- coef_tbl_all %>%
548   dplyr::filter(sample == "main_1972_start", se_type == config$main_se_type, year < config$reference_year)
549
550 rank_failures <- model_diagnostics %>%
551   dplyr::filter(residualized_rank_deficiency > 0)
552 main_vcov_problem_count <- vcov_diagnostics %>%
553   dplyr::filter(
554     sample == "main_1972_start",
555     spec %in% c("minimal_full_panel", "interacted_controls_full_panel"),
556     se_type == config$main_se_type,
557     !status %in% c("ok", "repaired")
558   ) %>%
559   nrow()
560 main_vcov_repair_count <- vcov_diagnostics %>%
561   dplyr::filter(
562     sample == "main_1972_start",
563     spec %in% c("minimal_full_panel", "interacted_controls_full_panel"),
564     se_type == config$main_se_type,
565     repair_required %in% TRUE
566   ) %>%
567   nrow()
568 conley_repair_count <- vcov_diagnostics %>%
569   dplyr::filter(grepl("^conley_", se_type), repair_required %in% TRUE) %>%
570   nrow()
571
572 build_checks <- if (file.exists(config$validation_checks_csv)) {
573   readr::read_csv(config$validation_checks_csv, show_col_types = FALSE)
574 } else {
575   tibble::tibble()
576 }
577 estimation_checks <- dplyr::bind_rows(
578   validation_check(
579     "event_study_residualized_design_full_rank",
580     nrow(rank_failures) == 0,
581     fatal = TRUE,
582     n_failed = nrow(rank_failures),
583     details = "Requires no residualized RHS rank deficiency in 1952-start or 1972-start specifications."
584   ),
585   validation_check(
586     "event_study_main_vcov_available_without_repair",
587     main_vcov_problem_count == 0 &&
588     (!isTRUE(config$require_no_vcov_repair_for_main) || main_vcov_repair_count == 0),
589     fatal = TRUE,
590     n_failed = main_vcov_problem_count + main_vcov_repair_count,
591     details = paste0("Main SE type: ", config$main_se_type)
592   ),
593   validation_check(
594     "event_study_conley_repair_diagnostics_recorded",
595     TRUE,
596     fatal = FALSE,
597     n_failed = conley_repair_count,
598     details = paste0("Conley VCOVs requiring repair across all samples/specs/cutoffs: ", conley_repair_count)
599   )
600 )
601 combined_checks <- dplyr::bind_rows(build_checks, estimation_checks)
602 readr::write_csv(combined_checks, config$validation_checks_csv)
603
604 write_model_object(all_results, config$all_specs_rds, config)
605 readr::write_csv(status_tbl, config$spec_status_csv)
606 readr::write_csv(coef_tbl_all, config$all_specs_csv)
607 readr::write_csv(model_diagnostics, config$model_diagnostics_csv)
608 readr::write_csv(controls_manifest, config$controls_manifest_csv)
609 readr::write_csv(interacted_control_variance, config$interacted_control_variance_csv)
610 readr::write_csv(vcov_diagnostics, config$vcov_diagnostics_csv)
611 readr::write_csv(vcov_eigenvalues, config$vcov_eigenvalues_csv)
612 readr::write_csv(vcov_rank_diagnostics, config$vcov_rank_csv)
613 readr::write_csv(se_warning_diagnostics, config$se_warning_diagnostics_csv)
614 readr::write_csv(summarise_conley_neighbors(main_panel, config$conley_cutoffs_km), config$conley_neighbor_counts_csv)
615 readr::write_csv(pretrend_tests, config$pretrend_tests_csv)
616 readr::write_csv(pretrend_coefficients, config$pretrend_coefficients_csv)
617
618 main_display_specs <- c("minimal_full_panel", "interacted_core_controls_diagnostic", "interacted_controls_full_panel")
619 main_coefficients <- coef_tbl_all %>%
620   dplyr::filter(
621     sample == "main_1972_start",
622     se_type == config$main_se_type,
623     spec %in% main_display_specs
624   ) %>%
625   dplyr::arrange(factor(spec, levels = main_display_specs), year)
626 main_models <- all_results[grepl("^main_1972_start_", names(all_results))]

```

```

627 main_out <- list(
628   coefficients = main_coefficients,
629   main_se_type = config$main_se_type,
630   display_specs = main_display_specs,
631   models = main_models
632 )
633 write_model_object(main_out, config$event_study_rds, config)
634 readr::write_csv(main_coefficients, config$event_study_csv)
635
636 failed_main_vcov <- vcov_diagnostics %>%
637   dplyr::filter(
638     sample == "main_1972_start",
639     spec %in% c("minimal_full_panel", "interacted_controls_full_panel"),
640     se_type == config$main_se_type,
641     !status %in% c("ok", "repaired")
642   )
643
644 write_pipeline_manifest(
645   config = config,
646   checks = combined_checks,
647   stage = "estimate_event_study",
648   sources = list(
649     analysis_panel = list(path = config$analysis_panel_rds, md5 = file_checksum(config$analysis_panel_rds)),
650     diagnostic_panel = list(path = config$diagnostic_panel_rds, md5 = file_checksum(config$diagnostic_panel_rds))
651   ),
652   extra = list(
653     specs = status_tbl,
654     conley_repair_count = conley_repair_count,
655     main_vcov_problem_count = main_vcov_problem_count,
656     main_vcov_repair_count = main_vcov_repair_count
657   )
658 )
659
660 if (nrow(failed_main_vcov) > 0 && isTRUE(stop_on_fatal) && !isTRUE(config$allow_failed_diagnostics)) {
661   stop(
662     "Main event-study VCOV failed or required repair. See ",
663     config$vcov_diagnostics_csv,
664     call. = FALSE
665   )
666 }
667
668 if (has_fatal_failures(combined_checks) && isTRUE(stop_on_fatal) && !isTRUE(config$allow_failed_diagnostics)) {
669   stop("Fatal event-study validation checks failed. See ", config$validation_checks_csv, call. = FALSE)
670 }
671
672 invisible(list(config = config, results = all_results, status = status_tbl, coefficients = coef_tbl_all))
673 }

```

5 04_export_outputs.R

```

1 # extension-adh2013/src/04_export_outputs.R
2
3 export_event_study_outputs <- function(config = CONFIG) {
4   load_required_packages(config)
5   config <- finalize_config(config)
6
7   checks <- if (file.exists(config$validation_checks_csv)) {
8     readr::read_csv(config$validation_checks_csv, show_col_types = FALSE)
9   } else {
10     tibble::tibble()
11   }
12   if (nrow(checks) > 0 && has_fatal_failures(checks) && !isTRUE(config$allow_failed_diagnostics)) {
13     stop("Refusing to export figures/tables because fatal validation checks are present.", call. = FALSE)
14   }
15   has_validation_failures <- nrow(checks) > 0 && has_any_failures(checks)
16   has_reported_nonfatal_conditions <- nrow(checks) > 0 && has_reported_conditions(checks)
17
18   res_all <- read_model_object(config$all_specs_rds)
19   spec_order <- c("minimal_full_panel", "interacted_core_controls_diagnostic", "interacted_controls_full_panel")
20   pretty_spec_names <- c(
21     minimal_full_panel = "Minimal",
22     interacted_core_controls_diagnostic = "Core controls",
23     interacted_controls_full_panel = "Full controls"
24   )
25
26   ok_keys <- names(res_all)[vapply(res_all, function(x) {
27     x$sample == "main1972_start" && x$spec %in% spec_order && x$status == "ok"
28   }, logical(1))]
29   if (length(ok_keys) == 0) stop("No successful main specifications found in ", config$all_specs_rds)
30
31   coef_tbl_all <- purrr::map_dfr(ok_keys, function(k) {
32     res_all[[k]]$coefficients %>%
33     dplyr::filter(se_type == config$main_se_type) %>%
34     dplyr::mutate(spec_label = pretty_spec_names[spec])
35   })
36
37   table_csv <- coef_tbl_all %>%
38     dplyr::transmute(
39       sample, spec, spec_label, se_type, year, term, estimate, std.error,
40       conf.low, conf.high, statistic, p.value, vcov_repair_required, vcov_repaired
41     ) %>%
42     dplyr::arrange(factor(spec, levels = spec_order), year)
43   readr::write_csv(table_csv, config$table_csv)
44   uses_repaired_main_vcov <- any(table_csv$vcov_repaired %in% TRUE, na.rm = TRUE)
45
46   event_years_display <- sort(unique(c(coef_tbl_all$year, config$reference_year)))
47   coef_cell <- function(df, year_value) {
48     row <- df %>% dplyr::filter(year == year_value)
49     if (year_value == config$reference_year) return("--")
50     if (nrow(row) == 0) return("")
51     paste0(fmt_num(row$estimate[[1]]), star_code(row$p.value[[1]]), " (", fmt_num(row$std.error[[1]]), ")")
52   }
53
54   table_body <- tibble::tibble(`Election year` = as.character(event_years_display))
55   for (s in spec_order) {
56     df_s <- coef_tbl_all %>% dplyr::filter(spec == s)
57     if (nrow(df_s) > 0) {
58       table_body[[pretty_spec_names[[s]]]] <- vapply(event_years_display, function(y) {
59         coef_cell(df_s, y)
60       }, character(1))
61     }
62   }
63
64   stat_names <- c(
65     "Observations", "Commuting zones", "Election years", "Reference election year",
66     "CZ fixed effects", "Election-year fixed effects", "Controls", "Sample",
67     "ADH weights", "Controls standardized", "Main SE type"
68   )
69
70   get_stat_value <- function(ms, nm) {
71     if (is.null(ms) || is.null(ms[[nm]]) || length(ms[[nm]]) == 0) return("")
72     as.character(ms[[nm]][1])
73   }
74   stat_rows <- tibble::tibble(`Election year` = stat_names)
75   for (s in spec_order) {
76     key <- ok_keys[vapply(ok_keys, function(k) res_all[[k]]$spec == s, logical(1))]
77     if (length(key) > 0) {
78       vals <- vapply(stat_names, function(nm) get_stat_value(res_all[[key[[1]]]]$model_stats, nm), character(1))
79       stat_rows[[pretty_spec_names[[s]]]] <- vals
80     }
81   }
82
83   table_for_tex <- dplyr::bind_rows(table_body, stat_rows)
84   fallback_label <- dplyr::case_when(
85     config$crosswalk_missing_weight_policy == "fallback_m2" ~ "M2 fallback",
86     config$crosswalk_missing_weight_policy == "fallback_m4" ~ "M4 fallback",
87     config$crosswalk_missing_weight_policy == "identity_if_same_fips" ~ "same-FIPS identity fallback",
88     TRUE ~ config$crosswalk_missing_weight_policy

```

```

88   )
89   crosswalk_note <- if (identical(config$crosswalk_missing_weight_policy, "fail")) {
90     paste0(toupper(config$crosswalk_weight_slug), " weights with no fallback")
91   } else {
92     paste0(
93       toupper(config$crosswalk_weight_slug),
94       " weights where valid, with ", fallback_label,
95       " for source counties whose selected weights are undefined or invalid"
96     )
97   }
98
99   se_label <- dplyr::case_when(
100     config$main_se_type == "cluster_cz" ~ "CZ-clustered",
101     config$main_se_type == "heteroskedastic" ~ "heteroskedastic",
102     config$main_se_type == "cluster_state" ~ "state-clustered",
103     config$main_se_type == "newey_west" ~ "Newey-West",
104     TRUE ~ config$main_se_type
105   )
106
107   notes_text <- paste0(
108     if (uses_repaired_main_vcov || has_validation_failures) {
109       "PROVISIONAL: at least one fatal/non-handled validation check failed or a main numerical repair/warning was recorded; treat these
110       ↪ as diagnostic output until reviewed. "
111     } else {
112       ""
113     },
114     if (has_reported_nonfatal_conditions) {
115       "Handled nonfatal crosswalk-support conditions are reported in the diagnostics and do not block the ADH-mainland estimating sample.
116       ↪ "
117     } else {
118       ""
119     },
120     "Each coefficient is the interaction between ADH's 1990-2007 import-exposure measure and the indicated presidential-election year, ",
121     "with ", config$reference_year, " omitted as the reference year. Exposure is measured in ",
122     config$exposure_units, ". The dependent variable is the commuting-zone Republican presidential vote margin, ",
123     "defined as Republican minus Democratic votes divided by two-party votes. County presidential returns are from Amlani and Algara
124     ↪ (2021), ",
125     "bridged to 1990 counties with the Ferrara, Testa, and Zhou (2024) ",
126     crosswalk_note, ", then aggregated to 1990 commuting zones. ",
127     "Standard errors use ", se_label, ". Significance stars: *** p<0.01, ** p<0.05, * p<0.10."
128   )
129
130   latex_tbl <- knitr::kable(
131     table_for_tex,
132     format = "latex",
133     booktabs = TRUE,
134     longtable = TRUE,
135     escape = TRUE,
136     caption = paste0(
137       if (uses_repaired_main_vcov || has_validation_failures) "Provisional " else "",
138       "event-study estimates: ADH China exposure and Republican presidential vote margin"
139     ),
140     align = c("l", rep("c", ncol(table_for_tex) - 1))
141   ) %>%
142   kableExtra::kable_styling(
143     latex_options = c("repeat_header", "hold_position"),
144     position = "center",
145     font_size = 9
146   ) %>%
147   kableExtra::footnote(general = notes_text, threeparttable = TRUE, escape = TRUE)
148
149   writelines(as.character(latex_tbl), config$table_tex)
150   render_table_pdf(config$table_tex, config$table_standalone_tex, config$table_pdf)
151
152   plot_tbl <- coef_tbl_all %>%
153     dplyr::mutate(
154       spec_label = factor(spec_label, levels = c("Minimal", "Core controls", "Full controls")),
155       line_group = ifelse(year < config$reference_year, "pre", "post")
156     )
157
158   ref_rows <- plot_tbl %>%
159     dplyr::distinct(sample, spec, se_type, spec_label) %>%
160     dplyr::mutate(
161       term = paste0("es_", config$reference_year),
162       year = config$reference_year,
163       estimate = 0,
164       std.error = NA_real_,
165       conf.low = 0,
166       conf.high = 0,
167       statistic = NA_real_,
168       p.value = NA_real_,
169       vcov_repair_required = FALSE,
170       vcov_repaired = FALSE,
171       line_group = "reference"
172     )
173
174   plot_with_ref <- dplyr::bind_rows(plot_tbl, ref_rows)
175   line_tbl <- dplyr::bind_rows(
176     plot_with_ref %>%
177       dplyr::filter(year <= config$reference_year) %>%
178       dplyr::mutate(line_group = "pre"),
179     plot_with_ref %>%

```

```

175     dplyr::filter(year >= config$reference_year) %>%
176     dplyr::mutate(line_group = "post")
177   )
178
179   period_bands <- tibble::tibble(
180     xmin = c(1990, 2008, 2016, 2020),
181     xmax = c(2007, 2012, 2019.8, 2020.8),
182     label = c("ADH exposure", "Great Recession / Obama", "Trump era", "COVID era"),
183     fill = c("#9ecae1", "#fdd0a2", "#c7e9c0", "#dadaeb")
184   )
185
186   event_plot <- ggplot2::ggplot() +
187     ggplot2::geom_rect(
188       data = period_bands,
189       ggplot2::aes(xmin = xmin, xmax = xmax, ymin = -Inf, ymax = Inf, fill = label),
190       alpha = 0.16,
191       inherit.aes = FALSE
192     ) +
193     ggplot2::geom_hline(yintercept = 0, linetype = "dashed", linewidth = 0.35) +
194     ggplot2::geom_vline(xintercept = config$reference_year, linetype = "dotted", linewidth = 0.35) +
195     ggplot2::geom_errorbar(
196       data = plot_tbl,
197       ggplot2::aes(x = year, ymin = conf.low, ymax = conf.high),
198       width = 0.6,
199       linewidth = 0.35
200     ) +
201     ggplot2::geom_line(
202       data = line_tbl,
203       ggplot2::aes(x = year, y = estimate, group = interaction(spec_label, line_group)),
204       linewidth = 0.4
205     ) +
206     ggplot2::geom_point(
207       data = plot_with_ref,
208       ggplot2::aes(x = year, y = estimate),
209       size = 1.8
210     ) +
211     ggplot2::facet_wrap(~ spec_label, ncol = 1) +
212     ggplot2::scale_fill_manual(values = stats::setNames(period_bands$fill, period_bands$label), name = NULL) +
213     ggplot2::scale_x_continuous(
214       breaks = seq(min(plot_with_ref$year), max(plot_with_ref$year), by = 8),
215       minor_breaks = seq(min(plot_with_ref$year), max(plot_with_ref$year), by = 4)
216     ) +
217     ggplot2::labs(
218       title = "China exposure and Republican presidential vote margin, 1972-2020",
219       subtitle = paste0("Event-study coefficients with 95% ", se_label, " CIs; ", config$reference_year, " normalized to zero."),
220       x = "Presidential election year",
221       y = paste0("Coefficient on ADH China exposure (per ", config$exposure_units, ")")
222     ) +
223     ggplot2::theme_bw(base_size = 11) +
224     ggplot2::theme(
225       plot.title = ggplot2::element_text(face = "bold"),
226       panel.grid.minor = ggplot2::element_blank(),
227       panel.grid.major.x = ggplot2::element_blank(),
228       legend.position = "bottom"
229     )
230
231   ggplot2::ggsave(filename = config$figure_pdf, plot = event_plot, width = 8.5, height = 7.5, bg = "white")
232   ggplot2::ggsave(filename = config$figure_png, plot = event_plot, width = 8.5, height = 7.5, dpi = 320, bg = "white")
233
234   output_inventory <- write_output_manifest(config)
235
236   write_pipeline_manifest(
237     config = config,
238     checks = checks,
239     stage = "export_event_study_outputs",
240     sources = list(
241       all_specs = model_object_reference(config$all_specs_rds, config)
242     ),
243     extra = list(
244       outputs = list(
245         table_csv = list(path = config$table_csv, md5 = file_checksum(config$table_csv)),
246         table_tex = list(path = config$table_tex, md5 = file_checksum(config$table_tex)),
247         table_pdf = list(path = config$table_pdf, md5 = file_checksum(config$table_pdf)),
248         figure_pdf = list(path = config$figure_pdf, md5 = file_checksum(config$figure_pdf)),
249         figure_png = list(path = config$figure_png, md5 = file_checksum(config$figure_png)),
250         output_manifest_csv = list(path = config$output_manifest_csv, md5 = file_checksum(config$output_manifest_csv)),
251         output_manifest_json = list(path = config$output_manifest_json, md5 = file_checksum(config$output_manifest_json))
252       ),
253       output_manifest_n_files = nrow(output_inventory),
254       validation_failures = has_validation_failures,
255       reported_nonfatal_conditions = has_reported_nonfatal_conditions
256     )
257   )
258
259   invisible(list(table_csv = table_csv, table_tex = config$table_tex, figure = event_plot))
260 }

```

6 05_export_diagnostic_maps.R

```
1 # extension-adh2013/src/05_export_diagnostic_maps.R
2
3 crosswalk_sensitivity_plan <- function() {
4   tibble::tribble(
5     ~crosswalk_weight, ~crosswalk_missing_weight_policy, ~label, ~render_support_maps,
6     "m5_weight", "fallback_m2", "M5 + fallback M2", TRUE,
7     "m6_weight", "fallback_m2", "M6 + fallback M2", FALSE,
8     "m2_weight", "fallback_m2", "Pure M2", FALSE,
9     "m4_weight", "fallback_m2", "M4 + fallback M2", FALSE
10  )
11 }
12
13 label_from_crosswalk_slug <- function(slug) {
14   dplyr::case_when(
15     slug == "m5" ~ "M5 + fallback M2",
16     slug == "m6" ~ "M6 + fallback M2",
17     slug == "m2" ~ "Pure M2",
18     slug == "m4" ~ "M4 + fallback M2",
19     TRUE ~ toupper(slug)
20   )
21 }
22
23 export_crosswalk_specification_comparison <- function(config = CONFIG) {
24   load_required_packages(config)
25   config <- finalize_config(config)
26
27   coef_files <- list.files(
28     config$intermediate_dir,
29     pattern = "~event_study_coefficients_all_specs_aa2021_rep_margin.*\\.csv$",
30     full.names = TRUE
31   )
32   if (length(coef_files) == 0) {
33     warning("No all-spec coefficient CSVs found; skipping crosswalk specification comparison plot.", call. = FALSE)
34     return(invisible(NULL))
35   }
36
37   crosswalk_levels <- c("M5 + fallback M2", "M6 + fallback M2", "Pure M2", "M4 + fallback M2")
38   display_specs <- c("minimal_full_panel", "interacted_core_controls_diagnostic", "interacted_controls_full_panel")
39
40   coef_long <- purrr::map_dfr(coef_files, function(path) {
41     slug <- sub("event_study_coefficients_all_specs_aa2021_rep_margin_", "", basename(path))
42     slug <- sub("\\.csv$", "", slug)
43     readr::read_csv(path, show_col_types = FALSE) %>%
44       dplyr::mutate(
45         crosswalk_specification = label_from_crosswalk_slug(slug),
46         crosswalk_slug = slug
47       )
48   }) %>%
49   dplyr::filter(
50     sample == "main_1972_start",
51     se_type == config$main_se_type,
52     spec %in% display_specs
53   ) %>%
54   dplyr::mutate(
55     spec_label = dplyr::recode(
56       spec,
57       minimal_full_panel = "Minimal",
58       interacted_core_controls_diagnostic = "Core controls",
59       interacted_controls_full_panel = "Full controls"
60     ),
61     spec_label = factor(spec_label, levels = c("Minimal", "Core controls", "Full controls")),
62     crosswalk_specification = factor(crosswalk_specification, levels = crosswalk_levels)
63   ) %>%
64   dplyr::filter(!is.na(crosswalk_specification))
65
66   if (nrow(coef_long) == 0) {
67     warning("Crosswalk comparison inputs exist but no matching main-spec coefficients were found.", call. = FALSE)
68     return(invisible(NULL))
69   }
70
71   # Draw thicker baseline/early layers first and thinner layers last. When the
72   # four crosswalk estimates are nearly identical, this makes the overlap show
73   # up as concentric colored strokes rather than one hidden line.
74   draw_order_tbl <- tibble::tibble(
75     crosswalk_specification = factor(crosswalk_levels, levels = crosswalk_levels),
76     draw_order = c(4L, 3L, 1L, 2L),
77     line_width = c(0.75, 0.95, 1.45, 1.20),
78     point_size = c(1.25, 1.45, 2.05, 1.75),
79     alpha_value = c(0.95, 0.90, 0.70, 0.82)
80   )
81
82   coef_long <- coef_long %>%
83     dplyr::left_join(draw_order_tbl, by = "crosswalk_specification")
84
85   readr::write_csv(coef_long, config$crosswalk_comparison_csv)
86
87   n_specs <- dplyr::n_distinct(coef_long$crosswalk_specification)
```

```

88 plot_title <- if (n_specs > 1) {
89   "Crosswalk-specification sensitivity"
90 } else {
91   paste0("Event-study coefficients: ", as.character(unique(coef_long$crosswalk_specification)))
92 }
93 plot_subtitle <- if (n_specs > 1) {
94   paste0("Main 1972-2020 event-study coefficients; colored/width-varied lines reveal nearly overlapping estimates. SE type = ",
95     config$main_se_type)
96 } else {
97   paste0("Only one crosswalk specification was found; rerun sensitivity mode to add M6, M2, and M4. SE type = ", config$main_se_type)
98 }
99
100 # Colors are intentionally specified here because the plot is meant for a
101 # final appendix figure where four nearly identical series must be visually
102 # distinguishable.
103 comparison_colors <- c(
104   "M5 + fallback M2" = "#1b9e77",
105   "M6 + fallback M2" = "#d95f02",
106   "Pure M2" = "#7570b3",
107   "M4 + fallback M2" = "#e7298a"
108 )
109 comparison_linetypes <- c(
110   "M5 + fallback M2" = "solid",
111   "M6 + fallback M2" = "22",
112   "Pure M2" = "longdash",
113   "M4 + fallback M2" = "dotdash"
114 )
115 comparison_linewidths <- c(
116   "M5 + fallback M2" = 0.75,
117   "M6 + fallback M2" = 0.95,
118   "Pure M2" = 1.45,
119   "M4 + fallback M2" = 1.20
120 )
121 comparison_alphas <- c(
122   "M5 + fallback M2" = 0.95,
123   "M6 + fallback M2" = 0.90,
124   "Pure M2" = 0.70,
125   "M4 + fallback M2" = 0.82
126 )
127 comparison_shapes <- c(
128   "M5 + fallback M2" = 16,
129   "M6 + fallback M2" = 17,
130   "Pure M2" = 15,
131   "M4 + fallback M2" = 18
132 )
133
134 coef_plot_data <- coef_long %>%
135   dplyr::arrange(spec_label, draw_order, year)
136
137 comp_plot <- ggplot2::ggplot(
138   coef_plot_data,
139   ggplot2::aes(
140     x = year, y = estimate,
141     color = crosswalk_specification,
142     linetype = crosswalk_specification,
143     linewidth = crosswalk_specification,
144     alpha = crosswalk_specification,
145     group = crosswalk_specification
146   )
147 ) +
148   ggplot2::geom_hline(yintercept = 0, linetype = "dashed", linewidth = 0.3, color = "grey45") +
149   ggplot2::geom_line(lineend = "round") +
150   ggplot2::geom_point(ggplot2::aes(shape = crosswalk_specification, size = crosswalk_specification), stroke = 0.2) +
151   ggplot2::scale_color_manual(values = comparison_colors, drop = FALSE) +
152   ggplot2::scale_linetype_manual(values = comparison_linetypes, drop = FALSE) +
153   ggplot2::scale_linewidth_manual(values = comparison_linewidths, drop = FALSE, guide = "none") +
154   ggplot2::scale_alpha_manual(values = comparison_alphas, drop = FALSE, guide = "none") +
155   ggplot2::scale_shape_manual(values = comparison_shapes, drop = FALSE) +
156   ggplot2::scale_size_manual(values = c(
157     "M5 + fallback M2" = 1.25,
158     "M6 + fallback M2" = 1.45,
159     "Pure M2" = 2.05,
160     "M4 + fallback M2" = 1.75
161   ), drop = FALSE, guide = "none") +
162   ggplot2::facet_wrap(~ spec_label, ncol = 1) +
163   ggplot2::labs(
164     title = plot_title,
165     subtitle = plot_subtitle,
166     x = "Presidential election year",
167     y = paste0("Coefficient on ADH China exposure (per ", config$exposure_units, ")"),
168     color = "Crosswalk",
169     linetype = "Crosswalk",
170     shape = "Crosswalk"
171   ) +
172   ggplot2::theme_bw(base_size = 11) +
173   ggplot2::theme(legend.position = "bottom", panel.grid.minor = ggplot2::element_blank())
174
175 ggplot2::ggsave(config$crosswalk_comparison_pdf, comp_plot, width = 8.5, height = 7.5, bg = "white")
176 ggplot2::ggsave(config$crosswalk_comparison_png, comp_plot, width = 8.5, height = 7.5, dpi = 320, bg = "white")

```

```

177 # Appendix companion: differences relative to Pure M2. This makes the actual
178 # magnitude of crosswalk sensitivity visible when the level plot overlaps.
179 pure_m2 <- coef_long %>%
180   dplyr::filter(crosswalk_specification == "Pure M2") %>%
181   dplyr::select(spec_label, year, pure_m2_estimate = estimate)
182
183 delta_long <- coef_long %>%
184   dplyr::left_join(pure_m2, by = c("spec_label", "year")) %>%
185   dplyr::mutate(delta_vs_pure_m2 = estimate - pure_m2_estimate) %>%
186   dplyr::arrange(spec_label, draw_order, year)
187
188 readr::write_csv(delta_long, config$crosswalk_comparison_delta_csv)
189
190 if (n_specs > 1 && any(is.finite(delta_long$delta_vs_pure_m2))) {
191   delta_plot <- ggplot2::ggplot(
192     delta_long,
193     ggplot2::aes(
194       x = year, y = delta_vs_pure_m2,
195       color = crosswalk_specification,
196       linetype = crosswalk_specification,
197       linewidth = crosswalk_specification,
198       alpha = crosswalk_specification,
199       group = crosswalk_specification
200     )
201   ) +
202     ggplot2::geom_hline(yintercept = 0, linetype = "dashed", linewidth = 0.3, color = "grey45") +
203     ggplot2::geom_line(linetype = "round") +
204     ggplot2::geom_point(ggplot2::aes(shape = crosswalk_specification, size = crosswalk_specification), stroke = 0.2) +
205     ggplot2::scale_color_manual(values = comparison_colors, drop = FALSE) +
206     ggplot2::scale_linetype_manual(values = comparison_linetypes, drop = FALSE) +
207     ggplot2::scale_linewidth_manual(values = comparison_linewidths, drop = FALSE, guide = "none") +
208     ggplot2::scale_alpha_manual(values = comparison_alphas, drop = FALSE, guide = "none") +
209     ggplot2::scale_shape_manual(values = comparison_shapes, drop = FALSE) +
210     ggplot2::scale_size_manual(values = c(
211       "M5 + fallback M2" = 1.25,
212       "M6 + fallback M2" = 1.45,
213       "Pure M2" = 2.05,
214       "M4 + fallback M2" = 1.75
215     ), drop = FALSE, guide = "none") +
216     ggplot2::facet_wrap(~ spec_label, ncol = 1, scales = "free_y") +
217     ggplot2::labs(
218       title = "Crosswalk-specification sensitivity relative to Pure M2",
219       subtitle = paste0("Main 1972-2020 event-study coefficients; SE type = ", config$main_se_type),
220       x = "Presidential election year",
221       y = paste0("Coefficient difference vs. Pure M2 (per ", config$exposure_units, ")"),
222       color = "Crosswalk",
223       linetype = "Crosswalk",
224       shape = "Crosswalk"
225     ) +
226     ggplot2::theme_bw(base_size = 11) +
227     ggplot2::theme(legend.position = "bottom", panel.grid.minor = ggplot2::element_blank())
228
229     ggplot2::ggsave(config$crosswalk_comparison_delta_pdf, delta_plot, width = 8.5, height = 7.5, bg = "white")
230     ggplot2::ggsave(config$crosswalk_comparison_delta_png, delta_plot, width = 8.5, height = 7.5, dpi = 320, bg = "white")
231   }
232
233   invisible(comp_plot)
234 }
235
236 simplify_sf_for_plot <- function(sf_obj, tolerance = 0.03) {
237   if (is.null(sf_obj)) return(sf_obj)
238   out <- tryCatch(
239     sf::st_simplify(sf_obj, dTolerance = tolerance, preserveTopology = TRUE),
240     error = function(e) sf_obj
241   )
242   out
243 }
244
245 save_crosswalk_support_plot_pair <- function(plot, simplified_plot, pdf_path, png_path, simplified_pdf_path, simplified_png_path,
246   ↪ small_png_path = NULL) {
247   ggplot2::ggsave(pdf_path, plot, width = 10, height = 7.5, bg = "white")
248   ggplot2::ggsave(png_path, plot, width = 10, height = 7.5, dpi = 320, bg = "white")
249   ggplot2::ggsave(simplified_pdf_path, simplified_plot, width = 10, height = 7.5, bg = "white")
250   ggplot2::ggsave(simplified_png_path, simplified_plot, width = 10, height = 7.5, dpi = 260, bg = "white")
251   if (!is.null(small_png_path)) {
252     ggplot2::ggsave(small_png_path, simplified_plot, width = 10, height = 7.5, dpi = 160, bg = "white")
253   }
254 }
255
256 make_support_map_plot <- function(map_data, title, subtitle, caption, boundary_color = NA) {
257   ggplot2::ggplot(map_data) +
258     ggplot2::geom_sf(ggplot2::aes(fill = map_fill), color = boundary_color, linewidth = 0.05) +
259     ggplot2::facet_wrap(~ weight_type, ncol = 2) +
260     ggplot2::scale_fill_manual(
261       values = c(
262         "No positive affected vote mass" = "grey90",
263         "Receives positive affected vote mass" = "grey25"
264       ),
265       name = NULL
266     ) +

```



```

266   ggplot2::labs(title = title, subtitle = subtitle, caption = caption) +
267   ggplot2::theme_void(base_size = 10) +
268   ggplot2::theme(legend.position = "bottom", plot.title = ggplot2::element_text(face = "bold"))
269 }
270
271 export_crosswalk_support_maps <- function(config = CONFIG) {
272   load_required_packages(config)
273   config <- finalize_config(config)
274   if (!isTRUE(config$export_crosswalk_maps)) return(invisible(NULL))
275
276   if (!file.exists(config$target_weight_support_county_csv) || !file.exists(config$target_weight_support_cz_csv)) {
277     warning("Crosswalk support diagnostics are missing; skipping support maps.", call. = FALSE)
278     return(invisible(NULL))
279   }
280   if (!requireNamespace("sf", quietly = TRUE)) {
281     warning("Package 'sf' is not installed; skipping support maps.", call. = FALSE)
282     return(invisible(NULL))
283   }
284
285   county_sf <- safe_read_county1990_shapefile(config)
286   if (is.null(county_sf)) return(invisible(NULL))
287
288   county_diag <- readr::read_csv(config$target_weight_support_county_csv, show_col_types = FALSE) %>%
289     dplyr::mutate(
290       weight_type = factor(
291         weight_type,
292         levels = c("m2_weight", "m4_weight", "m5_weight", "m6_weight"),
293         labels = c("M2 unavailable", "M4 unavailable", "M5 unavailable", "M6 unavailable")
294       )
295     )
296   county_map <- county_sf %>%
297     dplyr::left_join(county_diag, by = "county_fips_1990") %>%
298     dplyr::filter(!is.na(weight_type)) %>%
299     dplyr::mutate(
300       receives_unavailable_source = dplyr::coalesce(receives_unavailable_source, FALSE),
301       map_fill = dplyr::if_else(receives_unavailable_source, "Receives positive affected vote mass", "No positive affected vote mass")
302     )
303
304   county_plot <- make_support_map_plot(
305     county_map,
306     title = "ADH-mainland 1990 counties receiving affected bridged vote mass",
307     subtitle = "Demonstration map: flags counties receiving positive vote mass from a source county where the indicated FTZ weight type
308     ↪ has zero/undefined support.",
309     caption = "Uses the selected/fallback crosswalk as the vote-mass allocator and the 1990 county shapefile under
310     ↪ spatial-data/counties-1990 unless overridden.",
311     boundary_color = NA
312   )
313   county_map_simple <- simplify_sf_for_plot(county_map, tolerance = 0.03)
314   county_plot_simple <- make_support_map_plot(
315     county_map_simple,
316     title = "ADH-mainland 1990 counties receiving affected bridged vote mass",
317     subtitle = "Simplified-geometry version for smaller appendix files.",
318     caption = "Uses the selected/fallback crosswalk as the vote-mass allocator and simplified 1990 county geometry.",
319     boundary_color = NA
320   )
321   save_crosswalk_support_plot_pair(
322     county_plot, county_plot_simple,
323     config$crosswalk_support_county_map_pdf, config$crosswalk_support_county_map_png,
324     config$crosswalk_support_county_map_simplified_pdf, config$crosswalk_support_county_map_simplified_png,
325     config$crosswalk_support_county_map_small_png
326   )
327
328   cz_lookup <- readxl::read_xls(file.path(config$replication_dir, "cz-data", "cz-198090.xls")) %>%
329     dplyr::transmute(
330       county_fips_1990 = stringr::str_pad(as.character(`County FIPS Code`), width = 5, side = "left", pad = "0"),
331       czone = as.integer(CZ90)
332     )
333
334   cz_diag <- readr::read_csv(config$target_weight_support_cz_csv, show_col_types = FALSE) %>%
335     dplyr::mutate(
336       weight_type = factor(
337         weight_type,
338         levels = c("m2_weight", "m4_weight", "m5_weight", "m6_weight"),
339         labels = c("M2 unavailable", "M4 unavailable", "M5 unavailable", "M6 unavailable")
340       )
341     )
342
343   cz_sf <- county_sf %>%
344     dplyr::inner_join(cz_lookup, by = "county_fips_1990") %>%
345     dplyr::filter(czone %in% unique(cz_diag$czone)) %>%
346     dplyr::group_by(czone) %>%
347     dplyr::summarise(geometry = sf::st_union(geometry), .groups = "drop")
348
349   cz_map <- cz_sf %>%
350     dplyr::left_join(cz_diag, by = "czone") %>%
351     dplyr::filter(!is.na(weight_type)) %>%
352     dplyr::mutate(
353       receives_unavailable_source = dplyr::coalesce(receives_unavailable_source, FALSE),
354       map_fill = dplyr::if_else(receives_unavailable_source, "Receives positive affected vote mass", "No positive affected vote mass")
355     )

```

```

354 cz_plot <- make_support_map_plot(
355   cz_map,
356   title = "ADH-mainland 1990 CZs receiving affected bridged vote mass",
357   subtitle = "Demonstration map: flags CZs receiving positive vote mass from a source county where the indicated FTZ weight type has
358     ↳ zero/undefined support.",
359   caption = "CZ shapes are dissolved from 1990 counties using cz-198090.xls and limited to the ADH diagnostic CZ universe.",
360   boundary_color = "white"
361 )
362 cz_map_simple <- simplify_sf_for_plot(cz_map, tolerance = 0.05)
363 cz_plot_simple <- make_support_map_plot(
364   cz_map_simple,
365   title = "ADH-mainland 1990 CZs receiving affected bridged vote mass",
366   subtitle = "Simplified-geometry version for smaller appendix files.",
367   caption = "CZ shapes are dissolved from simplified 1990 counties and limited to the ADH diagnostic CZ universe.",
368   boundary_color = "white"
369 )
370 save_crosswalk_support_plot_pair(
371   cz_plot, cz_plot_simple,
372   config$crosswalk_support_cz_map_pdf, config$crosswalk_support_cz_map_png,
373   config$crosswalk_support_cz_map_simplified_pdf, config$crosswalk_support_cz_map_simplified_png,
374   config$crosswalk_support_cz_map_small_png
375 )
376
377 invisible(list(county_plot = county_plot, cz_plot = cz_plot))
378 }
379
380 copy_if_exists <- function(from, to_dir) {
381   if (is.null(from) || length(from) == 0 || is.na(from) || !file.exists(from)) return(invisible(FALSE))
382   dir.create(to_dir, recursive = TRUE, showWarnings = FALSE)
383   file.copy(from, file.path(to_dir, basename(from)), overwrite = TRUE)
384   invisible(TRUE)
385 }
386
387 archive_crosswalk_run_diagnostics <- function(config, label, status = "completed", error_message = NA_character_) {
388   config <- finalize_config(config)
389   slug <- config$crosswalk_weight_slug
390   out_dir <- file.path(config$diagnostic_dir, "by_crosswalk", slug)
391   dir.create(out_dir, recursive = TRUE, showWarnings = FALSE)
392
393   files <- unique(stats::na.omit(c(
394     config$validation_checks_csv, config$pipeline_manifest_json, config$crosswalk_diagnostics_csv,
395     config$crosswalk_zero_support_csv, config$crosswalk_row_diagnostics_csv,
396     config$crosswalk_issue_vote_report_csv, config$bridge_diagnostics_csv,
397     config$mainland_retention_csv, config$unmatched_source_counties_csv,
398     config$county_bridge_classification_csv, config$missing_cz_years_csv,
399     config$panel_coverage_csv, config$spec_status_csv, config$model_diagnostics_csv,
400     config$vcov_diagnostics_csv, config$sse_warning_diagnostics_csv, config$event_study_csv,
401     config$all_specs_csv, config$table_csv
402   )))
403   invisible(lapply(files, copy_if_exists, to_dir = out_dir))
404
405   tibble::tibble(
406     crosswalk_slug = slug,
407     crosswalk_specification = label,
408     crosswalk_weight = config$crosswalk_weight,
409     crosswalk_missing_weight_policy = config$crosswalk_missing_weight_policy,
410     status = status,
411     error_message = error_message,
412     diagnostics_dir = portable_path(out_dir, config),
413     archived_at = format(Sys.time(), "%Y-%m-%d %H:%M:%S %Z")
414   ) %>%
415     readr::write_csv(file.path(out_dir, "run_status.csv"))
416
417   invisible(out_dir)
418 }
419
420 write_crosswalk_sensitivity_status <- function(status_rows, config) {
421   config <- finalize_config(config)
422   status_tbl <- dplyr::bind_rows(status_rows)
423   path <- file.path(config$diagnostic_dir, "crosswalk_sensitivity_run_status.csv")
424   readr::write_csv(status_tbl, path)
425   invisible(path)
426 }
427
428 export_diagnostic_maps_and_comparisons <- function(config = CONFIG) {
429   config <- finalize_config(config)
430   export_crosswalk_specification_comparison(config)
431   export_crosswalk_support_maps(config)
432
433   checks <- if (file.exists(config$validation_checks_csv)) {
434     readr::read_csv(config$validation_checks_csv, show_col_types = FALSE)
435   } else {
436     tibble::tibble()
437   }
438   output_inventory <- write_output_manifest(config)
439
440   write_pipeline_manifest(
441     config = config,
442     checks = checks,

```

```

443 stage = "export_diagnostic_maps_and_comparisons",
444 sources = list(
445   county_support = list(path = config$target_weight_support_county_csv, md5 = file_checksum(config$target_weight_support_county_csv)),
446   cz_support = list(path = config$target_weight_support_cz_csv, md5 = file_checksum(config$target_weight_support_cz_csv))
447 ),
448 extra = list(
449   outputs = list(
450     crosswalk_comparison_png = list(path = config$crosswalk_comparison_png, md5 = file_checksum(config$crosswalk_comparison_png)),
451     crosswalk_comparison_delta_png = list(path = config$crosswalk_comparison_delta_png, md5 =
452       ↪ file_checksum(config$crosswalk_comparison_delta_png)),
453     crosswalk_comparison_delta_csv = list(path = config$crosswalk_comparison_delta_csv, md5 =
454       ↪ file_checksum(config$crosswalk_comparison_delta_csv)),
455     crosswalk_support_county_map_png = list(path = config$crosswalk_support_county_map_png, md5 =
456       ↪ file_checksum(config$crosswalk_support_county_map_png)),
457     crosswalk_support_county_map_simplified_png = list(path = config$crosswalk_support_county_map_simplified_png, md5 =
458       ↪ file_checksum(config$crosswalk_support_county_map_simplified_png)),
459     crosswalk_support_county_map_small_png = list(path = config$crosswalk_support_county_map_small_png, md5 =
460       ↪ file_checksum(config$crosswalk_support_county_map_small_png)),
461     crosswalk_support_cz_map_png = list(path = config$crosswalk_support_cz_map_png, md5 =
462       ↪ file_checksum(config$crosswalk_support_cz_map_png)),
463     crosswalk_support_cz_map_simplified_png = list(path = config$crosswalk_support_cz_map_simplified_png, md5 =
464       ↪ file_checksum(config$crosswalk_support_cz_map_simplified_png)),
465     crosswalk_support_cz_map_small_png = list(path = config$crosswalk_support_cz_map_small_png, md5 =
466       ↪ file_checksum(config$crosswalk_support_cz_map_small_png)),
467     output_manifest_csv = list(path = config$output_manifest_csv, md5 = file_checksum(config$output_manifest_csv)),
468     output_manifest_json = list(path = config$output_manifest_json, md5 = file_checksum(config$output_manifest_json))
469   ),
470   output_manifest_n_files = nrow(output_inventory)
471 )
472 invisible(NULL)
473 }
474
475 run_crosswalk_sensitivity_pipeline <- function(config = CONFIG) {
476   base_config <- finalize_config(config)
477   plan <- crosswalk_sensitivity_plan()
478   ensure_output_dirs(base_config)
479   write_dependency_manifest(base_config)
480
481   status_rows <- list()
482   for (i in seq_len(nrow(plan))) {
483     local_config <- base_config
484     local_config$crosswalk_weight <- plan$crosswalk_weight[[i]]
485     local_config$crosswalk_missing_weight_policy <- plan$crosswalk_missing_weight_policy[[i]]
486     local_config$export_crosswalk_maps <- isTRUE(base_config$export_crosswalk_maps) && isTRUE(plan$render_support_maps[[i]])
487     local_config <- finalize_config(local_config)
488
489     label <- plan$label[[i]]
490     message("Running crosswalk sensitivity specification: ", label, " (", local_config$crosswalk_weight, ", ",
491       ↪ local_config$crosswalk_missing_weight_policy, ")")
492
493     step_status <- "completed"
494     step_error <- NA_character_
495     started_at <- Sys.time()
496     tryCatch({
497       build_analysis_data(local_config, stop_on_fatal = TRUE)
498       estimate_event_study(local_config, stop_on_fatal = TRUE)
499       export_event_study_outputs(local_config)
500       if (isTRUE(local_config$export_crosswalk_maps)) export_crosswalk_support_maps(local_config)
501     }, error = function(e) {
502       step_status <- "failed"
503       step_error <- conditionMessage(e)
504       message("Crosswalk sensitivity specification failed: ", label, ": ", step_error)
505     })
506
507     archive_crosswalk_run_diagnostics(local_config, label, status = step_status, error_message = step_error)
508     status_rows[[length(status_rows) + 1L]] <- tibble::tibble(
509       crosswalk_slug = local_config$crosswalk_weight_slug,
510       crosswalk_specification = label,
511       crosswalk_weight = local_config$crosswalk_weight,
512       crosswalk_missing_weight_policy = local_config$crosswalk_missing_weight_policy,
513       status = step_status,
514       error_message = step_error,
515       started_at = format(started_at, "%Y-%m-%d %H:%M:%S %Z"),
516       finished_at = format(Sys.time(), "%Y-%m-%d %H:%M:%S %Z")
517     )
518     write_crosswalk_sensitivity_status(status_rows, base_config)
519   }
520
521   # After all slug-specific coefficient files exist, create the appendix-ready comparison figure.
522   comparison_config <- finalize_config(base_config)
523   export_crosswalk_specification_comparison(comparison_config)
524
525   if (isTRUE(comparison_config$run_sixth_extensions)) {
526     # Sixth-section extensions are estimated only once for the preferred M5 + fallback-M2 design.
527     run_sixth_extensions(comparison_config)
528   }
529
530   checks <- if (file.exists(comparison_config$validation_checks_csv)) {

```

```

523   readr::read_csv(comparison_config$validation_checks_csv, show_col_types = FALSE)
524 } else {
525   tibble::tibble()
526 }
527 status_tbl <- dplyr::bind_rows(status_rows)
528 write_crosswalk_sensitivity_status(status_rows, comparison_config)
529 output_inventory <- write_output_manifest(comparison_config)
530 write_pipeline_manifest(
531   config = comparison_config,
532   checks = checks,
533   stage = "crosswalk_sensitivity_complete",
534   sources = list(
535     sensitivity_plan = list(specifications = plan),
536     sensitivity_status = list(path = file.path(comparison_config$diagnostic_dir, "crosswalk_sensitivity_run_status.csv"), md5 =
537       ↪ file_checksum(file.path(comparison_config$diagnostic_dir, "crosswalk_sensitivity_run_status.csv")))
538   ),
539   extra = list(
540     any_failed_specifications = any(status_tbl$status != "completed"),
541     completed_specifications = status_tbl %>% dplyr::filter(status == "completed") %>% dplyr::pull(crosswalk_specification),
542     failed_specifications = status_tbl %>% dplyr::filter(status != "completed") %>% dplyr::pull(crosswalk_specification),
543     comparison_csv = list(path = comparison_config$crosswalk_comparison_csv, md5 =
544       ↪ file_checksum(comparison_config$crosswalk_comparison_csv)),
545     comparison_png = list(path = comparison_config$crosswalk_comparison_png, md5 =
546       ↪ file_checksum(comparison_config$crosswalk_comparison_png)),
547     comparison_delta_csv = list(path = comparison_config$crosswalk_comparison_delta_csv, md5 =
548       ↪ file_checksum(comparison_config$crosswalk_comparison_delta_csv)),
549     comparison_delta_png = list(path = comparison_config$crosswalk_comparison_delta_png, md5 =
550       ↪ file_checksum(comparison_config$crosswalk_comparison_delta_png)),
551     output_manifest_csv = list(path = comparison_config$output_manifest_csv, md5 =
552       ↪ file_checksum(comparison_config$output_manifest_csv)),
553     output_manifest_json = list(path = comparison_config$output_manifest_json, md5 =
554       ↪ file_checksum(comparison_config$output_manifest_json)),
555     output_manifest_n_files = nrow(output_inventory)
556   )
557 )
558 invisible(status_tbl)
559 }

```

7 06_sixth_extensions.R

```
1 # extension-adh2013/src/06_sixth_extensions.R
2
3 # Highest-priority Sixth-section extensions. These modules intentionally run once
4 # for the preferred M5 + fallback-M2 design, even when the crosswalk-sensitivity
5 # wrapper is active. The mechanism/identification diagnostics are meant to be
6 # interpreted for the preferred analysis sample rather than repeated for every
7 # crosswalk robustness specification.
8
9 extension_primary_config <- function(config = CONFIG) {
10   out <- config
11   out$crosswalk_weight <- config$extension_crosswalk_weight %||% "m5_weight"
12   out$crosswalk_missing_weight_policy <- config$extension_crosswalk_missing_weight_policy %||% "fallback_m2"
13   out <- finalize_config(out)
14   current <- finalize_config(config)
15   if (!file.exists(out$analysis_panel_rds) && file.exists(current$analysis_panel_rds)) out <- current
16   out
17 }
18
19 safe_num <- function(x) suppressWarnings(as.numeric(x))
20
21 safe_divide <- function(num, den) {
22   num <- safe_num(num); den <- safe_num(den)
23   out <- ifelse(is.na(num) | is.na(den) | den <= 0, NA_real_, num / den)
24   out[!is.finite(out)] <- NA_real_
25   out
26 }
27
28 scalar_or_na <- function(x) {
29   if (is.null(x) || length(x) == 0) return(NA_real_)
30   as.numeric(x[[1]])
31 }
32
33 weighted_mean_safe <- function(x, w = NULL) {
34   x <- safe_num(x)
35   if (is.null(w)) w <- rep(1, length(x)) else w <- safe_num(w)
36   ok <- !is.na(x) & !is.na(w) & w > 0
37   if (!any(ok)) return(NA_real_)
38   stats::weighted.mean(x[ok], w[ok], na.rm = TRUE)
39 }
40
41 extension_spec_label <- function(x) {
42   dplyr::recode(as.character(x),
43     minimal = "Minimal",
44     core_controls = "Core controls",
45     full_controls = "Full controls",
46     .default = as.character(x)
47   )
48 }
49
50 extension_outcome_label <- function(x) {
51   dplyr::recode(as.character(x),
52     rep_margin = "Republican margin",
53     rep_2party_share = "Republican two-party share",
54     dem_2party_share = "Democratic two-party share",
55     two_party_votes_per_1990_pop = "Two-party votes / 1990 population",
56     total_votes_per_1990_pop = "Total votes / 1990 population",
57     rep_votes_per_1990_pop = "Republican votes / 1990 population",
58     dem_votes_per_1990_pop = "Democratic votes / 1990 population",
59     rep_margin_swing = "Republican-margin swing",
60     rep_2party_share_swing = "Republican-share swing",
61     reg_voters_pct = "Registered voters / CVAP",
62     voter_turnout_pct = "Ballots cast / CVAP",
63     reg_voter_turnout_pct = "Ballots cast / registered voters",
64     pres_rep_ratio = "Presidential Republican ratio",
65     pres_dem_ratio = "Presidential Democratic ratio",
66     partisan_index_dem = "Democratic partisanship index",
67     partisan_index_rep = "Republican partisanship index",
68     .default = as.character(x)
69   )
70 }
71
72 adhm_outcome_label <- function(x) {
73   dplyr::recode(as.character(x),
74     d2_rwin_2002_2016 = "Republican House win share, 2002-2016",
75     d2_cfavg_repcon_2002_2016 = "Conservative Republican CF-score share",
76     d2_cfavg_repmod_2002_2016 = "Moderate Republican CF-score share",
77     d2_cfavg_demmod_2002_2016 = "Moderate Democratic CF-score share",
78     d2_cfavg_demlib_2002_2016 = "Liberal Democratic CF-score share",
79     d2_teaparty_2002_2010 = "Tea Party / Freedom Caucus outcome",
80     d_shnr_pres_2000_2016 = "Republican presidential share, 2000-2016",
81     d_shnr_pres_2000_2008 = "Republican presidential share, 2000-2008",
82     .default = as.character(x)
83   )
84 }
85
86 plot_pointrange_event <- function(df, title, subtitle, y_lab = "Coefficient on ADH exposure × year") {
87   ggplot2::ggplot(df, ggplot2::aes(x = year, y = estimate, ymin = estimate - 1.96 * std.error, ymax = estimate + 1.96 * std.error)) +
```

```

88   ggplot2::geom_hline(yintercept = 0, linewidth = 0.25) +
89   ggplot2::geom_pointrange(linewidth = 0.25) +
90   ggplot2::labs(title = title, subtitle = subtitle, x = NULL, y = y_lab) +
91   ggplot2::theme_minimal(base_size = 10)
92 }
93
94 first_existing_col <- function(df, candidates) {
95   nm <- names(df)
96   hit <- candidates[candidates %in% nm]
97   if (length(hit) == 0) NA_character_ else hit[[1]]
98 }
99
100 read_first_rdata_frame <- function(path) {
101   e <- new.env(parent = emptyenv())
102   loaded <- load(path, envir = e)
103   candidates <- loaded[vapply(loaded, function(nm) is.data.frame(e[[nm]]), logical(1))]
104   if (length(candidates) == 0) stop("No data.frame object found in ", path)
105   e[[candidates[[1]]]]
106 }
107
108 coeftable_to_tibble <- function(model, vcov = NULL, term_prefix = NULL) {
109   ct <- tryCatch({
110     if (is.null(vcov)) fixest::coeftable(model) else fixest::coeftable(model, vcov = vcov)
111   }, error = function(e) NULL)
112   if (is.null(ct)) return(tibble::tibble())
113   out <- as.data.frame(ct) %>% tibble::rownames_to_column("term") %>% tibble::as_tibble()
114   names(out) <- c("term", "estimate", "std.error", "statistic", "p.value")
115   if (!is.null(term_prefix)) out <- out %>% dplyr::filter(startsWith(term, term_prefix))
116   out
117 }
118
119 fit_weighted_feols <- function(formula, data, weight_var = "adh_weight", cluster_var = "czone") {
120   weights_fml <- if (!is.null(weight_var) && weight_var %in% names(data)) stats::as.formula(paste0("-", weight_var)) else NULL
121   cluster_fml <- if (!is.null(cluster_var) && cluster_var %in% names(data)) stats::as.formula(paste0("-", cluster_var)) else NULL
122   fixest::feols(formula, data = data, weights = weights_fml, cluster = cluster_fml, warn = FALSE, notes = FALSE)
123 }
124
125 write_plot_safely <- function(plot, png_path, pdf_path, width = 10, height = 6) {
126   try(ggplot2::ggsave(png_path, plot, width = width, height = height, dpi = 300, bg = "white"), silent = TRUE)
127   try(ggplot2::ggsave(pdf_path, plot, width = width, height = height, bg = "white"), silent = TRUE)
128   invisible(NULL)
129 }
130
131 load_adh_extension_data <- function(config = CONFIG) {
132   adh_stack_path <- file.path(config$replication_dir, "adh2013", "dta", "workfile_china.dta")
133   adh_long_path <- file.path(config$replication_dir, "adh2013", "dta", "workfile_china_long.dta")
134   stack <- haven::read_dta(adh_stack_path)
135   long <- haven::read_dta(adh_long_path)
136   baseline <- stack %>%
137     dplyr::filter(yr == 1990) %>%
138     dplyr::transmute(
139       czone = as.integer(czone),
140       statefip = as.integer(statefip),
141       dplyr::across(dplyr::all_of(config$baseline_controls), safe_num)
142     ) %>%
143     dplyr::distinct()
144   exposure <- long %>%
145     dplyr::transmute(
146       czone = as.integer(czone),
147       statefip = as.integer(statefip),
148       exposure_1990_2007 = safe_num(d_tradeusch_pw),
149       instrument_1990_2007 = safe_num(d_tradeotch_pw_lag),
150       adh_weight = safe_num(timepwt48)
151     ) %>%
152     dplyr::distinct()
153   baseline %>%
154     dplyr::inner_join(exposure, by = c("czone", "statefip")) %>%
155     standardize_baseline_controls(config$baseline_controls) %>%
156     purrr::pluck("data")
157 }
158
159 make_single_exposure_event_terms <- function(data, exposure_var, years, ref_year, prefix = "es") {
160   years <- sort(unique(years))
161   for (yr in setdiff(years, ref_year)) {
162     nm <- paste0(prefix, "-", yr)
163     data[[nm]] <- data[[exposure_var]] * as.integer(data$year == yr)
164   }
165   data
166 }
167
168 apply_control_interactions <- function(data, controls, years, ref_year, standardize = TRUE) {
169   controls_use <- model_control_names(controls, standardize = standardize)
170   controls_use <- controls_use[controls_use %in% names(data)]
171   if (length(controls_use) == 0) return(list(data = data, controls = controls_use, terms = character(0)))
172   built <- make_interacted_control_vars(data, controls_use, years, ref_year)
173   list(data = built$data, controls = controls_use, terms = built$vars)
174 }
175
176 fit_single_outcome_event_study <- function(panel, outcome_var, controls, spec_name, config,
177   ref_year = config$reference_year,

```

```

178                                     standardize_controls = TRUE,
179                                     sample_label = "main_1972_start") {
180   yrs <- sort(unique(panel$year[!is.na(panel[[outcome_var]])]))
181   if (length(yrs) == 0) return(list(status = "empty", coefficients = tibble::tibble()))
182   if (!ref_year %in% yrs) ref_year <- min(yrs, na.rm = TRUE)
183   dat <- panel %>% dplyr::filter(year %in% yrs, !is.na(.data[[outcome_var]]), !is.na(exposure))
184   if (nrow(dat) == 0) return(list(status = "empty", coefficients = tibble::tibble()))
185   dat <- make_single_exposure_event_terms(dat, "exposure", yrs, ref_year, prefix = "es")
186   event_terms <- paste0("es_", setdiff(yrs, ref_year))
187   ctrl_build <- apply_control_interactions(dat, controls, yrs, ref_year, standardize = standardize_controls)
188   dat <- ctrl_build$dat
189   rhs <- c(event_terms, ctrl_build$terms)
190   if (length(rhs) == 0) return(list(status = "no_terms", coefficients = tibble::tibble()))
191   fml <- stats::as.formula(paste(outcome_var, "~", paste(rhs, collapse = " + "), "| czone + year"))
192   tryCatch({
193     model <- fixest::feols(fml, data = dat, weights = ~adh_weight, cluster = ~czone, panel.id = ~czone + election_index, warn = FALSE,
194       ↪ notes = FALSE)
195     coef <- coef_table_to_tibble(model) %>%
196       dplyr::filter(term %in% event_terms) %>%
197       dplyr::mutate(
198         outcome = outcome_var,
199         spec = spec_name,
200         sample = sample_label,
201         se_type = "cluster_cz",
202         reference_year = ref_year,
203         year = as.integer(stringr::str_extract(term, "[0-9]{4}")),
204         n_obs = stats::nobs(model),
205         n_cz = dplyr::n_distinct(dat$czone),
206         controls = if (length(ctrl_build$controls) == 0) "none" else paste(ctrl_build$controls, collapse = "; ")
207       ) %>%
208       dplyr::select(outcome, spec, sample, se_type, term, year, reference_year, estimate, std.error, statistic, p.value, n_obs, n_cz,
209         ↪ controls)
210     list(status = "ok", coefficients = coef)
211   }, error = function(e) {
212     list(status = paste0("error: ", conditionMessage(e)), coefficients = tibble::tibble())
213   })
214 }
215
216 # -----
217 # 1. Bartik identification diagnostics
218 # -----
219
220 extract_trade_shift_summary <- function(config) {
221   trade_path <- file.path(config$replication_dir, "adh2013", "dta", "sic87dd_trade_data.dta")
222   if (!file.exists(trade_path)) return(tibble::tibble(status = "missing", path = trade_path))
223   tr <- haven::read_dta(trade_path)
224   names(tr) <- tolower(names(tr))
225   req <- c("year", "importer", "exporter", "imports", "sic87dd")
226   if (!all(req %in% names(tr))) {
227     return(tibble::tibble(status = "missing_required_columns", columns_found = paste(names(tr), collapse = ";")))
228   }
229   tr <- tr %>%
230     dplyr::mutate(
231       year = as.integer(year),
232       importer = toupper(trimws(as.character(importer))),
233       exporter = toupper(trimws(as.character(exporter))),
234       imports = safe_num(imports),
235       sic87dd = as.character(sic87dd)
236     )
237   periods <- tibble::tibble(start_year = c(1991L, 2000L, 1991L), end_year = c(1999L, 2007L, 2007L), period = c("1991-1999", "2000-2007",
238     ↪ "1991-2007"))
239   purrr::pmap_dfr(periods, function(start_year, end_year, period) {
240     base <- tr %>% dplyr::filter(year %in% c(start_year, end_year), exporter %in% c("CHN", "CHINA"), importer %in% c("USA", "OTH",
241       ↪ "OTHER", "OTHCAN"))
242     if (nrow(base) == 0) return(tibble::tibble(period = period, status = "no_matching_trade_rows"))
243     wide <- base %>%
244       dplyr::group_by(sic87dd, importer, year) %>%
245       dplyr::summarise(imports = sum(imports, na.rm = TRUE), .groups = "drop") %>%
246       tidyr::pivot_wider(names_from = year, values_from = imports, names_prefix = "imports_")
247     start_col <- paste0("imports_", start_year)
248     end_col <- paste0("imports_", end_year)
249     if (!all(c(start_col, end_col) %in% names(wide))) return(tibble::tibble(period = period, status = "missing_period_endpoints"))
250     wide %>%
251       dplyr::mutate(
252         start_imports = .data[[start_col]],
253         end_imports = .data[[end_col]],
254         import_growth = end_imports - start_imports
255       ) %>%
256       dplyr::group_by(period = period, importer) %>%
257       dplyr::slice_max(order_by = abs(import_growth), n = 15, with_ties = FALSE) %>%
258       dplyr::ungroup() %>%
259       dplyr::mutate(status = "ok") %>%
260       dplyr::select(status, period, importer, sic87dd, start_imports, end_imports, import_growth)
261   })
262 }
263
264 run_bartik_identification_diagnostics <- function(config = CONFIG) {
265   config <- finalize_config(config)
266   adh <- load_adh_extension_data(config)
267   controls_core <- model_control_names(config$core_interacted_controls, standardize = TRUE)

```

```

264 controls_full <- model_control_names(config$interacted_controls, standardize = TRUE)
265 controls_core <- controls_core[controls_core %in% names(adh)]
266 controls_full <- controls_full[controls_full %in% names(adh)]
267 fs_specs <- list(minimal = character(0), core_controls = controls_core, full_controls = controls_full)
268 fs_tbl <- purrr::imap_dfr(fs_specs, function(ctrls, spec) {
269   rhs <- c("instrument_1990_2007", ctrls)
270   fml <- stats::as.formula(paste("exposure_1990_2007 ~", paste(rhs, collapse = " + ")))
271   model <- fit_weighted_feols(fml, adh, weight_var = "adh_weight", cluster_var = "statefip")
272   ct <- coeftable_to_tibble(model) %>% dplyr::filter(term == "instrument_1990_2007")
273   tibble::tibble(
274     spec = spec,
275     endogenous_variable = "ADH exposure, 1990-2007",
276     instrument = "ADH other-high-income-country import growth instrument",
277     estimate = scalar_or_na(ct$estimate),
278     std.error = scalar_or_na(ct$std.error),
279     statistic = scalar_or_na(ct$statistic),
280     p.value = scalar_or_na(ct$p.value),
281     first_stage_F_t2 = (scalar_or_na(ct$statistic))^2,
282     n_obs = stats::nobs(model),
283     r2 = tryCatch(fixest::r2(model, type = "r2"), error = function(e) NA_real_),
284     controls = if (length(ctrls) == 0) "none" else paste(ctrls, collapse = "; ")
285   )
286 })
287 readr::write_csv(fs_tbl, config$bartik_first_stage_csv)
288 p_fs <- ggplot2::ggplot(adh, ggplot2::aes(x = instrument_1990_2007, y = exposure_1990_2007)) +
289   ggplot2::geom_point(ggplot2::aes(size = adh_weight), alpha = 0.35) +
290   ggplot2::geom_smooth(method = "lm", se = TRUE, linewidth = 0.5) +
291   ggplot2::guides(size = "none") +
292   ggplot2::labs(title = "ADH exposure first stage", subtitle = "CZ-level China exposure and ADH other-country instrument", x =
293     "Instrument", y = "Exposure") +
294   ggplot2::theme_minimal(base_size = 11)
295 write_plot_safely(p_fs, config$bartik_first_stage_plot_png, config$bartik_first_stage_plot_pdf, width = 8, height = 5)
296
297 balance_vars <- unique(c(config$baseline_controls, controls_full, "statefip"))
298 balance_vars <- balance_vars[balance_vars %in% names(adh)]
299 balance_tbl <- purrr::map_dfr(balance_vars, function(v) {
300   x <- adh[[v]]
301   tibble::tibble(
302     variable = v,
303     correlation_with_exposure = suppressWarnings(stats::cor(x, adh$exposure_1990_2007, use = "pairwise.complete.obs")),
304     correlation_with_instrument = suppressWarnings(stats::cor(x, adh$instrument_1990_2007, use = "pairwise.complete.obs")),
305     weighted_mean = weighted_mean_safe(x, adh$adh_weight),
306     sd = stats::sd(safe_num(x), na.rm = TRUE)
307   )
308 })
309 readr::write_csv(balance_tbl, config$bartik_balance_csv)
310
311 pretrend_tbl <- tibble::tibble()
312 if (file.exists(config$diagnostic_panel_rds)) {
313   panel <- readRDS(config$diagnostic_panel_rds) %>% dplyr::filter(year < config$reference_year) %>% dplyr::arrange(czone, year)
314   if (nrow(panel) > 0) {
315     slope_tbl <- panel %>%
316       dplyr::group_by(czone) %>%
317       dplyr::summarise(
318         pretrend_slope_rep_margin = tryCatch(stats::coef(stats::lm(rep_margin ~ year))[2], error = function(e) NA_real_),
319         pretrend_change_first_to_last = rep_margin[which.max(year)] - rep_margin[which.min(year)],
320         n_pre_years = sum(!is.na(rep_margin)),
321         .groups = "drop"
322       ) %>% dplyr::left_join(adh, by = "czone")
323   }
324   for (outcome in c("pretrend_slope_rep_margin", "pretrend_change_first_to_last")) {
325     for (rhs_var in c("exposure_1990_2007", "instrument_1990_2007")) {
326       for (spec_name in names(fs_specs)) {
327         ctrls <- fs_specs[[spec_name]]
328         dat <- slope_tbl %>% dplyr::filter(!is.na(.data[[outcome]]), !is.na(.data[[rhs_var]]), n_pre_years >= 2)
329         if (nrow(dat) > 0) {
330           rhs <- c(rhs_var, ctrls)
331           fml <- stats::as.formula(paste(outcome, "~", paste(rhs, collapse = " + ")))
332           model <- tryCatch(fit_weighted_feols(fml, dat, weight_var = "adh_weight", cluster_var = "statefip"), error = function(e)
333             NULL)
334           if (!is.null(model)) {
335             ct <- coeftable_to_tibble(model) %>% dplyr::filter(term == rhs_var)
336             pretrend_tbl <- dplyr::bind_rows(pretrend_tbl, tibble::tibble(
337               outcome = outcome, spec = spec_name, regressor = rhs_var,
338               estimate = scalar_or_na(ct$estimate), std.error = scalar_or_na(ct$std.error),
339               statistic = scalar_or_na(ct$statistic), p.value = scalar_or_na(ct$p.value),
340               n_obs = stats::nobs(model), note = "CZ-level pre-1988 presidential Republican-margin pretrend diagnostic; not a causal
341               ↪ test."
342             ))
343           }
344         }
345       }
346     }
347   }
348 }
349 readr::write_csv(pretrend_tbl, config$bartik_pretrend_csv)
350
351 preperiod_path <- file.path(config$replication_dir, "adh2013", "dta", "workfile_china_preperiod.dta")
352 preperiod_tbl <- tibble::tibble()
353 if (file.exists(preperiod_path)) {

```



```

351 pre_raw <- haven::read_dta(preperiod_path)
352 names(pre_raw) <- make.names(names(pre_raw), unique = TRUE)
353 names(pre_raw) <- gsub("\\.", "_", names(pre_raw))
354 pre_raw <- pre_raw %>% dplyr::mutate(czone = as.integer(czone))
355 candidate_outcomes <- names(pre_raw)[grepl("^d_", names(pre_raw))]
356 candidate_outcomes <- setdiff(candidate_outcomes, c("d_tradeotch_pw", "d_tradeotch_pw_lag", "d_tradeotch_pw_lag_future",
  ↪ "d_tradeotch_pw_lag_future"))
357 # The preperiod workfile can contain multiple rows per CZ. Aggregate to one
358 # row per CZ before running diagnostics so the output is comparable to the
359 # main CZ-level exposure regressions.
360 pre <- pre_raw %>%
361   dplyr::group_by(czone) %>%
362   dplyr::summarise(
363     dplyr::across(dplyr::all_of(candidate_outcomes), ~ mean(safe_num(.x), na.rm = TRUE)),
364     preperiod_rows_per_cz = dplyr::n(),
365     .groups = "drop"
366   ) %>%
367   dplyr::mutate(dplyr::across(dplyr::all_of(candidate_outcomes), ~ dplyr::if_else(is.nan(.x), NA_real_, .x))) %>%
368   dplyr::left_join(adh, by = "czone")
369 for (outcome in candidate_outcomes) {
370   for (rhs_var in c("exposure_1990_2007", "instrument_1990_2007")) {
371     dat <- pre %>% dplyr::filter(!is.na(.data[[outcome]]), !is.na(.data[[rhs_var]]))
372     if (nrow(dat) >= 50) {
373       rhs <- c(rhs_var, controls_core)
374       fml <- stats::as.formula(paste(outcome, "~", paste(rhs, collapse = " + ")))
375       mod <- tryCatch(fit_weighted_feols(fml, dat, weight_var = "adh_weight", cluster_var = "statefip"), error = function(e) NULL)
376       if (!is.null(mod)) {
377         ct <- coeftable_to_tibble(mod) %>% dplyr::filter(term == rhs_var)
378         preperiod_tbl <- dplyr::bind_rows(preperiod_tbl, tibble::tibble(
379           outcome = outcome, regressor = rhs_var, estimate = scalar_or_na(ct$estimate), std.error = scalar_or_na(ct$std.error),
380           statistic = scalar_or_na(ct$statistic), p.value = scalar_or_na(ct$p.value), n_obs = stats::nobs(mod),
381           median_source_rows_per_cz = stats::median(dat$preperiod_rows_per_cz, na.rm = TRUE),
382           note = "ADH preperiod-workfile diagnostic: source rows aggregated to one CZ row before regression; future
  ↪ exposure/instrument predicting preperiod/local-labor-market changes."
383         ))
384       }
385     }
386   }
387 }
388 }
389 readr::write_csv(preperiod_tbl, config$bartik_preperiod_workfile_csv)
390
391 trade_summary <- tryCatch(extract_trade_shift_summary(config), error = function(e) tibble::tibble(status = "error", message =
  ↪ conditionMessage(e)))
392 readr::write_csv(trade_summary, config$bartik_industry_shift_summary_csv)
393
394 rot_tbl <- tibble::tibble(
395   diagnostic = c("rotemberg_weights_exact", "industry_by_cz_shares_status", "industry_shift_summary_status", "next_step"),
396   status = c("not_yet_computed", "requires_reconstructing_CZ_by_industry_shares", ifelse(nrow(trade_summary) > 0, "written",
  ↪ "not_written"), "feasible_with_ADH_raw_CBP_scripts_but_not_a_one-file_join"),
397   details = c(
398     "Exact GPSS Rotemberg weights require a CZ-by-industry baseline share matrix matched to industry-level shifts. This patch adds
  ↪ industry-shift summaries and documents the reconstruction path, but does not yet run the full ADH CBP-to-CZ industry-share
  ↪ reconstruction.",
399     "Relevant public ADH files include other/cbp1980_to_cz.do, cbp1990_to_cz.do, cbp2000_to_cz.do, cbp*_imputations.do, cw_n97_s87.dta,
  ↪ and sic87dd_trade_data.dta.",
400     "Industry-level China import-growth summaries are exported from sic87dd_trade_data.dta where columns are available.",
401     "To compute exact GPSS weights: reconstruct 1990 CZ x SIC87DD employment shares, create shift vectors from Chinese import growth in
  ↪ the instrument country group, residualize by controls, and compute Rotemberg weights."
402   )
403 )
404 readr::write_csv(rot_tbl, config$bartik_data_availability_csv)
405
406 memo <- c(
407   "# Bartik / shift-share identification memo", "",
408   "This project estimates reduced-form event-study relationships between CZ-level ADH China import exposure and political outcomes. The
  ↪ ADH exposure variable is a shift-share object: local baseline industry composition is combined with national or foreign
  ↪ import-growth shocks.", "",
409   "## Required assumptions", "",
410   "A causal reading requires high-exposure CZs not to have been on different political trajectories for reasons unrelated to the China
  ↪ shock after conditioning on fixed effects and baseline-control-by-year interactions. For the ADH-style instrument, one can
  ↪ emphasize exogeneity of foreign import-growth shifts or exogeneity of baseline industry shares; both routes require substantive
  ↪ defense.", "",
411   "## Project-specific threats", "",
412   "- Initial industry shares may be correlated with prior political realignment, union decline, automation, racial composition,
  ↪ religiosity, education, or local media markets.",
413   "- If only a few high-shock industries drive exposure, the estimates may capture politics of those industries rather than a generic
  ↪ China-shock effect.",
414   "- Migration and turnout can make CZ-level political outcomes change even if individual beliefs do not change.",
415   "- Candidate supply and elite rhetoric may translate material shocks into political outcomes; the ADHM 2020 mechanism outputs are
  ↪ diagnostic for this channel.", "",
416   "## Diagnostics written by the pipeline", "",
417   "- `bartik_first_stage_diagnostics.csv` and `fig_bartik_first_stage.png`: first-stage strength of the ADH other-country instrument.",
418   "- `bartik_balance_correlations.csv`: exposure/instrument correlations with baseline controls.",
419   "- `bartik_pretrend_placebos.csv`: whether exposure/instrument predict pre-1988 Republican-margin trends.",
420   "- `bartik_preperiod_workfile_diagnostics.csv`: diagnostics using ADH's preperiod workfile.",
421   "- `bartik_industry_shift_summary.csv`: industry-level import-shift summaries from ADH public trade data.",
422   "- `bartik_rotemberg_data_availability.csv`: exact Rotemberg weights are a feasible future reconstruction, not a completed diagnostic
  ↪ here.", ""

```

```

423 )
424 writelines(memo, config$bartik_identification_memo_md)
425 invisible(list(first_stage = fs_tbl, balance = balance_tbl, pretrend = pretrend_tbl, preperiod = preperiod_tbl, trade = trade_summary))
426 }
427
428 # -----
429 # 2. Alternative political outcomes
430 # -----
431
432 add_alternative_political_outcomes <- function(panel) {
433   panel %>%
434     dplyr::arrange(czone, year) %>%
435     dplyr::group_by(czone) %>%
436     dplyr::mutate(
437       rep_2party_share = safe_divide(rep_votes, twoparty_votes),
438       dem_2party_share = safe_divide(dem_votes, twoparty_votes),
439       two_party_votes_per_1990_pop = safe_divide(twoparty_votes, pop1990),
440       total_votes_per_1990_pop = safe_divide(total_votes, pop1990),
441       rep_votes_per_1990_pop = safe_divide(rep_votes, pop1990),
442       dem_votes_per_1990_pop = safe_divide(dem_votes, pop1990),
443       rep_margin_swing = rep_margin - dplyr::lag(rep_margin),
444       rep_2party_share_swing = rep_2party_share - dplyr::lag(rep_2party_share)
445     ) %>%
446     dplyr::ungroup()
447 }
448
449 run_alternative_political_outcome_event_studies <- function(config = CONFIG) {
450   config <- finalize_config(config)
451   if (!file.exists(config$analysis_panel_rds)) {
452     readr::write_csv(tibble::tibble(module = "alternative_political_outcomes", status = "missing_panel", detail =
453       ↪ config$analysis_panel_rds), config$alternative_outcomes_status_csv)
454     return(invisible(NULL))
455   }
456   panel <- readRDS(config$analysis_panel_rds) %>% add_alternative_political_outcomes()
457   outcomes <- c("rep_margin", "rep_2party_share", "dem_2party_share", "two_party_votes_per_1990_pop", "total_votes_per_1990_pop",
458     ↪ "rep_votes_per_1990_pop", "dem_votes_per_1990_pop", "rep_margin_swing", "rep_2party_share_swing")
459   outcome_labels <- tibble::tibble(outcome = outcomes, label = c("Republican margin", "Republican two-party share", "Democratic two-party
460     ↪ share", "Two-party votes / 1990 population", "Total votes / 1990 population", "Republican votes / 1990 population", "Democratic
461     ↪ votes / 1990 population", "Republican-margin swing since previous election", "Republican-share swing since previous election"))
462   specs <- list(minimal = list(controls = character(0), standardize = FALSE), core_controls = list(controls =
463     ↪ config$core_interacted_controls, standardize = TRUE), full_controls = list(controls = config$interacted_controls, standardize =
464     ↪ TRUE))
465   coef_tbl <- tibble::tibble(); status_tbl <- tibble::tibble()
466   for (outcome in outcomes) {
467     for (spec_name in names(specs)) {
468       spec <- specs[[spec_name]]
469       fit <- fit_single_outcome_event_study(panel, outcome, spec$controls, config, standardize_controls = spec$standardize)
470       coef_tbl <- dplyr::bind_rows(coef_tbl, fit$coefficients)
471       status_tbl <- dplyr::bind_rows(status_tbl, tibble::tibble(module = "alternative_political_outcomes", outcome = outcome, spec =
472         ↪ spec_name, status = fit$status, n_coefficients = nrow(fit$coefficients)))
473     }
474   }
475   coef_tbl <- coef_tbl %>% dplyr::left_join(outcome_labels, by = "outcome") %>% dplyr::mutate(spec_label = extension_spec_label(spec))
476   readr::write_csv(coef_tbl, config$alternative_outcomes_coefficients_csv)
477   readr::write_csv(status_tbl, config$alternative_outcomes_status_csv)
478   summary_tbl <- panel %>%
479     dplyr::summarise(dplyr::across(dplyr::all_of(outcomes), list(n = ~sum(!is.na(.x)), mean = ~mean(.x, na.rm = TRUE), sd =
480       ↪ ~stats::sd(.x, na.rm = TRUE), min = ~min(.x, na.rm = TRUE), max = ~max(.x, na.rm = TRUE)), .names = "{.col}_{.fn}")) %>%
481     tidyr::pivot_longer(dplyr::everything(), names_to = "stat", values_to = "value")
482   readr::write_csv(summary_tbl, config$alternative_outcomes_summary_csv)
483   if (nrow(coef_tbl) > 0) {
484     plot_df <- coef_tbl %>% dplyr::filter(spec %in% c("minimal", "core_controls", "full_controls"), outcome %in% outcomes[1:7])
485     p <- ggplot2::ggplot(plot_df, ggplot2::aes(x = year, y = estimate, ymin = estimate - 1.96 * std.error, ymax = estimate + 1.96 *
486       ↪ std.error)) +
487       ggplot2::geom_hline(yintercept = 0, linewidth = 0.25) + ggplot2::geom_pointrange(linewidth = 0.25) +
488       ggplot2::facet_grid(label ~ spec_label, scales = "free_y") +
489       ggplot2::labs(title = "Alternative political outcome event studies", subtitle = "CZ-clustered 95% intervals; 1988 reference where
490         ↪ available", x = NULL, y = "Coefficient on ADH exposure × election year") +
491       ggplot2::theme_minimal(base_size = 10)
492     write_plot_safely(p, config$alternative_outcomes_plot_png, config$alternative_outcomes_plot_pdf, width = 12, height = 10)
493     vote_choice <- coef_tbl %>% dplyr::filter(spec == "full_controls", outcome %in% c("rep_margin", "rep_2party_share",
494       ↪ "dem_2party_share"))
495     if (nrow(vote_choice) > 0) {
496       p_choice <- plot_pointrange_event(vote_choice, "Vote-choice outcome diagnostics", "Full-control specification; CZ-clustered 95%
497         ↪ intervals") +
498         ggplot2::facet_wrap(~label, scales = "free_y", ncol = 1)
499       write_plot_safely(p_choice, config$alternative_outcomes_vote_choice_plot_png, config$alternative_outcomes_vote_choice_plot_pdf,
500         ↪ width = 8.5, height = 7)
501     }
502     vote_volume <- coef_tbl %>% dplyr::filter(spec == "full_controls", outcome %in% c("two_party_votes_per_1990_pop",
503       ↪ "total_votes_per_1990_pop", "rep_votes_per_1990_pop", "dem_votes_per_1990_pop"))
504     if (nrow(vote_volume) > 0) {
505       p_volume <- plot_pointrange_event(vote_volume, "Vote-volume outcome diagnostics", "Full-control specification; CZ-clustered 95%
506         ↪ intervals") +
507         ggplot2::facet_wrap(~label, scales = "free_y", ncol = 2)
508       write_plot_safely(p_volume, config$alternative_outcomes_vote_volume_plot_png, config$alternative_outcomes_vote_volume_plot_pdf,
509         ↪ width = 9.5, height = 7)
510     }
511   }

```

```

495   decomp <- coef_tbl %>% dplyr::filter(spec == "full_controls", outcome %in% c("rep_margin", "rep_votes_per_1990_pop",
496     ↳ "dem_votes_per_1990_pop", "two_party_votes_per_1990_pop"))
497   p2 <- ggplot2::ggplot(decomp, ggplot2::aes(x = year, y = estimate, ymin = estimate - 1.96 * std.error, ymax = estimate + 1.96 *
498     ↳ std.error)) +
499     ggplot2::geom_hline(yintercept = 0, linewidth = 0.25) + ggplot2::geom_pointrange(linewidth = 0.25) +
500     ggplot2::facet_wrap(~label, scales = "free_y", ncol = 2) +
501     ggplot2::labs(title = "Vote-margin decomposition diagnostics", subtitle = "Full-control specification; CZ-clustered 95% intervals",
502       ↳ x = NULL, y = "Coefficient") +
503     ggplot2::theme_minimal(base_size = 10)
504   write_plot_safely(p2, config$alternative_outcomes_decomposition_plot_png, config$alternative_outcomes_decomposition_plot_pdf, width =
505     ↳ 10, height = 7)
506 }
507 invisible(coef_tbl)
508 }
509 # -----
510 # Shared county-to-CZ bridge for county-year auxiliary outcomes
511 # -----
512 build_generic_county_to_cz_panel <- function(config, county_year_data, value_cols) {
513   config <- finalize_config(config)
514   cz_path <- file.path(config$replication_dir, "cz-data", "cz-198090.xls")
515   ftz_zip <- file.path(config$replication_dir, "ftz2024", "crosswalks", "CountyToCounty", "1990", "1990_csv.zip")
516   cz_lookup <- readxl::read_xls(cz_path) %>%
517     dplyr::transmute(county_fips_1990 = stringr::str_pad(as.character("County FIPS Code"), width = 5, side = "left", pad = "0"), czzone =
518       ↳ as.integer(CZ90), pop1990 = safe_num("Population 1990"))
519   source_decades <- sort(unique(year_to_source_decade(county_year_data$year)))
520   xwalk <- purrr::map_dfr(source_decades, ~ read_ftz_crosswalk(.x, ftz_zip, cz_lookup$county_fips_1990, config$crosswalk_weight,
521     ↳ config$crosswalk_missing_weight_policy, config$renormalize_crosswalk_weights, config$weight_sum_tolerance)) %>%
522     dplyr::filter(!is.na(crosswalk_weight), crosswalk_weight > 0)
523   bridge_exceptions <- read_county_bridge_exceptions(config)
524   county_year_data %>%
525     dplyr::mutate(source_decade_default = year_to_source_decade(year)) %>%
526     dplyr::left_join(
527       bridge_exceptions %>% dplyr::select(year, county_fips, override_source_decade, override_county_fips_source),
528       by = c("year", "county_fips")
529     ) %>%
530     dplyr::mutate(
531       source_decade = dplyr::coalesce(override_source_decade, source_decade_default),
532       county_fips_source = dplyr::coalesce(override_county_fips_source, county_fips)
533     ) %>%
534     dplyr::left_join(xwalk, by = c("source_decade", "county_fips_source", relationship = "many-to-many")) %>%
535     dplyr::filter(!is.na(county_fips_1990), !is.na(crosswalk_weight), crosswalk_weight > 0) %>%
536     dplyr::mutate(dplyr::across(dplyr::all_of(value_cols), ~ safe_num(.x) * crosswalk_weight)) %>%
537     dplyr::group_by(year, county_fips_1990) %>%
538     dplyr::summarise(dplyr::across(dplyr::all_of(value_cols), ~ sum(.x, na.rm = TRUE)), .groups = "drop") %>%
539     dplyr::inner_join(cz_lookup, by = "county_fips_1990") %>%
540     dplyr::group_by(czzone, year) %>%
541     dplyr::summarise(dplyr::across(dplyr::all_of(value_cols), ~ sum(.x, na.rm = TRUE)), pop1990 = sum(pop1990, na.rm = TRUE), .groups =
542       ↳ "drop")
543 }
544 # -----
545 # 3. NaNDA turnout and partisanship
546 # -----
547 run_nanda_outcome_event_studies <- function(config = CONFIG) {
548   config <- finalize_config(config)
549   nanda_path <- file.path(config$replication_dir, "outcomes-data", "nanda-2004-2022", "DS0001", "38506-0001-Data.rda")
550   if (!file.exists(nanda_path)) {
551     readr::write_csv(tibble::tibble(status = "missing", path = nanda_path), config$nanda_outcomes_summary_csv)
552     return(invisible(NULL))
553   }
554   raw <- read_first_rdata_frame(nanda_path)
555   names(raw) <- toupper(names(raw))
556   fips_col <- first_existing_col(raw, c("STCOFIPS10", "STCOFIPS20", "FIPS", "COUNTY_FIPS"))
557   if (is.na(fips_col) || !"YEAR" %in% names(raw)) stop("NaNDA data must contain county FIPS and YEAR columns.")
558   fips_source <- if (all(c("STCOFIPS10", "STCOFIPS20") %in% names(raw))) {
559     dplyr::if_else(as.integer(raw$YEAR) >= 2020L, as.character(raw$STCOFIPS20), as.character(raw$STCOFIPS10))
560   } else {
561     as.character(raw[[fips_col]])
562   }
563   numeric_candidates <- c("REG_VOTERS", "BALLOTS_CAST", "CVAP", "PRES_DEM_VOTES", "PRES_REP_VOTES", "SEN_DEM_VOTES", "SEN_REP_VOTES")
564   count_cols <- numeric_candidates[numeric_candidates %in% names(raw)]
565   index_cols <- c("PARTISAN_INDEX_DEM", "PARTISAN_INDEX_REP")
566   index_cols <- index_cols[index_cols %in% names(raw)]
567   raw$COUNTY_FIPS_SELECTED_FOR_BRIDGE <- fips_source
568   nanda <- raw %>%
569     dplyr::transmute(county_fips = stringr::str_pad(as.character(COUNTY_FIPS_SELECTED_FOR_BRIDGE), 5, pad = "0"), year =
570       ↳ as.integer(YEAR), dplyr::across(dplyr::all_of(c(count_cols, index_cols)), safe_num)) %>%
571     dplyr::mutate(dplyr::across(dplyr::all_of(c(count_cols, index_cols)), ~ dplyr::na_if(.x, -1)))
572   # Restrict to documented NaNDA availability; avoid legacy/structural zeros.
573   nanda <- nanda %>% dplyr::filter(year >= 2004, year <= 2022)
574   county_validation <- nanda %>%
575     dplyr::mutate(
576       reg_voters_pct_raw = if (all(c("REG_VOTERS", "CVAP") %in% names(nanda))) safe_divide(REG_VOTERS, CVAP) else NA_real_,
577       voter_turnout_pct_raw = if (all(c("BALLOTS_CAST", "CVAP") %in% names(nanda))) safe_divide(BALLOTS_CAST, CVAP) else NA_real_,
578       reg_voter_turnout_pct_raw = if (all(c("BALLOTS_CAST", "REG_VOTERS") %in% names(nanda))) safe_divide(BALLOTS_CAST, REG_VOTERS) else
579         ↳ NA_real_,

```

```

575     county_rate_flag = dplyr::if_else(reg_voters_pct_raw > 1.5 | voter_turnout_pct_raw > 1.5 | reg_voter_turnout_pct_raw > 1.5, TRUE,
576     ↪ FALSE, missing = FALSE)
577   )
578   readr::write_csv(county_validation %>% dplyr::select(county_fips, year, dplyr::ends_with("_raw"), county_rate_flag),
579   ↪ config$nanda_county_validation_csv)
580   # Convert index shares to CVAP-weighted pseudo-counts before bridging.
581   for (idx in index_cols) nanda[[paste0(idx, "_NUM")]] <- nanda[[idx]] * if ("CVAP" %in% count_cols) nanda$CVAP else 1
582   bridge_cols <- unique(c(count_cols, paste0(index_cols, "_NUM")))
583   bridge_cols <- bridge_cols[bridge_cols %in% names(nanda)]
584   cz_panel <- build_generic_county_to_cz_panel(config, nanda, bridge_cols)
585   if ("CVAP" %in% names(cz_panel)) {
586     if ("REG_VOTERS" %in% names(cz_panel)) cz_panel$reg_voters_pct <- safe_divide(cz_panel$REG_VOTERS, cz_panel$CVAP)
587     if ("BALLOTS_CAST" %in% names(cz_panel)) cz_panel$voter_turnout_pct <- safe_divide(cz_panel$BALLOTS_CAST, cz_panel$CVAP)
588     if (all(c("BALLOTS_CAST", "REG_VOTERS") %in% names(cz_panel))) cz_panel$reg_voter_turnout_pct <- safe_divide(cz_panel$BALLOTS_CAST,
589     ↪ cz_panel$REG_VOTERS)
590     if (all(c("PRES_REP_VOTES", "PRES_DEM_VOTES") %in% names(cz_panel))) {
591       denom <- cz_panel$PRES_REP_VOTES + cz_panel$PRES_DEM_VOTES
592       cz_panel$pres_rep_ratio <- safe_divide(cz_panel$PRES_REP_VOTES, denom)
593       cz_panel$pres_dem_ratio <- safe_divide(cz_panel$PRES_DEM_VOTES, denom)
594     }
595     for (idx in index_cols) {
596       num <- paste0(idx, "_NUM")
597       if (num %in% names(cz_panel)) cz_panel[[tolower(idx)]] <- safe_divide(cz_panel[[num]], cz_panel$CVAP)
598     }
599   }
600   rate_cols <- c("reg_voters_pct", "voter_turnout_pct", "reg_voter_turnout_pct", "pres_rep_ratio", "pres_dem_ratio",
601   ↪ "partisan_index_dem", "partisan_index_rep")
602   rate_cols <- rate_cols[rate_cols %in% names(cz_panel)]
603   cz_initial <- cz_panel %>%
604     dplyr::mutate(dplyr::across(dplyr::all_of(rate_cols), ~ dplyr::if_else(.x < 0 | .x > 1.5 | !is.finite(.x), NA_real_, .x)))
605   outlier_rows <- cz_panel %>%
606     dplyr::filter(dplyr::if_any(dplyr::all_of(rate_cols), ~ !is.na(.x) & (!is.finite(.x) | .x < 0 | .x > 1.5))) %>%
607     dplyr::select(czone, year, dplyr::all_of(rate_cols))
608   readr::write_csv(outlier_rows, config$nanda_outlier_rates_csv)
609
610   availability_long <- cz_initial %>%
611     dplyr::select(czone, year, dplyr::all_of(rate_cols)) %>%
612     tidyr::pivot_longer(dplyr::all_of(rate_cols), names_to = "outcome", values_to = "value") %>%
613     dplyr::group_by(outcome, year) %>%
614     dplyr::summarise(
615       n_cz = dplyr::n(),
616       n_nonmissing = sum(!is.na(value)),
617       n_zero = sum(!is.na(value) & abs(value) < 1e-12),
618       n_nonzero = sum(!is.na(value) & abs(value) >= 1e-12),
619       all_nonmissing_values_zero = n_nonmissing > 0 && n_nonzero == 0,
620       min = if (n_nonmissing > 0) min(value, na.rm = TRUE) else NA_real_,
621       max = if (n_nonmissing > 0) max(value, na.rm = TRUE) else NA_real_,
622       mean = if (n_nonmissing > 0) mean(value, na.rm = TRUE) else NA_real_,
623       .groups = "drop"
624     ) %>%
625     dplyr::mutate(available_for_event_study = n_nonmissing >= 50 & !all_nonmissing_values_zero)
626   readr::write_csv(availability_long, config$nanda_outcome_availability_csv)
627
628   unavailable_years <- availability_long %>%
629     dplyr::filter(!available_for_event_study) %>%
630     dplyr::select(outcome, year)
631   cz_val <- cz_initial
632   if (nrow(unavailable_years) > 0) {
633     for (i in seq_len(nrow(unavailable_years))) {
634       outcome_i <- unavailable_years$outcome[[i]]
635       year_i <- unavailable_years$year[[i]]
636       cz_val[[outcome_i]][cz_val$year == year_i] <- NA_real_
637     }
638   }
639
640   cz_summary <- cz_val %>%
641     dplyr::summarise(n_cz = dplyr::n_distinct(czone), min_year = min(year, na.rm = TRUE), max_year = max(year, na.rm = TRUE),
642     ↪ dplyr::across(dplyr::all_of(rate_cols), list(n_nonmissing = ~sum(!is.na(.x)), min = ~min(.x, na.rm = TRUE), max = ~max(.x, na.rm =
643     ↪ TRUE)), .names = "{.col}_{.fn}"))
644   readr::write_csv(cz_summary, config$nanda_cz_validation_csv)
645   readr::write_csv(cz_val, config$nanda_cz_panel_csv)
646   adh <- load_adh_extension_data(config) %>% dplyr::select(czone, statefip, exposure = exposure_1990_2007, adh_weight,
647   ↪ dplyr::all_of(config$baseline_controls), dplyr::starts_with("z_"))
648   nanda_panel <- cz_val %>% dplyr::inner_join(adh, by = "czone") %>% dplyr::arrange(czone, year) %>% dplyr::group_by(czone) %>%
649     dplyr::mutate(election_index = dplyr::dense_rank(year)) %>% dplyr::ungroup()
650   outcomes <- rate_cols
651   specs <- list(minimal = list(controls = character(0), standardize = FALSE), core_controls = list(controls =
652   ↪ config$core_interacted_controls, standardize = TRUE), full_controls = list(controls = config$interacted_controls, standardize =
653   ↪ TRUE))
654   coef_tbl <- tibble::tibble(); status_tbl <- tibble::tibble()
655   for (outcome in outcomes) {
656     outcome_i <- outcome
657     valid_years <- availability_long %>% dplyr::filter(.data$outcome == outcome_i, available_for_event_study) %>% dplyr::pull(year) %>%
658     ↪ sort()
659     ref <- if (outcome_i %in% c("partisan_index_dem", "partisan_index_rep")) {
660       if (2006L %in% valid_years) 2006L else if (length(valid_years) > 0) min(valid_years) else NA_integer_
661     } else if (outcome_i %in% c("pres_rep_ratio", "pres_dem_ratio")) {
662       if (2004L %in% valid_years) 2004L else if (length(valid_years) > 0) min(valid_years) else NA_integer_
663     } else {

```

```

653   if (2004L %in% valid_years) 2004L else if (length(valid_years) > 0) min(valid_years) else NA_integer_
654 }
655 for (spec_name in names(specs)) {
656   spec <- specs[[spec_name]]
657   fit <- if (length(valid_years) >= 2 && !is.na(ref)) {
658     panel_subset <- nanda_panel %>% dplyr::filter(year %in% valid_years)
659     fit_single_outcome_event_study(panel_subset, outcome_i, spec$controls, spec_name, config, ref_year = ref, standardize_controls =
        ↪ spec$standardize, sample_label = "nanda_2004_2022")
660   } else {
661     list(status = "insufficient_valid_years", coefficients = tibble::tibble())
662   }
663   coef_tbl <- dplyr::bind_rows(coef_tbl, fit$coefficients)
664   status_tbl <- dplyr::bind_rows(status_tbl, tibble::tibble(module = "nanda_turnout_partisanship", outcome = outcome_i, outcome_label
        ↪ = extension_outcome_label(outcome_i), spec = spec_name, spec_label = extension_spec_label(spec_name), reference_year = ref,
        ↪ valid_years = paste(valid_years, collapse = ";"), status = fit$status, n_coefficients = nrow(fit$coefficients)))
665 }
666 }
667 coef_tbl <- coef_tbl %>% dplyr::mutate(outcome_label = extension_outcome_label(.data$outcome), spec_label =
        ↪ extension_spec_label(.data$spec))
668 readr::write_csv(coef_tbl, config$nanda_outcomes_coefficients_csv)
669 readr::write_csv(status_tbl, config$nanda_outcomes_status_csv)
670 readr::write_csv(availability_long, config$nanda_outcomes_summary_csv)
671 if (nrow(coef_tbl) > 0) {
672   p <- plot_pointrange_event(coef_tbl, "NaNDA turnout and partisanship outcomes", "Outcome-specific valid years and reference years;
        ↪ rates outside [0, 1.5] and all-zero structural years set missing; CZ-clustered intervals") +
        ↪ ggplot2::facet_grid(outcome_label ~ spec_label, scales = "free_y")
673   write_plot_safely(p, config$nanda_outcomes_plot_png, config$nanda_outcomes_plot_pdf, width = 12, height = 10)
674   plot_family <- function(family_outcomes, title, png_path, pdf_path, height = 6) {
675     df <- coef_tbl %>% dplyr::filter(outcome %in% family_outcomes, spec == "full_controls")
676     if (nrow(df) > 0) {
677       pp <- plot_pointrange_event(df, title, "Full-control specification; outcome-specific reference years; CZ-clustered intervals") +
678         ↪ ggplot2::facet_wrap(~outcome_label, scales = "free_y", ncol = 1)
679       write_plot_safely(pp, png_path, pdf_path, width = 8.5, height = height)
680     }
681   }
682   plot_family(c("reg_voters_pct", "voter_turnout_pct", "reg_voter_turnout_pct"), "NaNDA turnout and registration diagnostics",
        ↪ config$nanda_turnout_plot_png, config$nanda_turnout_plot_pdf, height = 7)
683   plot_family(c("pres_rep_ratio", "pres_dem_ratio"), "NaNDA presidential vote-ratio diagnostics", config$nanda_vote_plot_png,
        ↪ config$nanda_vote_plot_pdf, height = 5.5)
684   plot_family(c("partisan_index_dem", "partisan_index_rep"), "NaNDA partisanship-index diagnostics",
        ↪ config$nanda_partisanship_plot_png, config$nanda_partisanship_plot_pdf, height = 5.5)
685 }
686 invisible(coef_tbl)
687 }
688
689 # -----
690 # 4. ADHM 2020 political-supply/mechanism diagnostics
691 # -----
692
693 run_adhm2020_mechanism_diagnostics <- function(config = CONFIG) {
694   config <- finalize_config(config)
695   adhm_dir <- file.path(config$replication_dir, "adhm2020", "dta")
696   house_path <- file.path(adhm_dir, "house_2002_2016.dta")
697   pres_path <- file.path(adhm_dir, "president_2000_2016.dta")
698   if (!file.exists(house_path) && !file.exists(pres_path)) {
699     readr::write_csv(tibble::tibble(status = "missing", path = adhm_dir), config$adhm2020_summary_csv)
700     return(invisible(NULL))
701   }
702 }
703 adh <- load_adh_extension_data(config)
704 spec_controls <- list(minimal = character(0), core_controls = model_control_names(config$core_interacted_controls, standardize = TRUE),
        ↪ full_controls = model_control_names(config$interacted_controls, standardize = TRUE))
705 spec_controls <- purrr::map(spec_controls, ~ .x[x %in% names(adh)])
706 regressions <- tibble::tibble(); status <- tibble::tibble()
707 outcome_dict <- tibble::tibble(
708   outcome = c("d2_rwin_2002_2016", "d2_cfavg_repcon_2002_2016", "d2_cfavg_repmod_2002_2016", "d2_cfavg_demmod_2002_2016",
        ↪ "d2_cfavg_demlib_2002_2016", "d2_teparty_2002_2010", "d_shnr_pres_2000_2016", "d_shnr_pres_2000_2008"),
709   outcome_label = adh$outcome_label(outcome),
710   interpretation = c("Change in Republican House win indicator/share", "Change in conservative Republican CF-score share", "Change in
        ↪ moderate Republican CF-score share", "Change in moderate Democratic CF-score share", "Change in liberal Democratic CF-score
        ↪ share", "Change in Tea Party outcome", "Change in Republican presidential vote share, 2000-2016", "Change in Republican
        ↪ presidential vote share, 2000-2008"),
711   report_family = c("House control", "House ideology", "House ideology", "House ideology", "House ideology", "Tea Party",
        ↪ "Presidential", "Presidential")
712 )
713 readr::write_csv(outcome_dict, config$adhm2020_outcome_dictionary_csv)
714 fit_cross_section <- function(df, outcomes, source, weight_var = NULL) {
715   purrr::map_dfr(outcomes[outcomes %in% names(df)], function(outcome) {
716     purrr::imap_dfr(spec_controls, function(ctrls, spec_name) {
717       dat <- df %>% dplyr::inner_join(adh, by = "czone") %>% dplyr::filter(!is.na(.data[[outcome]]), !is.na(exposure_1990_2007))
718       if (nrow(dat) == 0) {
719         status <- dplyr::bind_rows(status, tibble::tibble(source = source, outcome = outcome, spec = spec_name, status = "empty",
        ↪ n_obs = 0L, note = NA_character_))
720       }
721       return(tibble::tibble())
722     })
723   }
724   rhs <- c("exposure_1990_2007", ctrls)
725   fml <- stats::as.formula(paste(outcome, "~", paste(rhs, collapse = " + ")))
726   mod <- tryCatch(fit_weighted_feols(fml, dat, weight_var = if (!is.null(weight_var) && weight_var %in% names(dat)) weight_var else
        ↪ "adh_weight", cluster_var = "stateip"), error = function(e) e)
727   if (inherits(mod, "error")) {

```

```

726     status <- dplyr::bind_rows(status, tibble::tibble(source = source, outcome = outcome, spec = spec_name, status = "error",
727       ↪ n_obs = nrow(dat), note = conditionMessage(mod)))
728     return(tibble::tibble())
729   }
730   ct <- coeftable_to_tibble(mod) %>% dplyr::filter(term == "exposure_1990_2007")
731   status <- dplyr::bind_rows(status, tibble::tibble(source = source, outcome = outcome, spec = spec_name, status = "ok", n_obs =
732     ↪ stats::nobs(mod), note = NA_character_))
733   tibble::tibble(source = source, outcome = outcome, outcome_label = adhm_outcome_label(outcome), spec = spec_name, spec_label =
734     ↪ extension_spec_label(spec_name), estimate = scalar_or_na(ct$estimate), std.error = scalar_or_na(ct$std.error), statistic =
735     ↪ scalar_or_na(ct$statistic), p.value = scalar_or_na(ct$p.value), n_obs = stats::nobs(mod), controls = if (length(ctrls) == 0)
736     ↪ "none" else paste(ctrls, collapse = "; "))
737   })
738   })
739 }
740 if (file.exists(house_path)) {
741   house <- haven::read_dta(house_path) %>% dplyr::mutate(czone = as.integer(czone), sh_district_2002 =
742     ↪ dplyr::coalesce(safe_num(sh_district_2002), 1))
743   house_outcomes <- c("d2_rwin_2002_2016", "d2_cfavg_repcon_2002_2016", "d2_cfavg_repmod_2002_2016", "d2_cfavg_demmod_2002_2016",
744     ↪ "d2_cfavg_demlib_2002_2016", "d2_teaparty_2002_2010")
745   house_cz <- house %>% dplyr::group_by(czone) %>% dplyr::summarise(dplyr::across(dplyr::all_of(house_outcomes[house_outcomes %in%
746     ↪ names(house)]), ~ weighted_mean_safe(.x, sh_district_2002)), .groups = "drop")
747   regressions <- dplyr::bind_rows(regressions, fit_cross_section(house_cz, house_outcomes, "adhm2020_house_2002_2016"))
748 }
749 if (file.exists(pres_path)) {
750   pres <- haven::read_dta(pres_path) %>% dplyr::mutate(czone = as.integer(czone), totvote_2000pres =
751     ↪ dplyr::coalesce(safe_num(totvote_2000pres), 1))
752   pres_cz <- pres %>%
753     dplyr::group_by(czone) %>%
754     dplyr::summarise(shnr_pres2000 = weighted_mean_safe(shnr_pres2000, totvote_2000pres), shnr_pres2008 =
755       ↪ weighted_mean_safe(shnr_pres2008, totvote_2000pres), shnr_pres2016 = weighted_mean_safe(shnr_pres2016, totvote_2000pres),
756       ↪ .groups = "drop") %>%
757     dplyr::mutate(d_shnr_pres_2000_2016 = shnr_pres2016 - shnr_pres2000, d_shnr_pres_2000_2008 = shnr_pres2008 - shnr_pres2000)
758   regressions <- dplyr::bind_rows(regressions, fit_cross_section(pres_cz, c("d_shnr_pres_2000_2016", "d_shnr_pres_2000_2008"),
759     ↪ "adhm2020_president_2000_2016"))
760 }
761 readr::write_csv(regressions, config$adhm2020_regressions_csv)
762 readr::write_csv(status, config$adhm2020_status_csv)
763 hetero_vars <- c("l_sh_pop_white", "l_sh_popwhit", "sh_pop_white", "l_sh_pop_black", "l_sh_pop_hispanic")
764 avail <- tibble::tibble(candidate_variable = hetero_vars, available_in_current_ADH_extension_data = hetero_vars %in% names(adh), note =
765   ↪ "If a majority-white or race-composition variable is available, add exposure × baseline-race heterogeneity to compare with ADHM
766   ↪ 2020 heterogeneity results.")
767 readr::write_csv(avail, config$adhm2020_heterogeneity_availability_csv)
768 summary <- tibble::tibble(source = c("house_2002_2016", "president_2000_2016"), file_present = c(file.exists(house_path),
769   ↪ file.exists(pres_path)), purpose = c("Political-supply and House ideology outcomes from ADHM public replication data",
770   ↪ "Presidential-vote benchmark outcomes from ADHM public replication data"))
771 readr::write_csv(summary, config$adhm2020_summary_csv)
772 if (nrow(regressions) > 0) {
773   plot_dat <- regressions %>%
774     dplyr::mutate(outcome_label = factor(outcome_label, levels = rev(unique(outcome_label))))
775   p <- ggplot2::ggplot(plot_dat, ggplot2::aes(x = estimate, y = outcome_label, xmin = estimate - 1.96 * std.error, xmax = estimate +
776     ↪ 1.96 * std.error)) +
777     ggplot2::geom_vline(xintercept = 0, linewidth = 0.25) + ggplot2::geom_pointrange(linewidth = 0.25) +
778     ggplot2::facet_grid(source ~ spec_label, scales = "free_y", space = "free_y") +
779     ggplot2::labs(title = "ADHM 2020 political-supply diagnostics", subtitle = "Cross-sectional CZ regressions on ADH exposure; public
780       ↪ ADHM outcomes aggregated to CZs", x = "Coefficient on ADH exposure", y = NULL) +
781     ggplot2::theme_minimal(base_size = 10)
782   write_plot_safely(p, config$adhm2020_plot_png, config$adhm2020_plot_pdf, width = 12, height = 7)
783 }
784 invisible(regressions)
785 }
786 # -----
787 # 6. Subperiod exposure
788 # -----
789 run_subperiod_exposure_event_studies <- function(config = CONFIG) {
790   config <- finalize_config(config)
791   adh_stack_path <- file.path(config$replication_dir, "adh2013", "dta", "workfile_china.dta")
792   if (!file.exists(adh_stack_path) || !file.exists(config$analysis_panel_rds)) return(invisible(NULL))
793   stack <- haven::read_dta(adh_stack_path)
794   exp_period <- stack %>%
795     dplyr::filter(yr %in% c(1990, 2000)) %>%
796     dplyr::transmute(czone = as.integer(czone), period = dplyr::case_when(yr == 1990 ~ "1990_2000", yr == 2000 ~ "2000_2007", TRUE ~
797       ↪ as.character(yr)), exposure = safe_num(d_tradeusch_pw), instrument = safe_num(d_tradeotch_pw_lag)) %>%
798     tidyr::pivot_wider(names_from = period, values_from = c(exposure, instrument), names_sep = "_")
799   readr::write_csv(exp_period, config$subperiod_exposure_csv)
800   readr::write_csv(exp_period %>% dplyr::summarise(dplyr::across(-czone, list(mean = ~mean(.x, na.rm = TRUE), sd = ~stats::sd(.x, na.rm =
801     ↪ TRUE), min = ~min(.x, na.rm = TRUE), max = ~max(.x, na.rm = TRUE))), .names = "{.col}_{.fn}")),
802     ↪ config$subperiod_exposure_summary_csv)
803   adh <- load_adh_extension_data(config)
804   fs <- exp_period %>% dplyr::inner_join(adh %>% dplyr::select(czone, adh_weight, statefip, dplyr::starts_with("z_")), by = "czone")
805   fs_tbl <- tibble::tibble()
806   for (pair in list(c("exposure_1990_2000", "instrument_1990_2000"), c("exposure_2000_2007", "instrument_2000_2007"))) {
807     if (all(pair %in% names(fs))) {
808       for (spec_name in c("minimal", "core_controls", "full_controls")) {
809         ctrls <- if (spec_name == "minimal") character(0) else if (spec_name == "core_controls")
810           ↪ model_control_names(config$core_interacted_controls, TRUE) else model_control_names(config$interacted_controls, TRUE)
811         ctrls <- ctrls[ctrls %in% names(fs)]
812         fml <- stats::as.formula(paste(pair[1], "~", paste(c(pair[2], ctrls), collapse = " + ")))

```

```

793   mod <- tryCatch(fit_weighted_feols(fml, fs, weight_var = "adh_weight", cluster_var = "statefip"), error = function(e) NULL)
794   if (!is.null(mod)) {
795     ct <- coeftable_to_tibble(mod) %>% dplyr::filter(term == pair[2])
796     fs_tbl <- dplyr::bind_rows(fs_tbl, tibble::tibble(exposure_period = gsub("exposure_", "", pair[1]), spec = spec_name,
      ↪ instrument = pair[2], estimate = scalar_or_na(ct$estimate), std.error = scalar_or_na(ct$std.error), statistic =
      ↪ scalar_or_na(ct$statistic), p.value = scalar_or_na(ct$p.value), first_stage_F_t2 = scalar_or_na(ct$statistic)^2, n_obs =
      ↪ stats::nobs(mod)))
797   }
798 }
799 }
800 }
801 readr::write_csv(fs_tbl, config$subperiod_first_stage_csv)
802
803 panel <- readRDS(config$analysis_panel_rds) %>% dplyr::left_join(exp_period, by = "czone")
804 yrs <- config$event_years; ref <- config$reference_year
805 for (ev in c("exposure_1990_2000", "exposure_2000_2007")) for (yr in setdiff(yrs, ref)) panel[[paste0(ev, "_x_", yr)]] <- panel[[ev]] *
      ↪ as.integer(panel$year == yr)
806 specs <- list(minimal = list(controls = character(0), standardize = FALSE), core_controls = list(controls =
      ↪ config$core_interacted_controls, standardize = TRUE), full_controls = list(controls = config$interacted_controls, standardize =
      ↪ TRUE))
807 coef_tbl <- tibble::tibble(); status_tbl <- tibble::tibble()
808 for (spec_name in names(specs)) {
809   spec <- specs[[spec_name]]
810   dat <- panel
811   ctrl_build <- apply_control_interactions(dat, spec$controls, yrs, ref, standardize = spec$standardize)
812   dat <- ctrl_build$data
813   event_terms <- unlist(lapply(c("exposure_1990_2000", "exposure_2000_2007"), function(ev) paste0(ev, "_x_", setdiff(yrs, ref))))
814   fml <- stats::as.formula(paste("rep_margin ~", paste(c(event_terms, ctrl_build$terms), collapse = " + "), "| czone + year"))
815   fit <- tryCatch(fixest::feols(fml, data = dat, weights = ~adh_weight, cluster = ~czone, panel.id = ~czone + election_index, warn =
      ↪ FALSE, notes = FALSE), error = function(e) e)
816   if (inherits(fit, "error")) {
817     status_tbl <- dplyr::bind_rows(status_tbl, tibble::tibble(spec = spec_name, status = "error", error_message =
      ↪ conditionMessage(fit), n_coefficients = 0L))
818   } else {
819     ct <- coeftable_to_tibble(fit) %>%
820     dplyr::filter(term %in% event_terms) %>%
821     dplyr::mutate(
822       spec = spec_name,
823       exposure_period = dplyr::case_when(startsWith(term, "exposure_1990_2000") ~ "1990-2000", startsWith(term, "exposure_2000_2007")
      ↪ ~ "2000-2007", TRUE ~ NA_character_),
824       year = as.integer(stringr::str_extract(term, "[0-9]{4}$")),
825       period_type = dplyr::case_when(year < 1990 ~ "placebo_pre_shock", year <= 2008 ~ "during_or_immediate_post_shock", year <= 2016
      ↪ ~ "medium_run", TRUE ~ "long_run"),
826       reference_year = ref,
827       n_obs = stats::nobs(fit), n_cz = dplyr::n_distinct(dat$czone)
828     ) %>%
829     dplyr::select(spec, exposure_period, period_type, term, year, reference_year, estimate, std.error, statistic, p.value, n_obs,
      ↪ n_cz)
830   coef_tbl <- dplyr::bind_rows(coef_tbl, ct)
831   status_tbl <- dplyr::bind_rows(status_tbl, tibble::tibble(spec = spec_name, status = "ok", error_message = NA_character_,
      ↪ n_coefficients = nrow(ct)))
832 }
833 }
834 readr::write_csv(coef_tbl, config$subperiod_event_study_coefficients_csv)
835 readr::write_csv(status_tbl, config$subperiod_event_study_status_csv)
836 equality <- tibble::tibble()
837 if (nrow(coef_tbl) > 0) {
838   wide <- coef_tbl %>% dplyr::select(spec, year, exposure_period, estimate, std.error) %>% tidyr::pivot_wider(names_from =
      ↪ exposure_period, values_from = c(estimate, std.error), names_sep = "_")
839   if (all(c("estimate_1990-2000", "estimate_2000-2007") %in% names(wide))) {
840     equality <- wide %>% dplyr::mutate(difference_early_minus_late = `estimate_1990-2000` - `estimate_2000-2007`, approx_se_difference =
      ↪ sqrt((`std.error_1990-2000`)^2 + (`std.error_2000-2007`)^2), approx_t_difference = difference_early_minus_late /
      ↪ approx_se_difference, approx_p_difference = 2 * stats::pnorm(-abs(approx_t_difference)), note = "Approximate test ignores
      ↪ covariance between early and late exposure coefficients; use as descriptive diagnostic.")
841   }
842 }
843 readr::write_csv(equality, config$subperiod_equality_tests_csv)
844 selected_horizons <- tibble::tibble()
845 if (nrow(equality) > 0) {
846   selected_horizons <- equality %>%
847   dplyr::filter(year %in% c(1996L, 2000L, 2008L, 2016L, 2020L)) %>%
848   dplyr::transmute(
849     spec, year,
850     early_exposure_coefficient = `estimate_1990-2000`,
851     late_exposure_coefficient = `estimate_2000-2007`,
852     early_minus_late = difference_early_minus_late,
853     approx_se_difference, approx_t_difference, approx_p_difference,
854     interpretation_note = "Approximate early-vs-late comparison; covariance ignored. Use for descriptive horizon interpretation."
855   )
856 }
857 readr::write_csv(selected_horizons, config$subperiod_selected_horizons_csv)
858 if (nrow(coef_tbl) > 0) {
859   p <- ggplot2::ggplot(coef_tbl, ggplot2::aes(x = year, y = estimate, ymin = estimate - 1.96 * std.error, ymax = estimate + 1.96 *
      ↪ std.error, color = exposure_period)) +
860   ggplot2::geom_hline(yintercept = 0, linewidth = 0.25) + ggplot2::geom_pointrange(position = ggplot2::position_dodge(width = 0.6),
      ↪ linewidth = 0.25) +
861   ggplot2::facet_wrap(~spec, ncol = 1, scales = "free_y") +
862   ggplot2::labs(title = "Subperiod ADH exposure event study", subtitle = "1990-2000 and 2000-2007 exposure terms entered jointly", x =
      ↪ NULL, y = "Coefficient", color = "Exposure period") +

```

```

863     ggplot2::theme_minimal(base_size = 11)
864     write_plot_safely(p, config$subperiod_event_study_plot_png, config$subperiod_event_study_plot_pdf, width = 10, height = 8)
865   }
866   invisible(coef_tbl)
867 }
868
869 run_sixth_extensions <- function(config = CONFIG) {
870   config <- extension_primary_config(config)
871   ensure_output_dirs(config)
872   load_required_packages(config)
873   message("Running Sixth-section extension diagnostics for primary design: ", toupper(config$crosswalk_weight_slug), " + ",
874     ↪ config$crosswalk_missing_weight_policy)
875   results <- list()
876   results$bartik <- tryCatch(run_bartik_identification_diagnostics(config), error = function(e) { warning("Bartik diagnostics failed: ",
877     ↪ conditionMessage(e), call. = FALSE); NULL })
878   results$alternative_outcomes <- tryCatch(run_alternative_political_outcome_event_studies(config), error = function(e) {
879     ↪ warning("Alternative political outcomes failed: ", conditionMessage(e), call. = FALSE); NULL })
880   results$nanda <- tryCatch(run_nanda_outcome_event_studies(config), error = function(e) { warning("NaNDA outcomes failed: ",
881     ↪ conditionMessage(e), call. = FALSE); NULL })
882   results$adhm2020 <- tryCatch(run_adhm2020_mechanism_diagnostics(config), error = function(e) { warning("ADHM 2020 diagnostics failed:
883     ↪ ", conditionMessage(e), call. = FALSE); NULL })
884   results$subperiod <- tryCatch(run_subperiod_exposure_event_studies(config), error = function(e) { warning("Subperiod exposure
885     ↪ diagnostics failed: ", conditionMessage(e), call. = FALSE); NULL })
886   checks <- if (file.exists(config$validation_checks_csv)) readr::read_csv(config$validation_checks_csv, show_col_types = FALSE) else
887     ↪ tibble::tibble()
888   write_pipeline_manifest(config = config, checks = checks, stage = "sixth_extensions", sources = list(adh2013 = list(path =
889     ↪ file.path(config$replication_dir, "adh2013")), nanda = list(path = file.path(config$replication_dir, "outcomes-data",
890     ↪ "nanda-2004-2022")), adhm2020 = list(path = file.path(config$replication_dir, "adhm2020")), extra = list(extension_modules =
891     ↪ c("bartik_identification", "alternative_political_outcomes", "nanda_turnout_partisanship", "adhm2020_political_supply",
892     ↪ "subperiod_exposure"), crosswalk_used_for_extensions = paste0(config$crosswalk_weight, " / ",
893     ↪ config$crosswalk_missing_weight_policy)))
894   invisible(results)
895 }

```


8 run_pipeline.R

```
1 # extension-adh2013/src/run_pipeline.R
2
3 if (!requireNamespace("here", quietly = TRUE)) {
4   stop("Package 'here' is required. Install it before running the pipeline.", call. = FALSE)
5 }
6 library(here)
7 here::i_am("src/run_pipeline.R")
8
9 source(here::here("src", "00_config.R"))
10 source(here::here("src", "01_helpers.R"))
11 source(here::here("src", "02_build_analysis_data.R"))
12 source(here::here("src", "03_estimate_event_study.R"))
13 source(here::here("src", "04_export_outputs.R"))
14 source(here::here("src", "05_export_diagnostic_maps.R"))
15 source(here::here("src", "06_sixth_extensions.R"))
16
17 read_bool_env <- function(name, default) {
18   value <- Sys.getenv(name, unset = "")
19   if (!nzchar(value)) return(default)
20   tolower(value) %in% c("1", "true", "t", "yes", "y")
21 }
22
23 run_single_pipeline <- function(config) {
24   config <- finalize_config(config)
25   ensure_output_dirs(config)
26   write_dependency_manifest(config)
27
28   message("Building analysis data...")
29   build_analysis_data(config, stop_on_fatal = TRUE)
30
31   message("Estimating event-study specifications...")
32   estimate_event_study(config, stop_on_fatal = TRUE)
33
34   message("Exporting tables and figures...")
35   export_event_study_outputs(config)
36
37   message("Exporting diagnostic maps and crosswalk comparisons when inputs are available...")
38   export_diagnostic_maps_and_comparisons(config)
39
40   if (isTRUE(config$run_sixth_extensions)) {
41     message("Running Sixth-section extensions...")
42     run_sixth_extensions(config)
43   }
44
45   invisible(config)
46 }
47
48 policy_override <- Sys.getenv("CROSSWALK_MISSING_WEIGHT_POLICY", unset = "")
49 if (nzchar(policy_override)) CONFIG$crosswalk_missing_weight_policy <- policy_override
50
51 weight_override <- Sys.getenv("CROSSWALK_WEIGHT", unset = "")
52 if (nzchar(weight_override)) CONFIG$crosswalk_weight <- weight_override
53
54 ext_weight_override <- Sys.getenv("EXTENSION_CROSSWALK_WEIGHT", unset = "")
55 if (nzchar(ext_weight_override)) CONFIG$extension_crosswalk_weight <- ext_weight_override
56
57 ext_policy_override <- Sys.getenv("EXTENSION_CROSSWALK_MISSING_WEIGHT_POLICY", unset = "")
58 if (nzchar(ext_policy_override)) CONFIG$extension_crosswalk_missing_weight_policy <- ext_policy_override
59
60 CONFIG$renormalize_crosswalk_weights <- read_bool_env(
61   "RENORMALIZE_CROSSWALK_WEIGHTS", CONFIG$renormalize_crosswalk_weights
62 )
63 CONFIG$require_balanced_panel <- read_bool_env(
64   "REQUIRE_BALANCED_PANEL", CONFIG$require_balanced_panel
65 )
66 CONFIG$require_no_vcov_repair_for_main <- read_bool_env(
67   "REQUIRE_NO_VCOV_REPAIR_FOR_MAIN", CONFIG$require_no_vcov_repair_for_main
68 )
69 CONFIG$allow_vcov_repair <- read_bool_env("ALLOW_VCOV_REPAIR", CONFIG$allow_vcov_repair)
70 CONFIG$allow_failed_diagnostics <- read_bool_env(
71   "ALLOW_FAILED_DIAGNOSTICS", CONFIG$allow_failed_diagnostics
72 )
73 CONFIG$standardize_interacted_controls <- read_bool_env(
74   "STANDARDIZE_INTERACTED_CONTROLS", CONFIG$standardize_interacted_controls
75 )
76 CONFIG$export_crosswalk_maps <- read_bool_env(
77   "EXPORT_CROSSWALK_MAPS", CONFIG$export_crosswalk_maps
78 )
79 CONFIG$run_crosswalk_sensitivity <- read_bool_env(
80   "RUN_CROSSWALK_SENSITIVITY", CONFIG$run_crosswalk_sensitivity
81 )
82 CONFIG$run_sixth_extensions <- read_bool_env(
83   "RUN_SIXTH_EXTENSIONS", CONFIG$run_sixth_extensions
84 )
85 CONFIG$save_single_rds <- read_bool_env(
86   "SAVE_SINGLE_RDS", CONFIG$save_single_rds
87 )
```

```
88 |  
89 | if (isTRUE(CONFIG$run_crosswalk_sensitivity)) {  
90 |   message("Running the four-specification crosswalk sensitivity pipeline...")  
91 |   run_crosswalk_sensitivity_pipeline(CONFIG)  
92 | } else {  
93 |   run_single_pipeline(CONFIG)  
94 | }  
95 |  
96 | message("Pipeline complete.")
```

9 county_bridge_exceptions.csv

```
1 year,county_fips,county_name,pattern,override_source_decade,override_county_fips_source,issue_class,reason,apply
2 1984,04012,La Paz,1990,04012,county_boundary_change_needs_bridge,"La Paz County, AZ was created after 1980 but exists in 1990 target
   ↳ geography; use 1990 identity mapping for 1984/1988 presidential returns.",TRUE
3 1988,04012,La Paz,1990,04012,county_boundary_change_needs_bridge,"La Paz County, AZ was created after 1980 but exists in 1990 target
   ↳ geography; use 1990 identity mapping for 1984/1988 presidential returns.",TRUE
4 1984,35006,Cibola,1990,35006,county_boundary_change_needs_bridge,"Cibola County, NM was created after 1980 but exists in 1990 target
   ↳ geography; use 1990 identity mapping for 1984/1988 presidential returns.",TRUE
5 1988,35006,Cibola,1990,35006,county_boundary_change_needs_bridge,"Cibola County, NM was created after 1980 but exists in 1990 target
   ↳ geography; use 1990 identity mapping for 1984/1988 presidential returns.",TRUE
6 2016,46102,Oglala Lakota,2010,46113,county_boundary_change_needs_bridge,"Oglala Lakota County, SD is the renamed Shannon County; map 2016
   ↳ returns to the 2010/1990 Shannon FIPS where needed.",TRUE
7 2020,46102,Oglala Lakota,2010,46113,county_boundary_change_needs_bridge,"Oglala Lakota County, SD is the renamed Shannon County; map 2020
   ↳ returns to the 2010/1990 Shannon FIPS where needed.",FALSE
8 2004,08014,Broomfield,2010,08014,county_boundary_change_needs_bridge,"Broomfield County, CO was created after the 2000 Census; use the
   ↳ 2010 source crosswalk for 2004 returns so its vote mass is allocated to 1990 counties.",TRUE
9 2008,08014,Broomfield,2010,08014,county_boundary_change_needs_bridge,"Broomfield County, CO was created after the 2000 Census; use the
   ↳ 2010 source crosswalk for 2008 returns so its vote mass is allocated to 1990 counties.",TRUE
10 1976,51683,Manassas,1980,51683,county_boundary_change_needs_bridge,"Manassas city appears in 1976 returns but not the 1970 source
   ↳ crosswalk used by the decade rule; use the 1980 source crosswalk.",TRUE
11 1976,51685,Manassas Park,1980,51685,county_boundary_change_needs_bridge,"Manassas Park city appears in 1976 returns but not the 1970
   ↳ source crosswalk used by the decade rule; use the 1980 source crosswalk.",TRUE
12 1976,51735,Poquoson,1980,51735,county_boundary_change_needs_bridge,"Poquoson city appears in 1976 returns but not the 1970 source
   ↳ crosswalk used by the decade rule; use the 1980 source crosswalk.",TRUE
```